

■ **TABLA 12.4** Condiciones necesarias y suficientes para la optimalidad

Problema	Condiciones necesarias para la optimalidad	También suficientes si:
Una variable, no restringido	$\frac{df}{dx} = 0$	$f(x)$ cóncava
Multivariantes, no restringido	$\frac{\partial f}{\partial x_j} = 0 \quad (j = 1, 2, \dots, n)$	$f(\mathbf{x})$ cóncava
Restringido, sólo restricciones de no negatividad	$\frac{\partial f}{\partial x_j} = 0 \quad (j = 1, 2, \dots, n)$ (o ≤ 0 si $x_j = 0$)	$f(\mathbf{x})$ cóncava
Problema general restringido	Condiciones de Karush-Kuhn-Tucker	$f(\mathbf{x})$ cóncava y $g_i(\mathbf{x})$ convexa ($i = 1, 2, \dots, m$)

muestran estas condiciones. Como se indica en el último renglón de la tabla, las condiciones para el caso general se llaman **condiciones de Karush-Kuhn-Tucker** (o **condiciones KKT**), porque fueron desarrolladas de manera independiente por Karush¹¹ y por Kuhn y Tucker.¹² Su resultado básico se expresa en el siguiente teorema.

Teorema. Suponga que $f(\mathbf{x})$, $g_1(\mathbf{x})$, $g_2(\mathbf{x})$, $\dots$, $g_m(\mathbf{x})$ son funciones *diferenciables* que satisfacen ciertas condiciones de regularidad.¹³ Entonces,

$$\mathbf{x}^* = (x_1^*, x_2^*, \dots, x_n^*)$$

puede ser una *solución óptima* para el problema de programación no lineal, sólo si existen m números $u_1, u_2, \dots, u_m$, que satisfagan *todas* las siguientes *condiciones KKT*:

$$\left. \begin{array}{l} 1. \quad \frac{\partial f}{\partial x_j} - \sum_{i=1}^m u_i \frac{\partial g_i}{\partial x_j} \leq 0 \\ 2. \quad x_j^* \left(\frac{\partial f}{\partial x_j} - \sum_{i=1}^m u_i \frac{\partial g_i}{\partial x_j} \right) = 0 \end{array} \right\} \quad \text{en } \mathbf{x} = \mathbf{x}^*, \text{ para } j = 1, 2, \dots, n.$$

$$\left. \begin{array}{l} 3. \quad g_i(\mathbf{x}^*) - b_i \leq 0 \\ 4. \quad u_i [g_i(\mathbf{x}^*) - b_i] = 0 \end{array} \right\} \quad \text{para } i = 1, 2, \dots, m.$$

$$\begin{array}{ll} 5. \quad x_j^* \geq 0, & \text{para } j = 1, 2, \dots, n. \\ 6. \quad u_i \geq 0, & \text{para } i = 1, 2, \dots, m. \end{array}$$

Observe que las condiciones 2 y 4 requieren que el producto de dos cantidades sea 0. Por tanto, cada una de estas condiciones en realidad dice que al menos una de las dos cantidades debe ser cero. En consecuencia, la condición 4 se puede combinar con la condición 3 para expresarlas en otra forma equivalente, tal como

$$(3, 4) \quad g_i(\mathbf{x}^*) - b_i = 0 \quad (o \leq 0 \text{ si } u_i = 0), \quad \text{para } i = 1, 2, \dots, m.$$

De manera similar, la condición 2 se puede combinar con la 1 de la siguiente manera:

$$(1, 2) \quad \frac{\partial f}{\partial x_j} - \sum_{i=1}^m u_i \frac{\partial g_i}{\partial x_j} = 0 \quad (o \leq 0 \text{ si } x_j^* = 0), \quad \text{para } j = 1, 2, \dots, n.$$

¹¹ W. Karush, "Minima of Functions of Several Variables with Inequalities as Side Conditions", M. S. thesis, Department of Mathematics, University of Chicago, 1939.

¹² H. W. Kuhn y A. W. Tucker: "Nonlinear Programming", en Jerzy Newman (ed.), *Proceedings of the Second Berkeley Symposium*, University of California Press, Berkeley, 1951, pp. 481-492.

¹³ *Ibid.*, p. 483.

Cuando $m = 0$ (sin restricciones funcionales), esta suma se elimina y la condición combinada (1, 2) se reduce a la condición que se presenta en el tercer renglón de la tabla 12.4. Así, para $m > 0$, cada término de la suma modifica la condición de $m = 0$ para incorporar el efecto de la restricción funcional correspondiente.

En las condiciones 1, 2, 4 y 6, las u_i corresponden a las *variables duales* de programación lineal —al final de la sección se estudia más a fondo esta correspondencia— que tienen una interpretación económica comparable. En realidad, las u_i surgieron de la derivación matemática, como los *multiplicadores de Lagrange* (que se presentaron en el apéndice 3). Las condiciones 3 y 5 sólo ayudan a asegurar la factibilidad de la solución. Las otras condiciones eliminan la mayor parte de las soluciones factibles como posibles candidatos para una solución óptima.

Debe hacerse notar que el cumplimiento de estas condiciones no garantiza que la solución sea óptima. Como se resume en la última columna de la tabla 12.4, son necesarios ciertos supuestos de *convexidad* adicionales para obtener esta garantía. Estos supuestos se establecen de manera formal en la siguiente extensión del teorema.

Corolario. Suponga que $f(\mathbf{x})$ es una función *cóncava* y que $g_1(\mathbf{x}), g_2(\mathbf{x}), \dots, g_m(\mathbf{x})$ son funciones *convexas* —es decir, se trata de un problema de programación convexa—, en donde todas estas funciones satisfacen las condiciones de regularidad. Entonces, $\mathbf{x}^* = (x_1^*, x_2^*, \dots, x_n^*)$ es una *solución óptima* si y sólo si se satisfacen todas las condiciones del teorema.

Ejemplo. Para ilustrar esta formulación y la aplicación de las *condiciones KKT*, considere el siguiente problema de programación no lineal con dos variables:

$$\text{Maximizar} \quad f(\mathbf{x}) = \ln(x_1 + 1) + x_2,$$

sujeta a

$$2x_1 + x_2 \leq 3$$

y

$$x_1 \geq 0, \quad x_2 \geq 0,$$

donde $\ln$ denota el logaritmo natural. De esta forma, $m = 1$ (una restricción funcional) y $g_1(\mathbf{x}) = 2x_1 + x_2$, de manera que $g_1(\mathbf{x})$ es convexa. Aún más, es fácil verificar (vea el apéndice 2) que $f(\mathbf{x})$ es cóncava. Entonces se puede aplicar el corolario, por lo que cualquier solución que satisfaga las condiciones KKT será en definitiva una solución óptima. Al aplicar las fórmulas en el teorema, se obtienen las siguientes condiciones KKT para este ejemplo:

$$1(j = 1). \quad \frac{1}{x_1 + 1} - 2u_1 \leq 0.$$

$$2(j = 1). \quad x_1 \left(\frac{1}{x_1 + 1} - 2u_1 \right) = 0.$$

$$1(j = 2). \quad 1 - u_1 \leq 0.$$

$$2(j = 2). \quad x_2(1 - u_1) = 0.$$

$$3. \quad 2x_1 + x_2 - 3 \leq 0.$$

$$4. \quad u_1(2x_1 + x_2 - 3) = 0.$$

$$5. \quad x_1 \geq 0, \quad x_2 \geq 0.$$

$$6. \quad u_1 \geq 0.$$

A continuación se describen los pasos para resolver las condiciones KKT para este ejemplo.

1. $u_1 \geq 1$, de la condición 1($j = 2$).
 $x_1 \geq 0$, por la condición 5.
2. Por lo tanto, $\frac{1}{x_1 + 1} - 2u_1 < 0$.
3. Por lo tanto, $x_1 = 0$, de la condición 2($j = 1$).
4. $u_1 \neq 0$ implica que $2x_1 + x_2 - 3 = 0$, de la condición 4.
5. Los pasos 3 y 4 implican que $x_2 = 3$.

6. $x_2 \neq 0$ implica que $u_1 = 1$, de la condición 2($j = 2$).
7. Los valores $x_1 = 0$, $x_2 = 3$ y $u_1 = 1$ no violan ninguna condición.

Por lo tanto, existe un número $u_1 = 1$ tal que $x_1 = 0$, $x_2 = 3$ y $u_1 = 1$ satisfacen todas las condiciones. En consecuencia, $\mathbf{x}^* = (0, 3)$ es una solución óptima para este problema.

Éste, en particular, fue un problema relativamente fácil de resolver porque los primeros dos pasos condujeron con rapidez a las conclusiones restantes. Con frecuencia resulta más difícil encontrar la forma de comenzar. La sucesión particular de pasos que se necesita para resolver las condiciones KKT difiere de un problema a otro. Cuando la lógica no es evidente, a veces es útil considerar por separado los distintos casos en donde cada x_j y u_i se especifican como iguales o mayores que 0, y después se prueba cada caso hasta que uno conduce a una solución.

Para ilustrar, suponga que este enfoque de considerar los diferentes casos de manera separada se ha aplicado al ejemplo previo en lugar de emplear la lógica implicada en los siete pasos anteriores. Para este ejemplo, hay ocho casos que corresponden a las ocho combinaciones de $x_1 = 0$ contra $x_1 > 0$, $x_2 = 0$ contra $x_2 > 0$, o $u_1 = 0$ contra $u_1 > 0$. Cada caso conduce a una afirmación y análisis más sencillos de las condiciones. Como ejemplo, considere primero el caso que se expone en seguida, en donde $x_1 = 0$, $x_2 = 0$ y $u_1 = 0$.

Condiciones KKT para el caso $x_1 = 0$, $x_2 = 0$, $u_1 = 0$

$$\begin{aligned} 1(j = 1). \quad & \frac{1}{0 + 1} \leq 0. && \text{Contradicción.} \\ 1(j = 2). \quad & 1 - 0 \leq 0. && \text{Contradicción.} \\ 3. \quad & 0 + 0 \leq 3. \end{aligned}$$

(Todas las demás condiciones son redundantes.)

Como se enumera después, los otros tres casos donde $u_1 = 0$ también llevan de inmediato a contradicciones en forma muy parecida, por lo cual no se dispone de soluciones.

Caso $x_1 = 0$, $x_2 > 0$, $u_1 = 0$ contradice las condiciones 1($j = 1$), 1($j = 2$) y 2($j = 2$).

Caso $x_1 > 0$, $x_2 = 0$, $u_1 = 0$ contradice las condiciones 1($j = 1$), 2($j = 1$) y 1($j = 2$).

Caso $x_1 > 0$, $x_2 > 0$, $u_1 = 0$ contradice las condiciones 1($j = 1$), 2($j = 1$), 1($j = 2$) y 2($j = 2$).

El caso $x_1 > 0$, $x_2 > 0$, $u_1 > 0$ permite eliminar estos tres multiplicadores diferentes de cero de las condiciones 2($j = 1$), 2($j = 2$) y 4, lo que a su vez permite eliminar las condiciones 1($j = 1$), 1($j = 2$) y 3 como redundantes, tal como se resume en seguida.

Condiciones KKT para el caso $x_1 > 0$, $x_2 > 0$, $u_1 > 0$

$$\begin{aligned} 1(j = 1). \quad & \frac{1}{x_1 + 1} - 2u_1 = 0. \\ 2(j = 2). \quad & 1 - u_1 = 0. \\ 4. \quad & 2x_1 + x_2 - 3 = 0. \end{aligned}$$

(Todas las demás condiciones son redundantes.)

Entonces, $u_1 = 1$, así $x_1 = -\frac{1}{2}$, lo que contradice $x_1 > 0$.

Ahora suponga que se intenta el caso $x_1 = 0$, $x_2 > 0$, $u_1 > 0$.

Condiciones KKT para el caso $x_1 = 0$, $x_2 > 0$, $u_1 > 0$

$$\begin{aligned} 1(j = 1). \quad & \frac{1}{0 + 1} - 2u_1 = 0. \\ 2(j = 2). \quad & 1 - u_1 = 0. \\ 4. \quad & 0 + x_2 - 3 = 0. \end{aligned}$$

(Todas las demás condiciones son redundantes.)

Por lo tanto, $x_1 = 0$, $x_2 = 3$, $u_1 = 1$. Una vez que se ha encontrado una solución, se sabe que no es necesario considerar casos adicionales.

Si desea ver **otro ejemplo** de las condiciones KKT para encontrar una solución óptima, en la sección Worked Examples del CD-ROM se proporciona uno más.

Para problemas más complicados que éste puede ser difícil, si no es que materialmente imposible, derivar una solución óptima *directa* de las condiciones KKT. De todas maneras, estas condiciones proporcionan información valiosa en cuanto a la identidad de una solución óptima y también permiten verificar que una solución propuesta pueda ser óptima.

Existen muchas aplicaciones *indirectas* valiosas de las condiciones KKT. Una de ellas surge de la *teoría de dualidad* que se desarrolló para programación no lineal, que se puede poner en paralelo con la teoría de dualidad para programación lineal que se presentó en el capítulo 6. En particular para cualquier problema de maximización restringida (llamado *problema primal*), se pueden usar las condiciones KKT para definir un problema dual asociado, que es un problema de minimización restringido. Las variables del problema dual consisten en multiplicadores de Lagrange, u_i ($i = 1, 2, \dots, m$) y las variables primales x_j ($j = 1, 2, \dots, n$). En el caso especial de que el problema primal es un problema de programación lineal, las variables x_j desaparecerán del problema dual y quedará el ya conocido problema dual de programación lineal, en el que las variables u_i corresponden a las variables y_i del capítulo 6. Cuando un problema primal es un problema de programación convexa, es posible establecer relaciones entre el problema primal y el problema dual, que son muy parecidas a las de programación lineal. Por ejemplo, la *propiedad de dualidad fuerte* de la sección 6.1, que establece que los valores óptimos de las funciones objetivo son iguales, también se cumple aquí. Más aún, los valores de las variables u_i en una solución óptima para el problema dual, de nuevo se pueden interpretar como los *precios sombra* (vea las secciones 4.7 y 6.2); es decir, proporcionan la tasa a la que se puede aumentar el valor óptimo de la función objetivo del problema primal, si se aumenta (ligeramente) el lado derecho de la restricción correspondiente. Como la teoría de dualidad para programación no lineal es un tema relativamente avanzado, se pide al lector interesado que consulte otros libros para obtener más información.¹⁴

En la sección siguiente se verá otra aplicación indirecta de las condiciones KKT.

12.7 PROGRAMACIÓN CUADRÁTICA

Como se indicó en la sección 12.3, el problema de programación cuadrática difiere del problema de programación lineal nada más en que la función objetivo incluye también términos x_j^2 y $x_i x_j$ ($i \neq j$). En consecuencia, si se usa la notación matricial como la que se introdujo al inicio de la sección 5.2, el problema es encontrar $\mathbf{x}$ para

$$\text{Maximizar} \quad f(\mathbf{x}) = \mathbf{c}\mathbf{x} - \frac{1}{2}\mathbf{x}^T\mathbf{Q}\mathbf{x},$$

sujeta a

$$\mathbf{A}\mathbf{x} \leq \mathbf{b} \quad \text{y} \quad \mathbf{x} \geq \mathbf{0},$$

donde $\mathbf{c}$ es un vector renglón y $\mathbf{x}$ y $\mathbf{b}$ son vectores columna. $\mathbf{Q}$ y $\mathbf{A}$ son matrices y el superíndice T denota la transpuesta (vea el apéndice 4). Las q_{ij} (elementos de $\mathbf{Q}$) son constantes dadas tales que $q_{ij} = q_{ji}$ (que es la razón para que aparezca el factor de $\frac{1}{2}$ en la función objetivo). Al realizar las multiplicaciones entre vectores y matrices indicadas, la función objetivo se expresa en términos de estas q_{ij} , las c_j (elementos de $\mathbf{c}$) y las variables, de la siguiente manera:

$$f(\mathbf{x}) = \mathbf{c}\mathbf{x} - \frac{1}{2}\mathbf{x}^T\mathbf{Q}\mathbf{x} = \sum_{j=1}^n c_j x_j - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n q_{ij} x_i x_j.$$

Para cada término donde $i = j$ en esta doble suma, $x_i x_j = x_j^2$, así $-\frac{1}{2}q_{jj}$ es el coeficiente de x_j^2 . Cuando $i \neq j$, entonces $-\frac{1}{2}(q_{ij}x_i x_j + q_{ji}x_j x_i) = -q_{ij}x_i x_j$, de manera que $-q_{ij}$ es el coeficiente total del producto de x_i y x_j .

Para ilustrar esta notación, considere el siguiente ejemplo de un problema de programación cuadrática.

¹⁴ Una versión unificada de varios enfoques sobre la dualidad en programación no lineal se encuentra en A. M. Geoffrion, "Duality in Nonlinear Programming: A Simplified Applications-Oriented Development", en *SIAM Review*, **13**: 1-37, 1971.

Maximizar $f(x_1, x_2) = 15x_1 + 30x_2 + 4x_1x_2 - 2x_1^2 - 4x_2^2$,
 sujeta a

$$x_1 + 2x_2 \leq 30$$

y

$$x_1 \geq 0, \quad x_2 \geq 0.$$

En este caso,

$$\mathbf{c} = [15 \quad 30], \quad \mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}, \quad \mathbf{Q} = \begin{bmatrix} 4 & -4 \\ -4 & 8 \end{bmatrix},$$

$$\mathbf{A} = [1 \quad 2], \quad \mathbf{b} = [30].$$

Observe que

$$\begin{aligned} \mathbf{x}^T \mathbf{Q} \mathbf{x} &= [x_1 \quad x_2] \begin{bmatrix} 4 & -4 \\ -4 & 8 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \\ &= [(4x_1 - 4x_2) \quad (-4x_1 + 8x_2)] \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \\ &= 4x_1^2 - 4x_2x_1 - 4x_1x_2 + 8x_2^2 \\ &= q_{11}x_1^2 + q_{21}x_2x_1 + q_{12}x_1x_2 + q_{22}x_2^2. \end{aligned}$$

Al multiplicar por $-\frac{1}{2}$ se obtiene

$$-\frac{1}{2} \mathbf{x}^T \mathbf{Q} \mathbf{x} = -2x_1^2 + 4x_1x_2 - 4x_2^2,$$

que es la porción no lineal de la función objetivo de este ejemplo. Como $q_{11} = 4$ y $q_{22} = 8$, el ejemplo muestra que $-\frac{1}{2}q_{jj}$ es el coeficiente de x_j^2 de la función objetivo. El hecho de que $q_{12} = q_{21} = -4$ ilustra que (ambos) $-q_{ij}$ y $-q_{ji}$ proporcionan el coeficiente total del producto de x_i y x_j .

Se dispone de varios algoritmos para el caso especial de un problema de programación cuadrática donde la función objetivo es una función *cóncava*. (Una manera de comprobar que la función objetivo sea cóncava es verificar la condición equivalente de que

$$\mathbf{x}^T \mathbf{Q} \mathbf{x} \geq 0$$

para toda $\mathbf{x}$, es decir, que $\mathbf{Q}$ es una matriz *positiva semidefinida*.) Se describirá uno¹⁵ de estos algoritmos, el *método simplex modificado*, que es muy aceptado porque se basa en el método simplex con una pequeña modificación. La clave de este enfoque es construir las condiciones KKT de la sección anterior y después reexpresarlas de una manera conveniente que se parezca a programación lineal. Por esto, antes de describir el algoritmo, se desarrollará esta práctica forma.

Condiciones KKT para programación cuadrática

Para concretar, primero considere el ejemplo anterior. A partir de la forma que se presentó en la sección anterior, sus condiciones KKT son las siguientes.

- 1($j = 1$). $15 + 4x_2 - 4x_1 - u_1 \leq 0$.
- 2($j = 1$). $x_1(15 + 4x_2 - 4x_1 - u_1) = 0$.
- 1($j = 2$). $30 + 4x_1 - 8x_2 - 2u_1 \leq 0$.
- 2($j = 2$). $x_2(30 + 4x_1 - 8x_2 - 2u_1) = 0$.
3. $x_1 + 2x_2 - 30 \leq 0$.
4. $u_1(x_1 + 2x_2 - 30) = 0$.
5. $x_1 \geq 0, \quad x_2 \geq 0$.
6. $u_1 \geq 0$.

¹⁵ P. Wolfe, "The Simplex Method for Quadratic Programming", en *Econometrics*, 27: 382-398, 1959. Este artículo desarrolla tanto una forma corta como una larga del algoritmo. Aquí se presenta una versión de la *forma corta* que tiene el supuesto adicional de que o bien $\mathbf{c} = \mathbf{0}$ o bien la función objetivo es estrictamente cóncava.

Para poder reexpresar estas condiciones en una forma más conveniente, se mueven las constantes de las condiciones 1 ($j = 1$), 1 ($j = 2$) y 3 al lado derecho y después se introducen *variables de holgura* no negativas (denotadas por y_1, y_2 y v_1 , respectivamente) para convertir estas desigualdades en ecuaciones.

$$\begin{array}{llll} 1(j = 1). & -4x_1 + 4x_2 - u_1 + y_1 & & = -15 \\ 1(j = 2). & 4x_1 - 8x_2 - 2u_1 & + y_2 & = -30 \\ 3. & x_1 + 2x_2 & & + v_1 = 30 \end{array}$$

Observe que la condición 2 ($j = 1$) se puede reexpresar ahora en forma simple como la necesidad de que se cumpla una de las dos, $x_1 = 0$ o $y_1 = 0$; esto es,

$$2(j = 1). \quad x_1 y_1 = 0.$$

Exactamente en la misma forma, las condiciones 2 ($j = 2$) y 4 se pueden sustituir por

$$\begin{array}{ll} 2(j = 2). & x_2 y_2 = 0, \\ 4. & u_1 v_1 = 0. \end{array}$$

Para cada uno de estos tres pares $-(x_1, y_1), (x_2, y_2), (u_1, v_1)$ —las dos variables se llaman **variables complementarias**, porque sólo una de las dos puede ser diferente de cero. Estas nuevas formas de las condiciones 2 ($j = 1$), 2 ($j = 2$) y 4 se pueden combinar en una restricción,

$$x_1 y_1 + x_2 y_2 + u_1 v_1 = 0,$$

llamada **restricción de complementariedad**.

Después de multiplicar las ecuaciones de las condiciones 1 ($j = 1$) y 1 ($j = 2$) por -1 para obtener lados derechos no negativos, se obtiene la forma deseada del conjunto completo de condiciones:

$$\begin{array}{llllll} 4x_1 - 4x_2 + u_1 - y_1 & & & & & = 15 \\ -4x_1 + 8x_2 + 2u_1 & - y_2 & & & & = 30 \\ x_1 + 2x_2 & & & + v_1 & & = 30 \\ x_1 \geq 0, & x_2 \geq 0, & u_1 \geq 0, & y_1 \geq 0, & y_2 \geq 0, & v_1 \geq 0 \\ x_1 y_1 + x_2 y_2 + u_1 v_1 & & & & & = 0 \end{array}$$

Esta forma es en especial conveniente puesto que excepto por la restricción de complementariedad, estas *condiciones son restricciones de programación lineal*.

En cualquier problema de programación cuadrática, sus condiciones KKT se pueden reducir a la misma forma conveniente que contiene sólo restricciones de programación lineal y una restricción de complementariedad. En notación matricial, esta forma general es

$$\begin{array}{l} \mathbf{Q}\mathbf{x} + \mathbf{A}^T\mathbf{u} - \mathbf{y} = \mathbf{c}^T, \\ \mathbf{A}\mathbf{x} + \mathbf{v} = \mathbf{b}, \\ \mathbf{x} \geq \mathbf{0}, \quad \mathbf{u} \geq \mathbf{0}, \quad \mathbf{y} \geq \mathbf{0}, \quad \mathbf{v} \geq \mathbf{0}, \\ \mathbf{x}^T\mathbf{y} + \mathbf{u}^T\mathbf{v} = 0, \end{array}$$

donde los elementos del vector columna $\mathbf{u}$ son las u_i de la sección anterior y los elementos de los vectores columna $\mathbf{y}$ y $\mathbf{v}$ son variables de holgura.

Como se supone que la función objetivo del problema original es cóncava y como las funciones de restricción son lineales y, por tanto, convexas, se puede aplicar el corolario del teorema de la sección 12.6. Así, $\mathbf{x}$ es *óptima* si y sólo si existen valores de $\mathbf{y}$, $\mathbf{u}$ y $\mathbf{v}$ tales que los cuatro vectores juntos satisfacen todas estas condiciones. Así pues, el problema original se reduce al problema equivalente de encontrar una *solución factible* para estas *restricciones*.

Es interesante observar que este problema equivalente es un ejemplo del problema de *complementariedad lineal* que se introdujo en la sección 12.3 (vea el problema 12.3-6), y que una restricción clave de este problema de complementariedad lineal es su *restricción de complementariedad*.

Método simplex modificado

El *método simplex modificado* explota el importante hecho de que, con excepción de la restricción de complementariedad, las condiciones KKT expresadas en la forma conveniente que se acaba de

obtener, no son otra cosa que restricciones de programación lineal. Aún más, la restricción de complementariedad simplemente implica que no es permisible que las *dos* variables complementarias de un par dado sean básicas —no degeneradas, las únicas variables > 0 — al considerar soluciones BF (no degeneradas). Por tanto, el problema se reduce a encontrar una solución inicial BF para cualquier problema de programación lineal con estas restricciones, sujeta a la restricción adicional sobre la identidad de las variables básicas. (Esta solución inicial básica factible puede ser la única solución factible en este caso.)

Como se analizó en la sección 4.6, tal solución BF inicial se encuentra casi de modo directo. En el sencillo caso de que $\mathbf{c}^T \leq \mathbf{0}$ (poco probable) y $\mathbf{b} \geq \mathbf{0}$, las variables básicas iniciales son los elementos de $\mathbf{y}$ y $\mathbf{v}$ —se multiplica por -1 todo el primer conjunto de ecuaciones—, de manera que la solución deseada es $\mathbf{x} = \mathbf{0}$, $\mathbf{u} = \mathbf{0}$, $\mathbf{y} = -\mathbf{c}^T$, $\mathbf{v} = \mathbf{b}$. De otra manera, es necesario revisar el problema e introducir una *variable artificial* en cada una de las ecuaciones donde $c_j > 0$ —se agrega la variable de la izquierda— o $b_i < 0$ —se resta la variable de la izquierda y después se multiplican los dos lados por -1 —, con el fin de usar estas variables artificiales —que se denotan por z_1, z_2 , etc.— como variables básicas iniciales del problema revisado. (Observe que esta selección de variables básicas iniciales satisface la restricción de complementariedad, puesto que como variables no básicas $\mathbf{x} = \mathbf{0}$ y $\mathbf{u} = \mathbf{0}$, de manera automática.)

Después se utiliza la fase 1 del *método de dos fases* (vea la sección 4.6) para encontrar una solución BF para el problema real; es decir, se aplica el método símplex (con una modificación) al siguiente problema de programación lineal

$$\text{Minimizar} \quad Z = \sum_j z_j,$$

sujeta a las restricciones anteriores de programación lineal que se obtuvieron de las condiciones KKT, pero con estas variables artificiales incluidas.

La modificación del método símplex es el siguiente cambio en el procedimiento para elegir una variable básica entrante.

Regla de entrada restringida: cuando se elige la variable básica entrante, se excluye de las posibilidades cualquier variable no básica cuya *variable complementaria* sea básica; la elección debe hacerse entre las otras variables no básicas, según el criterio normal del método símplex.

Esta regla conserva satisfecha la restricción de complementariedad en todo el curso del algoritmo. Cuando se obtiene una solución óptima

$$\mathbf{x}^*, \mathbf{u}^*, \mathbf{y}^*, \mathbf{v}^*, z_1 = 0, \dots, z_n = 0$$

para la fase 1 de este problema, $\mathbf{x}^*$ es la solución óptima deseada para el problema original de programación cuadrática. La fase 2 del método de dos fases no se necesita.

Ejemplo. Se ilustrará este enfoque con el problema que se presentó al principio de la sección. Como se puede verificar con los resultados del apéndice 2 [vea el problema 12.7-1a], $f(x_1, x_2)$ es *estrictamente cóncava*, es decir,

$$\mathbf{Q} = \begin{bmatrix} 4 & -4 \\ -4 & 8 \end{bmatrix}$$

es positiva definida, por lo que se puede aplicar el algoritmo.

El punto de partida para resolver este ejemplo son las condiciones KKT en la forma conveniente que se obtuvo con anterioridad. Una vez que se introducen las variables artificiales necesarias, el problema de programación lineal que se resolverá en forma explícita por el método símplex modificado es

$$\text{Minimizar} \quad Z = z_1 + z_2,$$

sujeta a

$$\begin{array}{rclclcl} 4x_1 - 4x_2 + u_1 - y_1 & & + z_1 & = & 15 \\ -4x_1 + 8x_2 + 2u_1 & - y_2 & + z_2 & = & 30 \\ x_1 + 2x_2 & & + v_1 & = & 30 \end{array}$$

y

$$\begin{aligned} x_1 \geq 0, \quad x_2 \geq 0, \quad u_1 \geq 0, \quad y_1 \geq 0, \quad y_2 \geq 0, \quad v_1 \geq 0, \\ z_1 \geq 0, \quad z_2 \geq 0. \end{aligned}$$

La restricción de complementariedad adicional

$$x_1 y_1 + x_2 y_2 + u_1 v_1 = 0,$$

no se incluye de manera explícita, porque el algoritmo refuerza esta restricción de manera automática con la *regla de entrada restringida*. En particular, en cada uno de los tres pares de variables complementarias $-(x_1, y_1)$, (x_2, y_2) , (u_1, v_1) siempre que una de las dos variables sea ya una variable básica, la otra se excluye de las candidatas como variable básica entrante. Recuerde que sólo las variables *diferentes de cero* son variables básicas. Dado que el conjunto inicial de variables básicas del problema de programación lineal $-z_1, z_2, v_1$ proporciona una solución BF inicial que satisface la restricción de complementariedad, no hay manera de violar esta restricción con ninguna de las soluciones BF subsecuentes.

En la tabla 12.5 se muestran los resultados de la aplicación del método símplex modificado a este problema. En la primera tabla símplex se presenta el sistema de ecuaciones inicial *después* de convertir de minimización de Z a maximización de $-Z$ y de eliminar algebraicamente las variables básicas iniciales de la ecuación (0), lo mismo que se hizo en el ejemplo de terapia de radiación en la sección 4.6. Las tres iteraciones proceden igual que en el método símplex normal, *excepto* por la eliminación de ciertos candidatos para la variable básica entrante, según la regla de entrada restringida. En la primera tabla se elimina u_1 como candidato porque su variable complementaria (v_1) es ya una variable básica (de todas maneras, habría entrado x_2 puesto que $-4 < -3$). En la segunda

■ **TABLA 12.5** Aplicación del método símplex modificado al ejemplo de programación cuadrática

Iteración	Variable básica	Ec.	Z	x_1	x_2	u_1	y_1	y_2	v_1	z_1	z_2	Lado derecho
0	Z	(0)	-1	0	-4	-3	1	1	0	0	0	-45
	z_1	(1)	0	4	-4	1	-1	0	0	1	0	15
	z_2	(2)	0	-4	8	2	0	-1	0	0	1	30
	v_1	(3)	0	1	2	0	0	0	1	0	0	30
1	Z	(0)	-1	-2	0	-2	1	$\frac{1}{2}$	0	0	$\frac{1}{2}$	-30
	z_1	(1)	0	2	0	2	-1	$-\frac{1}{2}$	0	1	$\frac{1}{2}$	30
	x_2	(2)	0	$-\frac{1}{2}$	1	$\frac{1}{4}$	0	$-\frac{1}{8}$	0	0	$\frac{1}{8}$	$3\frac{3}{4}$
	v_1	(3)	0	2	0	$-\frac{1}{2}$	0	$\frac{1}{4}$	1	0	$-\frac{1}{4}$	$22\frac{1}{2}$
2	Z	(0)	-1	0	0	$-\frac{5}{2}$	1	$\frac{3}{4}$	1	0	$\frac{1}{4}$	$-7\frac{1}{2}$
	z_1	(1)	0	0	0	$\frac{5}{2}$	-1	$-\frac{3}{4}$	-1	1	$\frac{3}{4}$	$7\frac{1}{2}$
	x_2	(2)	0	0	1	$\frac{1}{8}$	0	$-\frac{1}{16}$	$\frac{1}{4}$	0	$\frac{1}{16}$	$9\frac{3}{8}$
	x_1	(3)	0	1	0	$-\frac{1}{4}$	0	$\frac{1}{8}$	$\frac{1}{2}$	0	$-\frac{1}{8}$	$11\frac{1}{4}$
3	Z	(0)	-1	0	0	0	0	0	0	1	1	0
	u_1	(1)	0	0	0	1	$-\frac{2}{5}$	$-\frac{3}{10}$	$-\frac{2}{5}$	$\frac{2}{5}$	$\frac{3}{10}$	3
	x_2	(2)	0	0	1	0	$\frac{1}{20}$	$-\frac{1}{40}$	$\frac{3}{10}$	$-\frac{1}{20}$	$\frac{1}{40}$	9
	x_1	(3)	0	1	0	0	$-\frac{1}{10}$	$\frac{1}{20}$	$\frac{2}{5}$	$\frac{1}{10}$	$-\frac{1}{20}$	12

tabla se elimina tanto u_1 como y_2 —puesto que v_1 y x_2 son variables básicas— y de manera automática se elige x_1 como único candidato con coeficientes negativos en el renglón 0, mientras que el método *simplex normal* habría permitido la entrada de x_1 o bien de u_1 , porque están empatadas con el coeficiente más negativo. En la tercera tabla se eliminan y_1 y y_2 (porque x_1 y x_2 son variables básicas). Esta vez u_1 no se elimina, puesto que v_1 ya no es una variable básica, así que se elige u_1 como variable entrante.

La solución óptima que resulta para esta fase 1 del problema es $x_1 = 12$, $x_2 = 9$, $u_1 = 3$, mientras el resto de las variables es igual a cero. (El problema 12.7-1c pide que se verifique que esta solución es óptima por medio de demostrar que si $x_1 = 12$, $x_2 = 9$ y $u_1 = 3$, se satisfacen las condiciones KKT del problema original, cuando se escriben en la forma dada en la sección 12.6.) Entonces, la solución óptima para el problema de programación cuadrática —que sólo incluye las variables x_1 y x_2 — es $(x_1, x_2) = (12, 9)$.

En la sección Worked Examples de la página en internet de este libro se incluye **otro ejemplo** que ilustra la aplicación del método *simplex* modificado a un problema de programación cuadrática. En este ejemplo también se aplican las condiciones KKT.

Algunas opciones de software

En el IOR Tutorial se incluye una rutina interactiva del método *simplex* modificado para ayudar a aprender este algoritmo. Además, tanto Excel como LINGO, LINDO y MPL/CPLEX pueden resolver problemas de programación cuadrática.

El procedimiento para usar Excel es casi el mismo que se emplea en programación lineal. La única diferencia crucial es que la ecuación que se introduce a la celda que contiene el valor de la función objetivo ahora debe ser una ecuación cuadrática. Para ilustrar este caso, considere de nuevo el ejemplo que se introdujo al principio de la sección, que tiene la función objetivo

$$f(x_1, x_2) = 15x_1 + 30x_2 + 4x_1x_2 - 2x_1^2 - 4x_2^2.$$

Suponga que los valores de x_1 y x_2 están en las celdas B4 y C4 de la hoja de cálculo de Excel, y que el valor de la función objetivo está en la celda F4. En este caso, la ecuación para la celda F4 debe ser

$$F4 = 15*B4 + 30*C4 + 4*B4*C4 - 2*(B4^2) - 4*(C4^2),$$

donde el símbolo 2 indica un exponente de 2. Antes de resolver el modelo, se debe oprimir el botón opciones y asegurarse de que la opción *Assume Linear Model* (Adoptar modelo lineal) *no* esté seleccionada (puesto que éste no es un modelo de programación *lineal*).

Cuando se use MPL/CPLEX se debe establecer el tipo de modelo como cuadrático mediante la introducción del siguiente enunciado al inicio del archivo del modelo:

OPTIONS

ModelType = Quadratic

(En forma alternativa, se puede seleccionar la opción Quadratic Models en el cuadro de diálogo de MPL Language, pero debe cambiar la configuración cuando de nuevo se trabaje con un modelo lineal.) Por lo demás, el procedimiento es el mismo que para programación lineal, excepto que la expresión de la función objetivo ahora es una función cuadrática. Así, en el ejemplo, la función objetivo se expresaría como

$$15x_1 + 30x_2 + 4x_1x_2 - 2(x_1^2) - 4(x_2^2).$$

No es necesario hacer más al correr CPLEX, pues reconocerá de manera automática que el modelo se refiere a un problema de programación cuadrática.

Esta función objetivo se debe expresar de la misma manera que para un modelo de LINGO. Después LINGO hará una llamada automática a su solucionador no lineal para resolverlo.

En realidad, todos los archivos de Excel, MPL/CPLEX y LINGO/LINDO para este capítulo del OR Courseware demuestran sus procedimientos, pues exhiben los detalles de la forma en que estos paquetes de software establecen y resuelven el ejemplo.

Algunos de estos paquetes se pueden aplicar a tipos más complicados de problemas de programación no lineal que la programación cuadrática. Aunque CPLEX no puede, la versión profesional de MPL tiene otros solucionadores que sí lo hacen. La versión para estudiantes de MPL que se encuentra en la página en internet de este libro, incluye un solucionador llamado CONOPT (un producto de ARKI Consulting) que está diseñado para resolver problemas de programación convexa. Se puede emplear después de agregar el siguiente enunciado al inicio del archivo del modelo.

OPTIONS

ModelType = Nonlinear

Tanto Excel como LINGO incluyen solucionadores no lineales versátiles. Sin embargo, debe tenerse en mente que no se garantiza la obtención de una solución óptima para problemas complicados, en especial para problemas de programación no convexa (el tema de la sección 12.10). Por otro lado, LINGO contiene un *optimizador global* que encontrará una solución óptima global para problemas de programación no convexa que sean lo suficientemente pequeños. MPL también contiene un optimizador global llamado LGO como uno de los solucionadores que se proporcionan en el sitio en internet de este libro.

■ 12.8 PROGRAMACIÓN SEPARABLE

En la sección anterior se presentó la forma en que se puede resolver una clase de problemas de programación no lineal mediante una extensión del método símplex. Ahora se estudiará otra clase llamada *programación separable*, para la que se puede encontrar una aproximación tan cercana como se quiera con un problema de programación lineal que tiene un número mayor de variables.

Como se indicó en la sección 12.3, la programación separable supone que la función objetivo $f(\mathbf{x})$ es cóncava, todas las restricciones $g_i(\mathbf{x})$ son convexas y todas estas funciones son funciones separables (lo que significa que cada término incluye una sola variable). Sin embargo, para simplificar la presentación, esta sección se centra en el caso especial en el que las funciones $g_i(\mathbf{x})$ convexas y separables, en realidad son *funciones lineales* como las de programación lineal. (Al final de esta sección volveremos a revisar el caso general de manera breve.) En consecuencia, sólo la función objetivo requerirá un tratamiento especial.

Bajo los supuestos anteriores, la función objetivo se puede expresar como una suma de funciones cóncavas de variables individuales,

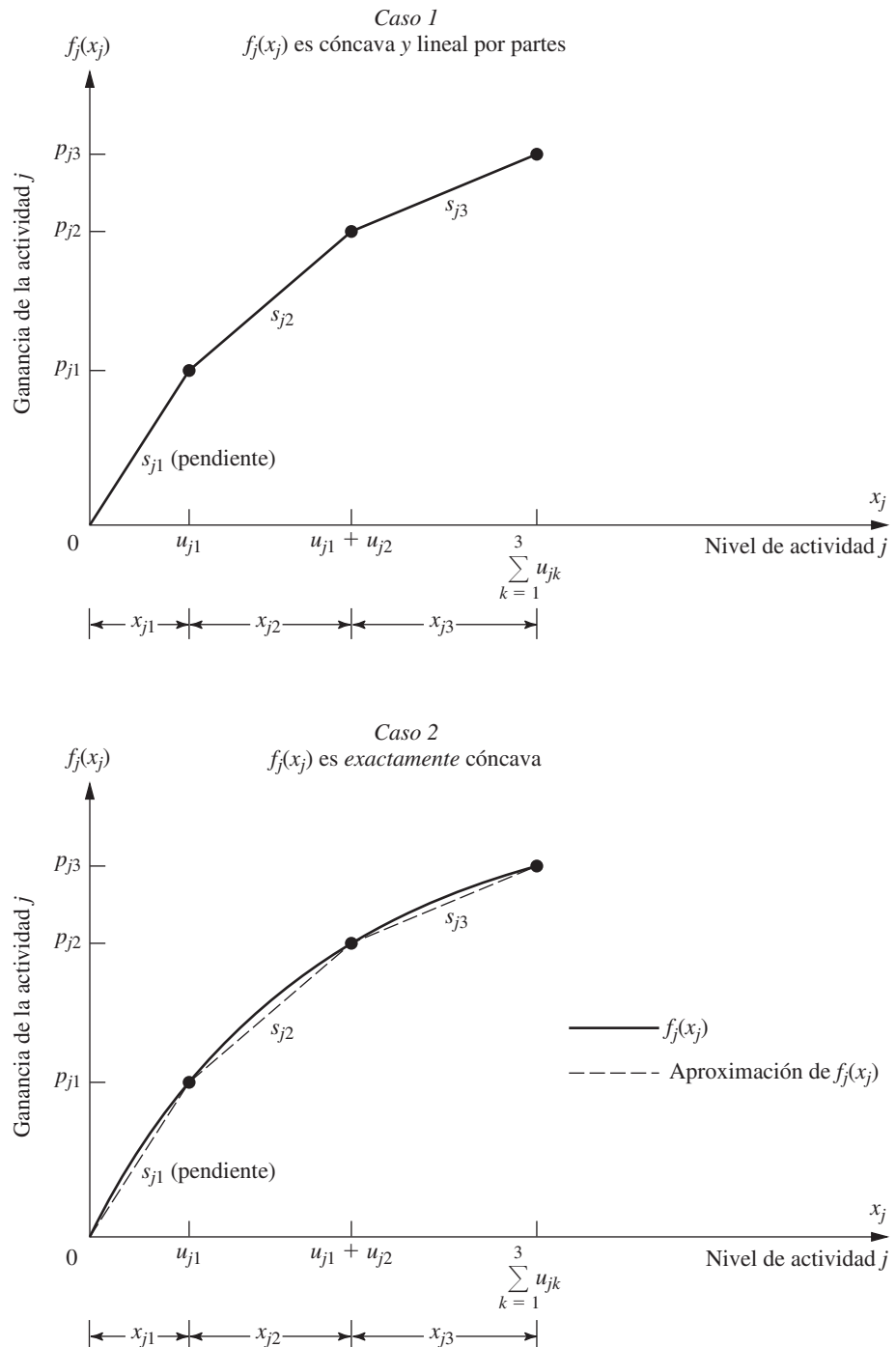
$$f(\mathbf{x}) = \sum_{j=1}^n f_j(x_j),$$

de manera que cada $f_j(x_j)$ tiene una forma¹⁶ como la que se muestra en la figura 12.15, en el rango de valores factibles de x_j . Como $f(\mathbf{x})$ representa la medida de desempeño (por ejemplo, ganancia) de todas las actividades juntas, $f_j(x_j)$ representa la *contribución a la ganancia* por parte de la actividad j cuando se realiza al nivel x_j . La condición de que $f(\mathbf{x})$ sea separable sólo implica aditividad (vea la sección 3.3); esto es, no existe interacción entre las actividades —no existen términos de productos cruzados— que afecten la ganancia total más allá de sus contribuciones independientes. El supuesto de que cada $f_j(x_j)$ es cóncava indica que la *ganancia marginal* —pendiente de la curva de ganancia—, se mantiene igual o decrece (*nunca* aumenta) a medida que x_j crece.

Las curvas cóncavas de la ganancia se presentan con mucha frecuencia. Por ejemplo, puede ser posible vender una cantidad limitada de algún producto a un cierto precio y después una cantidad adicional a un precio menor, y tal vez, otra cantidad adicional a un precio todavía menor. De igual manera, puede ser necesario comprar materias primas a fuentes cada vez más costosas. Otra situación común es aquella en la que debe emplearse un proceso de producción más caro —como el uso de tiempo extra—, para aumentar la tasa de producción sobre cierta cantidad.

Este tipo de situaciones puede conducir a cualquiera de las curvas de ganancia que se muestran en la figura 12.15. En el caso 1, la pendiente decrece *por intervalos*, de manera que $f_j(x_j)$ es una *función lineal por partes* (una sucesión de segmentos de recta conectados). En el caso 2, la

¹⁶ $f(\mathbf{x})$ es cóncava si y sólo si cada $f_j(x_j)$ es cóncava.



■ **FIGURA 12.15**
Forma de curvas de ganancia para programación separable.

pendiente puede decrecer en forma continua al aumentar x_j , de manera que $f_j(x_j)$ es una función cóncava general. Cualquier función de este tipo se puede aproximar tanto como se quiera por una función lineal por partes y este tipo de aproximación se usa, según sea necesario, para problemas de programación separable. (En la figura 12.15 se muestra una función de aproximación que consiste en sólo tres segmentos de recta, pero la aproximación se puede mejorar si se introducen más intervalos.) Esta aproximación es muy conveniente, puesto que una función lineal por partes de una sola variable se puede escribir como una *función lineal* de múltiples variables, con la restricción especial sobre estas variables que se describe a continuación.

Reformulación como un problema de programación lineal

La clave para dar a una función lineal por partes la forma de una función lineal es usar variables separadas para cada segmento de recta. Como ilustración, considere la función lineal por partes $f_j(x_j)$ que se muestra en la figura 12.15, caso 1 —o la aproximación por la función lineal por partes del caso 2—, que tiene tres segmentos de recta en el intervalo de valores factibles de x_j . Se introducen las tres nuevas variables, x_{j1} , x_{j2} y x_{j3} , y se establece

$$x_j = x_{j1} + x_{j2} + x_{j3},$$

donde

$$0 \leq x_{j1} \leq u_{j1}, \quad 0 \leq x_{j2} \leq u_{j2}, \quad 0 \leq x_{j3} \leq u_{j3}.$$

Después se usan las pendientes, s_{j1} , s_{j2} y s_{j3} para volver a escribir $f_j(x_j)$ como

$$f_j(x_j) = s_{j1}x_{j1} + s_{j2}x_{j2} + s_{j3}x_{j3},$$

Con la **restricción especial**

$$\begin{aligned} x_{j2} &= 0 & \text{siempre que} & & x_{j1} < u_{j1}, \\ x_{j3} &= 0 & \text{siempre que} & & x_{j2} < u_{j2}. \end{aligned}$$

Para entender por qué se requiere esta restricción especial, suponga que $x_j = 1$, donde $u_{jk} > 1$ ($k = 1, 2, 3$), de manera que $f_j(1) = s_{j1}$. Observe que

$$x_{j1} + x_{j2} + x_{j3} = 1$$

permite

$$\begin{aligned} x_{j1} = 1, \quad x_{j2} = 0, \quad x_{j3} = 0 & \Rightarrow f_j(1) = s_{j1}, \\ x_{j1} = 0, \quad x_{j2} = 1, \quad x_{j3} = 0 & \Rightarrow f_j(1) = s_{j2}, \\ x_{j1} = 0, \quad x_{j2} = 0, \quad x_{j3} = 1 & \Rightarrow f_j(1) = s_{j3}, \end{aligned}$$

y así sucesivamente, donde

$$s_{j1} > s_{j2} > s_{j3}.$$

Sin embargo, la restricción especial sólo permite la primera posibilidad, que es la única que da el valor correcto de $f_j(1)$.

Desafortunadamente, la restricción especial no se ajusta al formato que requieren las restricciones de programación lineal, por lo cual *algunas* funciones lineales por partes no se pueden reescribir en el formato de programación lineal. Sin embargo, se supone que *nuestra* $f_j(x_j)$ es cóncava, por lo que $s_{j1} > s_{j2} > \dots$, por lo cual un algoritmo para maximizar $f(\mathbf{x})$ de manera *automática* asigna la prioridad más alta al uso de x_{j1} cuando (en realidad) aumenta el valor de x_j desde cero, la siguiente prioridad al uso de x_{j2} , etc., sin incluir siquiera la restricción especial explícita en el modelo. Esta observación conduce a la siguiente propiedad esencial.

Propiedad esencial de programación separable. Cuando $f(\mathbf{x})$ y $g_i(\mathbf{x})$ satisfacen los supuestos de programación separable y las funciones lineales por partes que resultan se ponen en la forma de funciones lineales, al eliminar la *restricción especial* se obtiene un *modelo de programación lineal* cuya solución óptima satisface de manera automática la restricción especial.

Más adelante, en esta sección, se profundizará un poco más en la lógica que sustenta esta propiedad esencial, en el contexto de un ejemplo específico. (Vea también el problema 12.8-6a.)

Para escribir el modelo completo de programación lineal con la notación anterior, sea n_j el número de segmentos de recta en $f_j(x_j)$ (o en la función lineal por partes que la aproxima), de manera que

$$x_j = \sum_{k=1}^{n_j} x_{jk}$$

se sustituirá en todo el modelo original y

$$f_j(x_j) = \sum_{k=1}^{n_j} s_{jk}x_{jk}$$

se sustituirá¹⁷ en la función objetivo para $j = 1, 2, \dots, n$. El modelo que se obtiene es

$$\text{Maximizar} \quad Z = \sum_{j=1}^n \left(\sum_{k=1}^{n_j} s_{jk}x_{jk} \right),$$

sujeta a

$$\begin{aligned} \sum_{j=1}^n a_{ij} \left(\sum_{k=1}^{n_j} x_{jk} \right) &\leq b_i, & \text{para } i = 1, 2, \dots, m \\ x_{jk} &\leq u_{jk}, & \text{para } k = 1, 2, \dots, n_j; j = 1, 2, \dots, n \end{aligned}$$

y

$$x_{jk} \geq 0, \quad \text{para } k = 1, 2, \dots, n_j; j = 1, 2, \dots, n.$$

(Las restricciones $\sum_{k=1}^{n_j} x_{jk} \geq 0$ se eliminan, puesto que están garantizadas por las restricciones $x_{jk} \geq 0$.) Si algunas variables originales x_j no tuvieran cota superior, entonces $u_{jn_j} = \infty$, por lo que se debe eliminar la restricción que incluye esta cantidad.

Una manera eficiente de resolver este modelo¹⁸ es usar la versión simplificada del método símplex para manejar restricciones de cota superior (descrito en la sección 7.3). Después de obtener una solución óptima para este modelo, se calcula

$$x_j = \sum_{k=1}^{n_j} x_{jk},$$

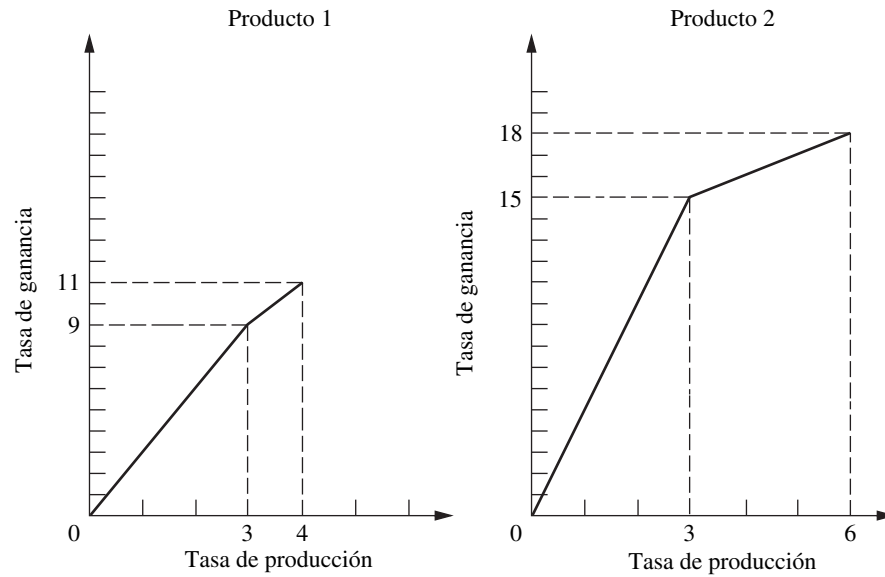
para $j = 1, 2, \dots, n$, con el fin de identificar una solución óptima para el problema original de programación separable (o su aproximación lineal por partes).

Ejemplo. La Wyndor Glass Co. (vea la sección 3.1) recibió un pedido especial de artículos procesados a mano que se debe elaborar en las plantas 1 y 2 durante los próximos cuatro meses. Para cumplir con este pedido será necesario asignar algunos empleados de las brigadas de trabajo de los productos normales, por lo que el resto del personal tendrá que trabajar horas extra para utilizar toda la capacidad de producción de la maquinaria y equipo de la planta para elaborar estos productos. En particular, a fin de fabricar los dos nuevos productos normales que se analizaron en la sección 3.1, el tiempo extra tendrá que utilizar el último 25% de la capacidad disponible en la planta 1 para el producto 1 y el último 50% de la capacidad disponible en la planta 2 para el producto 2. El costo adicional del tiempo extra reducirá la ganancia de cada unidad del producto 1 de 3 a 2 dólares y del producto 2, de 5 dólares a 1 dólar, de lo que resultarán las *curvas de ganancia* de la figura 12.16, las cuales se ajustan a la forma del caso 1 de la figura 12.15.

La administración ha decidido usar tiempo extra en lugar de contratar más trabajadores durante esta situación temporal. Sin embargo, insiste en que se aprovechen por completo las brigadas de trabajo de cada producto en tiempo normal antes de usar cualquier tiempo extra. Aún más, piensa que temporalmente se deben cambiar las tasas de producción actuales ($x_1 = 2$ para el producto 1 y $x_2 = 6$ para el producto 2), si esta medida mejora el rendimiento total. Por todo ello, ha girado instrucciones al equipo de IO para revisar los productos 1 y 2 y determinar la nueva mezcla de productos más redituable durante los próximos cuatro meses.

¹⁷ Si una o más de las $f_j(x_j)$ son ya funciones lineales, $f_j(x_j) = c_j x_j$, entonces $n_j = 1$, de manera que ninguna de estas sustituciones se hará para j .

¹⁸ Se puede encontrar un algoritmo especializado muy eficiente para resolver este problema en R. Fourer: "A Specialized Algorithm for Piecewise-Linear Programming III: Computational Analysis and Applications", en *Mathematical Programming*, **53**: 213-235, 1992. Vea también A. M. Geoffrion: "Objective Function Approximations in Mathematical Programming", en *Mathematical Programming*, **13**: 23-37, 1977.



■ **FIGURA 12.16**
 Datos de ganancia durante
 los próximos 4 meses de la
 Wyndor Glass Co.

Formulación. Para refrescar la memoria, el modelo de programación lineal del problema original de la Wyndor Glass Co., de la sección 3.1 es:

$$\text{Maximizar} \quad Z = 3x_1 + 5x_2,$$

sujeta a

$$\begin{aligned} x_1 &\leq 4 \\ 2x_2 &\leq 12 \\ 3x_1 + 2x_2 &\leq 18 \end{aligned}$$

y

$$x_1 \geq 0, \quad x_2 \geq 0.$$

Ahora se debe modificar este modelo para que se ajuste a la nueva situación que se acaba de describir. Para esto, sea $x_1 = x_{1R} + x_{1O}$ la tasa de elaboración del producto 1, en donde x_{1R} es la tasa de producción alcanzada con tiempos normales de trabajo y x_{1O} es la tasa de producción incremental al usar tiempo extra. Defina $x_2 = x_{2R} + x_{2O}$ de la misma manera para el producto 2. Por tanto, en la notación general del modelo de programación lineal para programación separable que se presentó antes del ejemplo, $n = 2$, $n_1 = 2$ y $n_2 = 2$. Al sustituir los datos de la figura 12.16—incluso las tasas máximas de producción en tiempo normal y tiempo extra— en este modelo general se obtiene el modelo específico para esta aplicación. En particular, el nuevo problema de programación lineal trata de determinar los valores de x_{1R} , x_{1O} , x_{2R} y x_{2O} para

$$\text{Maximizar} \quad Z = 3x_{1R} + 2x_{1O} + 5x_{2R} + x_{2O},$$

sujeta a

$$\begin{aligned} x_{1R} + x_{1O} &\leq 4 \\ 2(x_{2R} + x_{2O}) &\leq 12 \\ 3(x_{1R} + x_{1O}) + 2(x_{2R} + x_{2O}) &\leq 18 \\ x_{1R} \leq 3, \quad x_{1O} \leq 1, \quad x_{2R} \leq 3, \quad x_{2O} \leq 3 \end{aligned}$$

y

$$x_{1R} \geq 0, \quad x_{1O} \geq 0, \quad x_{2R} \geq 0, \quad x_{2O} \geq 0.$$

(Observe que las restricciones de cota superior del penúltimo renglón del modelo convierten a las primeras dos restricciones funcionales en *redundantes*, de manera que estas dos ecuaciones se pueden eliminar.)

Sin embargo, existe un factor importante que no se toma en cuenta en forma explícita en esta formulación. En particular, no existe nada en el modelo que requiera que se utilice todo el tiempo normal disponible, antes de emplear cualquier tiempo extra para elaborar ese producto. En otras palabras, puede ser factible tener $x_{1O} > 0$ aun cuando $x_{1R} < 3$ y tener $x_{2O} > 0$ aun cuando $x_{2R} < 3$. Tal solución sería inaceptable para la administración. (La prohibición de esta solución conduce a la *restricción especial* de la que se habló antes en esta misma sección.)

Como consecuencia, se llega a la *propiedad esencial de programación separable*. Aun cuando este modelo no toma en cuenta este factor de manera explícita, ¡lo hace en forma implícita! Aunque el modelo tenga un exceso de soluciones “factibles” que en realidad son inaceptables, *garantiza* que cualquier solución *óptima* será legítima, es decir, que no sustituirá tiempo de trabajo normal disponible por horas extra. (En este caso el razonamiento es análogo al del método de la *gran M* que se presentó en la sección 4.6, en el que también se permiten soluciones factibles adicionales, pero *no óptimas*, por conveniencia para trabajar.) En efecto, se puede aplicar con seguridad el método *símplex* a este modelo para encontrar la mezcla de productos más redituable. La razón es doble. Primero, las dos variables de decisión para cada producto *siempre* aparecen juntas como una *suma*, $x_{1R} + x_{1O}$ o $x_{2R} + x_{2O}$, en *cada* restricción funcional distinta de las restricciones de cota superior sobre las variables individuales. Por tanto, *siempre* es posible convertir una solución factible no aceptable en una aceptable que tenga las mismas tasas de producción totales, $x_1 = x_{1R} + x_{1O}$ y $x_2 = x_{2R} + x_{2O}$, con sólo reemplazar la producción en tiempo extra por la máxima producción normal. Segundo, la producción en tiempo extra es menos redituable que la normal —es decir, la pendiente de cada curva de ganancia de la figura 12.16 es una función monótona *decreciente* de la tasa de producción—, de modo que al convertir una solución factible no aceptable en una aceptable *debe* incrementarse la tasa total de la ganancia Z . En consecuencia, cualquier solución factible que usa tiempo extra de producción para un producto, cuando todavía se dispone de tiempo normal, *no puede* ser óptima respecto del modelo.

Por ejemplo, considere la solución factible inaceptable $x_{1R} = 1, x_{1O} = 1, x_{2R} = 1, x_{2O} = 3$, que proporciona una ganancia total $Z = 13$. La forma aceptable de lograr las mismas tasas de producción $x_1 = 2$ y $x_2 = 4$ es $x_{1R} = 2, x_{1O} = 0, x_{2R} = 3, x_{2O} = 1$. Esta última solución también es factible pero hace que el valor de Z aumente en $(3 - 2)(1) + (5 - 1)(2) = 9$ hasta una tasa total de ganancia $Z = 22$.

De manera similar, la solución óptima para este modelo resulta ser $x_{1R} = 3, x_{1O} = 1, x_{2R} = 3, x_{2O} = 0$, que es una solución factible aceptable.

En la sección Worked Examples de la página en internet de este libro se presenta **otro ejemplo** que ilustra la aplicación de la programación separable.

Extensiones

Hasta ahora, la presentación se ha centrado en el caso especial de programación separable, en el que la única función no lineal es la función objetivo $f(\mathbf{x})$. Ahora considere en forma breve el caso general en el que las funciones de restricción $g_i(\mathbf{x})$ no tienen que ser lineales, pero son convexas y separables, de manera que cada $g_i(\mathbf{x})$ se puede expresar como una suma de funciones de variables individuales

$$g_i(\mathbf{x}) = \sum_{j=1}^n g_{ij}(x_j),$$

donde cada $g_{ij}(x_j)$ es una función *convexa*. Una vez más, cada una de estas funciones se puede aproximar tanto como se quiera por una función *lineal por partes* (si no está ya en esa forma). La nueva restricción es que para cada variable x_j ($j = 1, 2, \dots, n$), todas las aproximaciones lineales por partes de las funciones de esa variable $[f_j(x_j), g_{1j}(x_j), \dots, g_{mj}(x_j)]$ deben tener los *mismos* límites de intervalos, para poder usar las mismas variables nuevas $(x_{j1}, x_{j2}, \dots, x_{jn_j})$ para todas las funciones lineales por partes. Esta formulación conduce a un modelo de programación lineal igual al que

se obtiene para el caso especial, excepto que para cada i y j , las variables x_{jk} tienen coeficientes distintos en la restricción i [donde estos coeficientes son las pendientes correspondientes de la función lineal por partes que aproxima a $g_{ij}(x_j)$]. Como $g_{ij}(x_j)$ tiene que ser convexa, en principio la misma lógica implica que debe cumplirse la propiedad esencial de programación separable. (Vea el problema 12.8-6b.)

Una desventaja de la aproximación de funciones por medio de funciones lineales por partes, descrita en esta sección, es que sólo se logra una buena aproximación con un número muy grande de segmentos de recta (variables), mientras que una malla fina de intervalos nada más se necesita en la vecindad inmediata de la solución óptima. Se han desarrollado¹⁹ enfoques más elaborados, que usan una sucesión de funciones lineales por partes de *dos segmentos*, para obtener *aproximaciones cada vez más cercanas* en esta vecindad inmediata. Este tipo de enfoques tiende a ser más rápido y más exacto cuando se desea aproximar una solución óptima.

■ 12.9 PROGRAMACIÓN CONVEXA

Se han estudiado algunos casos especiales de programación convexa en las secciones 12.4 y 12.5 (problemas no restringidos), 12.7 (función objetivo cuadrática con restricciones lineales) y 12.8 (funciones separables). También se vio, en la sección 12.6, una parte de la teoría del caso general (condiciones necesarias y suficientes para la optimalidad). En esta sección se presentarán de manera breve algunos tipos de enfoques que se utilizan para resolver el problema general de programación convexa [donde la función objetivo $f(\mathbf{x})$ que se debe maximizar es cóncava y las funciones de restricción $g_i(\mathbf{x})$ son convexas] y después se presentará un ejemplo de un algoritmo para programación convexa.

No existe un algoritmo estándar único que se pueda usar siempre para resolver problemas de programación convexa. Se han desarrollado muchos algoritmos diferentes, cada uno con ventajas y desventajas, y la investigación continúa activa en esta área. En términos generales, la mayor parte de estos algoritmos cae dentro de alguna de las tres categorías siguientes.

Una categoría la constituyen los **algoritmos de gradiente**, en los que se modifica de alguna manera el procedimiento de búsqueda del gradiente de la sección 12.5 para evitar que la trayectoria de búsqueda penetre la frontera de restricción. Por ejemplo, un método popular es el *método de gradiente reducido generalizado* (GRG). El Excel Solver emplea el método GRG para resolver problemas de programación convexa. (Como se explica en la sección siguiente, el Premium Solver también incluye una opción evolutiva que fue diseñada para tratar con problemas de programación *no convexa*.)

La segunda categoría, los **algoritmos secuenciales no restringidos**, incluye los métodos de *función de finalización* y de *función barrera*. Estos algoritmos convierten el problema de optimización restringida original en una sucesión de problemas de *optimización no restringida* cuyas soluciones óptimas convergen a la solución óptima del problema original. Cada uno de estos problemas de optimización no restringida se puede resolver por medio del procedimiento de búsqueda del gradiente de la sección 12.5. Esta conversión se logra al incorporar las restricciones a una función de penalización (o función barrera) que se resta de la función objetivo, con el fin de imponer un castigo grande a la violación de cualquier restricción —o aun al hecho de estar cerca de los límites—. En la última parte de esta sección se describirá un algoritmo de la década de 1960, llamado **técnica de minimización no restringida secuencial** (o SUMT, por sus siglas en inglés), que fue pionero en esta categoría de algoritmos. (SUMT también ayudó a motivar algunos de los *métodos de punto interior* para programación lineal.)

La tercera categoría, los **algoritmos de aproximación secuencial**, incluye métodos de *aproximación lineal* y *aproximación cuadrática*. Estos algoritmos sustituyen la función objetivo no lineal por una sucesión de aproximaciones lineales o cuadráticas. Para problemas de optimización linealmente restringidos, estas aproximaciones permiten la aplicación repetida de los algoritmos de programación lineal o cuadrática. Este trabajo se logra mediante otro análisis que conduce a una sucesión de soluciones que convergen a una solución óptima para el problema original. Aunque estos algoritmos son adecuados en especial para problemas de optimización linealmente restringi-

¹⁹ R. R. Meyer: "Two-Segment Separable Programming", en *Management Science*, **25**: 385-395, 1979.

da, algunos se pueden extender a problemas con funciones de restricción no lineales, si se usan las aproximaciones lineales adecuadas.

Como un ejemplo de algoritmo de *aproximación secuencial* se presentará el **algoritmo de Frank-Wolfe**²⁰ para el caso de programación convexa *linealmente restringida* (de manera que las restricciones son $\mathbf{Ax} \leq \mathbf{b}$ y $\mathbf{x} \geq \mathbf{0}$, en forma matricial). Este procedimiento es bastante directo: combina aproximaciones lineales de la función objetivo (que permiten usar el método símplex) con el procedimiento para la optimización no restringida de una variable (como se describió en la sección 12.4).

Algoritmo de aproximación lineal secuencial (Frank-Wolfe)

Dada una solución de prueba factible $\mathbf{x}'$, la aproximación lineal que se usa para la función objetivo $f(\mathbf{x})$ es la expansión de primer orden por series de Taylor, de $f(\mathbf{x})$ alrededor de $\mathbf{x} = \mathbf{x}'$, es decir,

$$f(\mathbf{x}') \approx f(\mathbf{x}') + \sum_{j=1}^n \frac{\partial f(\mathbf{x}')}{\partial x_j} (x_j - x'_j) = f(\mathbf{x}') + \nabla f(\mathbf{x}')(\mathbf{x} - \mathbf{x}'),$$

donde estas derivadas parciales se evalúan en $\mathbf{x} = \mathbf{x}'$. Como $f(\mathbf{x}')$ y $\nabla f(\mathbf{x}')\mathbf{x}'$ tienen valores fijos, se pueden eliminar para obtener una función objetivo lineal equivalente,

$$g(\mathbf{x}) = \nabla f(\mathbf{x}')\mathbf{x} = \sum_{j=1}^n c_j x_j, \quad \text{donde } c_j = \frac{\partial f(\mathbf{x}')}{\partial x_j} \quad \text{en } \mathbf{x} = \mathbf{x}'.$$

Después se aplica el método símplex (o el procedimiento gráfico, si $n = 2$) al problema de programación lineal (LP) que resulta [maximizar $g(\mathbf{x})$ sujeta a las restricciones originales $\mathbf{Ax} \leq \mathbf{b}$ y $\mathbf{x} \geq \mathbf{0}$] a fin de obtener su solución óptima $\mathbf{x}_{LP}$. Observe que la función objetivo lineal necesariamente se incrementa cuando se hace un recorrido sobre el segmento de recta desde $\mathbf{x}'$ hasta $\mathbf{x}_{LP}$ (que se encuentra en la frontera de la región factible); sin embargo, la aproximación lineal puede no ser buena para una $\mathbf{x}$ lejana a $\mathbf{x}'$, por lo que quizá la función objetivo *no lineal* puede no continuar creciendo durante toda la trayectoria desde $\mathbf{x}'$ hasta $\mathbf{x}_{LP}$. Así, en lugar de aceptar $\mathbf{x}_{LP}$ como la siguiente solución de prueba, se elige el punto sobre este segmento de recta que maximiza la función objetivo no lineal. Este punto se puede encontrar si se realiza una *búsqueda en una dimensión* mediante el procedimiento de la sección 12.4, donde la única variable para propósitos de esta búsqueda es la fracción t de la distancia total de $\mathbf{x}'$ a $\mathbf{x}_{LP}$. En consecuencia, este punto se convierte en una nueva solución de prueba para iniciar la siguiente iteración del algoritmo, como se acaba de describir. La sucesión de soluciones de prueba que generan las iteraciones converge hacia una solución óptima para el problema original, de manera que el algoritmo se detiene en cuanto las soluciones de prueba están lo suficientemente cerca entre sí como para poder concluir que se llegó a esta solución óptima.

Resumen de algoritmo de Frank-Wolfe

Paso inicial: se encuentra una solución de prueba factible inicial $\mathbf{x}^{(0)}$, por ejemplo, al aplicar los procedimientos de programación lineal para encontrar una solución BF. Se hace $k = 1$.

Iteración k :

1. Para $j = 1, 2, \dots, n$, se evalúa

$$\frac{\partial f(\mathbf{x})}{\partial x_j} \quad \text{en } \mathbf{x} = \mathbf{x}^{(k-1)}$$

y se iguala c_j a este valor.

²⁰ M. Frank y P. Wolfe, "An Algorithm for Quadratic Programming", en *Naval Research Logistics Quarterly*, **3**: 95-110, 1956. Aunque originalmente se diseñó para programación cuadrática, este algoritmo se adapta con facilidad al caso de una función objetivo cóncava general que se estudia aquí.

2. Se encuentra una solución óptima $\mathbf{x}_{LP}^{(k)}$ para el siguiente problema de programación lineal.

$$\text{Maximizar} \quad g(\mathbf{x}) = \sum_{j=1}^n c_j x_j,$$

sujeta a

$$\mathbf{Ax} \leq \mathbf{b} \quad \text{y} \quad \mathbf{x} \geq \mathbf{0}.$$

3. Para la variable t ($0 \leq t \leq 1$), se establece

$$h(t) = f(\mathbf{x}) \quad \text{para} \quad \mathbf{x} = \mathbf{x}^{(k-1)} + t(\mathbf{x}_{LP}^{(k)} - \mathbf{x}^{(k-1)}),$$

de manera que $h(t)$ proporciona el valor de $f(\mathbf{x})$ sobre el segmento de recta entre $\mathbf{x}^{(k-1)}$ (donde $t = 0$) y $\mathbf{x}_{LP}^{(k)}$ (donde $t = 1$). Se aplica algún procedimiento como el de búsqueda en una dimensión (vea la sección 12.4) para maximizar $h(t)$ en $0 \leq t \leq 1$ y se establece $\mathbf{x}^{(k)}$ igual a la $\mathbf{x}$ correspondiente. Se pasa a la regla de detención.

Regla de detención: si $\mathbf{x}^{(k-1)}$ y $\mathbf{x}^{(k)}$ están lo suficientemente cerca, el proceso se detiene y se usa $\mathbf{x}^{(k)}$ (o una extrapolación de $\mathbf{x}^{(0)}, \mathbf{x}^{(1)}, \dots, \mathbf{x}^{(k-1)}, \mathbf{x}^{(k)}$) como la estimación de una solución óptima. De otra manera se hace $k = k + 1$ y se realiza otra iteración.

Ahora se ilustrará este procedimiento.

Ejemplo. Considere el siguiente problema de programación convexa linealmente restringido;

$$\text{Maximizar} \quad f(\mathbf{x}) = 5x_1 - x_1^2 + 8x_2 - 2x_2^2,$$

sujeta a

$$3x_1 + 2x_2 \leq 6$$

y

$$x_1 \geq 0, \quad x_2 \geq 0.$$

Observe que

$$\frac{\partial f}{\partial x_1} = 5 - 2x_1, \quad \frac{\partial f}{\partial x_2} = 8 - 4x_2,$$

por lo que el máximo *no restringido* $\mathbf{x} = (\frac{5}{2}, 2)$, viola las restricciones funcionales. Por tanto, se necesita trabajar un poco más para encontrar el máximo *restringido*.

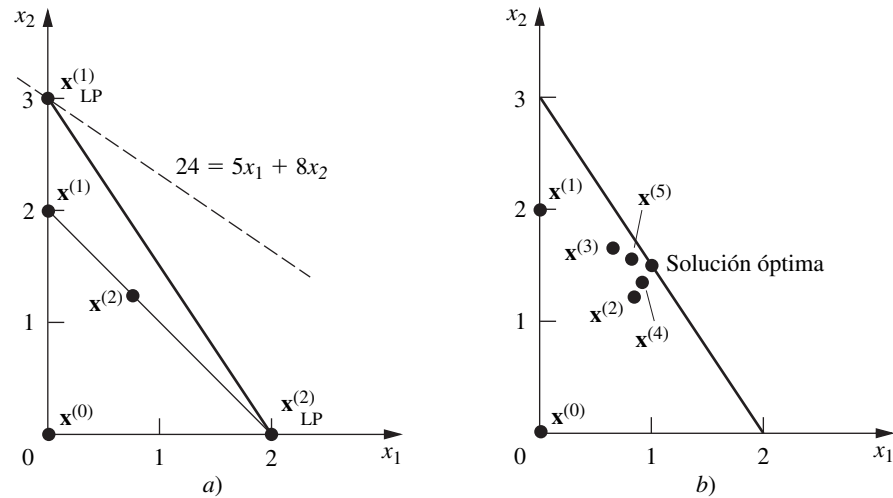
Iteración 1: como $\mathbf{x} = (0, 0)$, es claro que es factible (y corresponde a la solución inicial BF de las restricciones de programación lineal), se elige como la solución de prueba inicial $\mathbf{x}^{(0)}$ para el algoritmo de Frank-Wolfe. Al sustituir $x_1 = 0$ y $x_2 = 0$ en las expresiones de las derivadas parciales se obtiene $c_1 = 5$ y $c_2 = 8$, con lo que $g(\mathbf{x}) = 5x_1 + 8x_2$ es la primera aproximación lineal de la función objetivo. Con la solución gráfica de este problema de programación lineal (vea la figura 12.17a) se llega a $\mathbf{x}_{LP}^{(1)} = (0, 3)$. En la parte 3 del paso iterativo, los puntos en el segmento de recta que une a $(0, 0)$ y $(0, 3)$, que se muestran en la figura 12.17a, se expresan por

$$\begin{aligned} (x_1, x_2) &= (0, 0) + t[(0, 3) - (0, 0)] \quad \text{para } 0 \leq t \leq 1 \\ &= (0, 3t) \end{aligned}$$

como se muestra en la sexta columna de la tabla 12.6. Esta expresión da entonces

$$\begin{aligned} h(t) &= f(0, 3t) = 8(3t) - 2(3t)^2 \\ &= 24t - 18t^2, \end{aligned}$$

■ **FIGURA 12.17**
Ilustración del algoritmo
de Frank-Wolfe.



de modo que el valor $t = t^*$ que maximiza $h(t)$ en el intervalo $0 \leq t \leq 1$ se puede obtener, en este caso, al establecer

$$\frac{dh(t)}{dt} = 24 - 36t = 0,$$

por lo que $t^* = \frac{2}{3}$. Este resultado conduce a la siguiente solución de prueba

$$\begin{aligned}\mathbf{x}^{(1)} &= (0, 0) + \frac{2}{3}[(0, 3) - (0, 0)] \\ &= (0, 2),\end{aligned}$$

lo cual completa la primera iteración.

Iteración 2: para esbozar los cálculos que conducen a los resultados del segundo renglón de la tabla 12.6, observe que $\mathbf{x}^{(1)} = (0, 2)$ da

$$\begin{aligned}c_1 &= 5 - 2(0) = 5, \\ c_2 &= 8 - 4(2) = 0.\end{aligned}$$

Al resolver el problema en una gráfica con la función objetivo $g(\mathbf{x}) = 5x_1$ en la región factible de la figura 12.17a, se obtiene $\mathbf{x}^{(2)}_{LP} = (2, 0)$. De esta forma, la expresión del segmento de recta entre $\mathbf{x}^{(1)}$ y $\mathbf{x}^{(2)}_{LP}$ (vea la figura 12.17a) es

$$\begin{aligned}\mathbf{x} &= (0, 2) + t[(2, 0) - (0, 2)] \\ &= (2t, 2 - 2t),\end{aligned}$$

de manera que

$$\begin{aligned}h(t) &= f(2t, 2 - 2t) \\ &= 5(2t) - (2t)^2 + 8(2 - 2t) - 2(2 - 2t)^2 \\ &= 8 + 10t - 12t^2.\end{aligned}$$

■ **TABLA 12.6** Aplicación del algoritmo de Frank-Wolfe al ejemplo

k	$\mathbf{x}^{(k-1)}$	c_1	c_2	$\mathbf{x}^{(k)}_{LP}$	$\mathbf{x}$ para $h(t)$	$h(t)$	t^*	$\mathbf{x}^{(k)}$
1	(0, 0)	5	8	(0, 3)	(0, 3t)	$24t - 18t^2$	$\frac{2}{3}$	(0, 2)
2	(0, 2)	5	0	(2, 0)	$(2t, 2 - 2t)$	$8 + 10t - 12t^2$	$\frac{5}{12}$	$(\frac{5}{6}, \frac{7}{6})$

Si se establece

$$\frac{dh(t)}{dt} = 10 - 24t = 0$$

se llega a $t^* = \frac{5}{12}$. Entonces

$$\begin{aligned}\mathbf{x}^{(2)} &= (0, 2) + \frac{5}{12}[(2, 0) - (0, 2)] \\ &= \left(\frac{5}{6}, \frac{7}{6}\right),\end{aligned}$$

con lo que se completa la segunda iteración.

En la figura 12.17b se puede observar cómo las soluciones de prueba adquieren valores de puntos alternados entre dos trayectorias que parecen cruzarse aproximadamente en el punto $\mathbf{x} = (1, \frac{3}{2})$. En realidad, este punto es la solución óptima, como se puede verificar si se aplican las condiciones KKT de la sección 12.6.

Este ejemplo ilustra una característica común del algoritmo de Frank-Wolfe: que las soluciones de prueba alternan entre dos (o más) trayectorias. Cuando esto ocurre, se pueden extrapolar las trayectorias al punto aproximado en que se cruzan, para estimar una solución óptima. Esta estimación tiende a ser mejor que usar la última solución de prueba generada. Esto se debe a que la convergencia de las soluciones de prueba hacia la solución óptima tiende a ser lenta y por ello es posible que la última solución esté todavía alejada del óptimo.

Si desea ver **otro ejemplo** de aplicación del algoritmo de Frank-Wolfe, en la sección Worked Examples del sitio en internet de este libro se incluye uno. En el OR Tutor también se proporciona un **ejemplo adicional**. Asimismo en el IOR Tutorial se incluye un algoritmo interactivo para este algoritmo.

Algunos otros algoritmos

Se debe hacer hincapié en que el algoritmo de Frank-Wolfe es sólo un ejemplo de los algoritmos de aproximación secuencial. Muchos de ellos utilizan aproximaciones *cuadráticas* en lugar de *lineales* en cada iteración, porque la aproximación cuadrática proporciona un ajuste mucho mejor al problema original y por lo mismo permite que la sucesión de soluciones converja mucho más rápido hacia la solución óptima que en el caso de la figura 12.17b. Por esta razón, aun cuando el uso de los métodos de aproximación lineal secuencial, como el algoritmo de Frank-Wolfe, sea bastante sencillo y directo, en general, para las aplicaciones reales se prefieren los *métodos de aproximación cuadrática*. Entre éstos, los que más se usan son los métodos *cuasi-Newton* (o de *variables métricas*), que calculan una aproximación cuadrática a la curvatura de una función no lineal sin calcular en forma explícita las segundas derivadas (parciales). (En los problemas de optimización linealmente restringidos, esta función no lineal es justo la función objetivo, mientras que si se tienen restricciones no lineales, se trata de la función de Lagrange descrita en el apéndice 3.) Algunos algoritmos cuasinewtonianos no formulan ni dan una solución aproximada del problema de programación cuadrática en cada iteración, sino que incorporan algunos de los ingredientes básicos de los *algoritmos de gradiente*. (Vea la referencia seleccionada 2 para consultar detalles adicionales acerca de los algoritmos de aproximación secuencial.)

Ahora se pasará de los algoritmos de aproximación secuencial a los *algoritmos no restringidos secuenciales*. Como se mencionó al inicio de la sección, los algoritmos del último tipo resuelven el problema de optimización restringida original en lugar de resolver una secuencia de problemas de optimización *no restringida*.

Un procedimiento de este tipo que se utiliza de manera profusa desde su origen en la década de 1960 es la *técnica secuencial de minimización no restringida* (o SUMT, *sequential unconstrained minimization technique*).²¹ En realidad, existen dos versiones de SUMT, una de las cuales es un

²¹ Vea la referencia seleccionada 1.

algoritmo *de punto exterior* que utiliza soluciones *no factibles* al mismo tiempo que una *función de penalización* para forzar la convergencia hacia la región factible. Se describirá la otra versión, que es un algoritmo *de punto interior* que trabaja en forma directa con soluciones *factibles* y, de manera simultánea, utiliza una *función barrera* para forzarlas a mantenerse dentro de esa región factible. Aunque la técnica SUMT en un principio se presentó como una técnica de minimización, se convertirá aquí en una técnica de maximización, a fin de que sea congruente con el resto del capítulo. En consecuencia, se continuará con el supuesto de que el problema se encuentra en la forma dada al principio y que todas las funciones son diferenciables.

Técnica secuencial de minimización no restringida (SUMT)

Como su nombre lo indica, SUMT sustituye el problema original por una *sucesión* de problemas de optimización *no restringida*, cuyas soluciones *convergen* hacia una solución (máximo local) del problema original. Este enfoque es muy atractivo, pues es mucho más sencillo resolver los problemas de optimización no restringida (vea la sección 12.5) que los que tienen restricciones. Cada uno de los problemas no restringidos de esta sucesión incluye la selección de un valor estrictamente positivo (cada vez más pequeño) de un escalar r y después la obtención de $\mathbf{x}$ para

$$\text{Maximizar } P(\mathbf{x}; r) = f(\mathbf{x}) - rB(\mathbf{x}).$$

Aquí $B(\mathbf{x})$ es una **función barrera** que tiene las propiedades (para las $\mathbf{x}$ factibles del problema original) siguientes:

1. $B(\mathbf{x})$ es *pequeño* cuando $\mathbf{x}$ está *lejos* de la frontera de la región factible.
2. $B(\mathbf{x})$ es *grande* cuando $\mathbf{x}$ está *cerca* de la frontera de la región factible.
3. $B(\mathbf{x}) \rightarrow \infty$ cuando la distancia desde la frontera (más cercana) de la región factible $\rightarrow 0$.

Así, si se inicia el procedimiento de búsqueda con una solución de prueba inicial *factible* y después se intenta incrementar $P(\mathbf{x}; r)$, entonces $B(\mathbf{x})$ proporciona una *barrera* que evita que la búsqueda cruce (o llegue a) la frontera de la región factible del problema original.

La elección más natural para $B(\mathbf{x})$ es

$$B(\mathbf{x}) = \sum_{i=1}^m \frac{1}{b_i - g_i(\mathbf{x})} + \sum_{j=1}^n \frac{1}{x_j}.$$

Observe que para valores factibles de $\mathbf{x}$, el denominador de cada término es proporcional a la distancia de $\mathbf{x}$ a la frontera de restricciones de la restricción funcional o de no negatividad correspondiente. En consecuencia, *cada* término es un *término de rechazo de frontera* que tiene las tres propiedades anteriores con respecto a esta frontera de restricción específica. Otra característica atractiva de esta $B(\mathbf{x})$ es que cuando se satisfacen todos los supuestos *de programación convexa*, $P(\mathbf{x}; r)$ es una función *cóncava*.

Como $B(\mathbf{x})$ mantiene la búsqueda lejos de la frontera de la región factible, una pregunta natural es: ¿qué pasa si la solución que se quiere está ahí? Esta preocupación es la razón por la que SUMT incluye la solución de una *sucesión* de estos problemas de optimización no restringida para valores cada vez más pequeños de r que se acercan a cero (donde la solución de prueba final de cada uno se convierte en la solución de prueba inicial del siguiente). Por ejemplo, cada nueva r se puede obtener a partir de la anterior multiplicando por una constante θ ($0 < \theta < 1$), en donde un valor usual es $\theta = 0.01$. A medida que r se acerca a 0, $P(\mathbf{x}; r)$ se acerca a $f(\mathbf{x})$, de manera que el máximo local correspondiente de $P(\mathbf{x}; r)$ converge hacia un máximo local del problema original. Por ello, sólo es necesario resolver problemas de optimización no restringida, hasta que se puedan extrapolar sus soluciones a esta solución límite.

¿Cuántos problemas son suficientes para permitir esta extrapolación? Cuando el problema original satisface los supuestos de programación convexa, se cuenta con información útil como guía en esta decisión. En particular, si $\bar{\mathbf{x}}$ es un máximo global de $P(\mathbf{x}; r)$, entonces

$$f(\bar{\mathbf{x}}) \leq f(\mathbf{x}^*) \leq f(\bar{\mathbf{x}}) + rB(\bar{\mathbf{x}}),$$

donde $\mathbf{x}^*$ es la solución *óptima* (desconocida) para el problema original. Entonces, $rB(\bar{\mathbf{x}})$ es el *error máximo* (en el valor de la función objetivo) que puede resultar al usar $\bar{\mathbf{x}}$ para aproximar $\mathbf{x}^*$,

y este error disminuye al extrapolar a un valor mayor a $\bar{x}$ para incrementar $f(\mathbf{x})$. Si de antemano se establece la *tolerancia del error*, entonces puede detenerse el proceso tan pronto como $rB(\bar{\mathbf{x}})$ sea menor que esta cantidad.

Resumen de la SUMT

Paso inicial: se identifica una solución de prueba inicial *factible* $\mathbf{x}^{(0)}$ que no esté en la frontera de la región factible. Se hace $k = 1$ y se eligen valores adecuados estrictamente positivos para la r inicial y para $\theta < 1$ (por ejemplo, $r = 1$ y $\theta = 0.01$).²²

Iteración k : si se comienza con $\mathbf{x}^{(k-1)}$, se aplica el procedimiento de búsqueda del gradiente descrito en la sección 12.5 (o un método similar) para encontrar un máximo local, $\mathbf{x}^{(k)}$ de

$$P(\mathbf{x}; r) = f(\mathbf{x}) - r \left[\sum_{i=1}^m \frac{1}{b_i - g_i(\mathbf{x})} + \sum_{j=1}^n \frac{1}{x_j} \right].$$

Regla de detención: si el cambio de $\mathbf{x}^{(k-1)}$ a $\mathbf{x}^{(k)}$ es despreciable, el proceso se detiene y se usa $\mathbf{x}^{(k)}$ (o una extrapolación de $\mathbf{x}^{(0)}, \mathbf{x}^{(1)}, \dots, \mathbf{x}^{(k-1)}, \mathbf{x}^{(k)}$) como la estimación de un máximo local del problema original. De otra manera, se establece $k = k + 1$ y $r = \theta r$ y se realiza otra iteración.

Por último, observe que la técnica SUMT se puede extender para manejar restricciones de igualdad, $g_i(\mathbf{x}) = b_i$. Una forma estándar es la siguiente. Para cada restricción de igualdad,

$$\frac{-[b_i - g_i(\mathbf{x})]^2}{\sqrt{r}} \quad \text{sustituye a} \quad \frac{-r}{b_i - g_i(\mathbf{x})}$$

en la expresión de $P(\mathbf{x}; r)$ que se dio en el resumen de la técnica; después se usa el mismo procedimiento. El numerador, $-[b_i - g_i(\mathbf{x})]^2$, impone una gran penalización para la desviación grande de la satisfacción de la restricción de igualdad, y después el denominador aumenta en forma tremenda esta penalización a medida que r disminuye a una cantidad muy pequeña, lo que hace que la secuencia de soluciones de prueba converjan hacia el punto que satisface la restricción.

La SUMT se utiliza de manera profusa por su simplicidad y versatilidad. Sin embargo, quienes se dedican al análisis numérico han encontrado que es bastante propensa a la *inestabilidad numérica*, por lo que se aconseja tomar precauciones extremas. Para mayor información sobre este tema, igual que sobre análisis similares para otros algoritmos, vea la referencia 3.

Ejemplo Para ilustrar la SUMT considere el siguiente problema de dos variables.

$$\begin{aligned} &\text{Maximizar} && f(\mathbf{x}) = x_1 x_2, \\ &\text{sujeta a} && \\ &&& x_1^2 + x_2 \leq 3 \\ &\text{y} && \\ &&& x_1 \geq 0, \quad x_2 \geq 0. \end{aligned}$$

■ **TABLA 12.7** Ilustración de la SUMT

k	r	$x_1^{(k)}$	$x_2^{(k)}$
0		1	1
1	1	0.90	1.36
2	10^{-2}	0.987	1.925
3	10^{-4}	0.998	1.993
		↓	↓
		1	2

²² Un criterio razonable para elegir la r inicial es la que hace que $rB(\mathbf{x})$ sea de más o menos del mismo orden de magnitud que $f(\mathbf{x})$ para soluciones $\mathbf{x}$ que no se encuentren muy cerca de la frontera.

Aun cuando $g_1(\mathbf{x}) = x_1^2 + x_2$ es convexa (puesto que todos los términos son convexos), se trata de un problema de programación *no convexa* porque $f(\mathbf{x}) = x_1x_2$ *no* es cóncava (vea el apéndice 2). Sin embargo, el problema está tan cerca de ser un problema de programación convexa que necesariamente SUMT convergirá hacia una solución óptima en este caso. (En la siguiente sección se explicará más a fondo la programación no convexa, exposición que incluye el papel de la SUMT al enfrentar este tipo de problemas.)

En el paso inicial, $(x_1, x_2) = (1, 1)$ es una solución obvia que no está en la frontera de la región factible, de manera que se puede establecer $\mathbf{x}^{(0)} = (1, 1)$. Una elección razonable para r y θ es $r = 1$ y $\theta = 0.01$.

Para cada iteración,

$$P(\mathbf{x}; r) = x_1x_2 - r \left(\frac{1}{3 - x_1^2 - x_2} + \frac{1}{x_1} + \frac{1}{x_2} \right).$$

Con $r = 1$, se aplica el procedimiento de búsqueda del gradiente a partir de $(1, 1)$ para maximizar esta expresión, hasta llegar a $\mathbf{x}^{(1)} = (0.90, 1.36)$. Se establece $r = 0.01$ y se comienza de nuevo el procedimiento de búsqueda del gradiente a partir de $(0.90, 1.36)$, y se llega entonces a $\mathbf{x}^{(2)} = (0.983, 1.933)$. Una iteración más con $r = 0.01(0.01) = 0.0001$ conduce de $\mathbf{x}^{(2)}$ a $\mathbf{x}^{(3)} = (0.998, 1.994)$. Es claro que esta sucesión de puntos, que se resume en la tabla 12.7, es convergente a $(1, 2)$. Si se aplican las condiciones KKT a esta solución, se verifica que sin duda satisface la condición necesaria para la optimalidad. Un análisis gráfico demuestra que $(x_1, x_2) = (1, 2)$ es, en realidad, un máximo global (vea el problema 12.9-13b).

En este problema no existen máximos locales distintos de $(x_1, x_2) = (1, 2)$, por lo que si se vuelve a aplicar la técnica SUMT a partir de otras soluciones de prueba factibles iniciales, siempre se llegará a esta misma solución.²³

En la sección Worked Examples del sitio en internet de este libro se proporciona **otro ejemplo** que ilustra la aplicación de la SUMT a un problema de programación convexa en forma de minimización. También en el OR Tutor se puede ver un **ejemplo adicional**. Asimismo, se incluye un procedimiento automático para ejecutar la SUMT.

Algunas opciones de software para programación convexa

Como se indicó en la sección 12.7, tanto Excel como LINGO pueden resolver problemas de programación convexa, no así LINDO y CPLEX, excepto en el caso especial de programación cuadrática (cuyo ejemplo se incluye en esta sección). Los detalles de este ejemplo se dan en los archivos de Excel y LINGO/LINDO para este capítulo en el OR Courseware. La versión profesional de MPL maneja un gran número de solucionadores que incluyen algunos que pueden resolver problemas de programación convexa. Uno de ellos, llamado CONOPT, se incorpora a la versión de MPL que contiene el sitio en internet de este libro. Los ejemplos de programación convexa que se presentan en el archivo de este capítulo de MPL se resolvieron con este solucionador después de establecer el tipo de modelo no lineal (como se describió al final de la sección 12.7).

12.10 PROGRAMACIÓN NO CONVEXA

Los supuestos de programación convexa (la función $f(\mathbf{x})$ que debe maximizarse es *cóncava* y todas las restricciones $g_i(\mathbf{x})$ son *convexas*) son muy convenientes puesto que aseguran que cualquier *máximo local* es también un *máximo global*. (Si, por el contrario, el objetivo es minimizar $f(\mathbf{x})$, entonces la programación convexa supone que $f(\mathbf{x})$ es convexa, lo que asegura que un *mínimo local* también es un *mínimo global*.) Desafortunadamente, los problemas de programación no lineal que surgen en la práctica muchas veces no cumplen estos supuestos. ¿Qué tipo de enfoque conviene usar para manejar estos problemas de *programación no convexa*?

²³ La razón técnica es que $f(\mathbf{x})$ es una función (estrictamente) *cuasicóncava* que comparte la propiedad de las funciones cóncavas de que un máximo local es siempre un máximo global. Más información al respecto se encuentra en M. Avriel, W. E. Diewert, S. Schaible e I. Zang, *Generalized Concavity*, Plenum, Nueva York, 1985.

El reto de resolver problemas de programación no convexa

No existe una respuesta única para la pregunta anterior porque hay muchos tipos diferentes de problemas de programación no convexa. Algunos son mucho más difíciles de resolver que otros. Por ejemplo, un problema de maximización donde la función objetivo está cerca de ser *convexa* por lo general es mucho más difícil de resolver que uno donde la función objetivo es casi cóncava. (El ejemplo de la SUMT en la sección 12.9 ilustró un caso donde la función objetivo estaba tan cerca de ser cóncava que el problema podía ser tratado como si fuera un problema de programación convexa.) De manera similar, al tener una región factible que *no* es un conjunto convexo (porque algunas de las funciones $g_i(\mathbf{x})$ no son convexas) por lo general representa una complicación importante. Al tratar con funciones que no son diferenciables, o quizá ni siquiera continuas, también se presenta una complicación relevante.

La meta de una gran parte de la investigación que se realiza en la actualidad es desarrollar procedimientos eficientes de *optimización global* para encontrar una *solución óptima global* para diferentes tipos de problemas de programación no convexa, y se han hecho algunos progresos. Como ejemplo, LINDO Systems (cuyos procedimientos son LINDO, LINGO y What's Best!) ha incorporado un optimizador global en el solucionador avanzado que comparten algunos de sus productos de software. En particular, LINGO y What's Best! tienen una función de inicio múltiple para generar de manera automática un número de puntos de inicio para su solucionador de programación no lineal con el fin de encontrar con rapidez una buena situación. Si se selecciona la opción global, en seguida estos paquetes emplean el optimizador global. Este optimizador convierte un problema de programación no convexa (incluso aquellos cuya formulación involucra funciones lógicas como IF, AND, OR y NOT) en algunos subproblemas que son solturas de programación convexa de partes del problema original. Después se utiliza la técnica de ramificación y acotamiento para buscar de manera exhaustiva entre los subproblemas. Una vez que el procedimiento llega a su terminación, se garantiza que la función que se encuentre sea una solución óptima global. (La otra conclusión posible es que el problema no tenga soluciones factibles.) La versión para estudiantes de este optimizador global se incluye en la versión de LINGO que se proporciona en el sitio en internet de este libro. Sin embargo, está limitado a problemas relativamente pequeños (con un máximo de cinco variables). La versión profesional del optimizador global ha resuelto algunos problemas mucho más grandes.

De manera similar, en la actualidad MPL incluye un optimizador global llamado LGO. La versión para estudiantes de LGO está disponible como uno de los solucionadores de MPL que se incluye en el sitio en internet de este libro. LGO también se puede usar para resolver problemas de programación convexa.

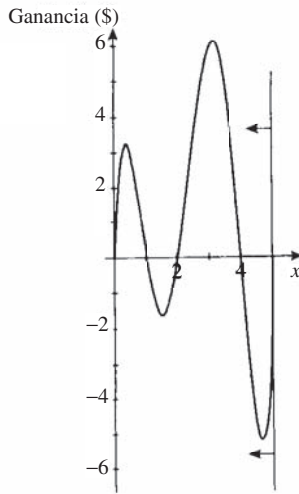
Se ha probado con diversos enfoques para abordar la optimización global (como el incorporado a LINGO que se describe a continuación). No se intentará afrontar este tema avanzado de ninguna forma (vea la referencia 5 para mayores detalles). En lugar de ello se iniciará con un caso simple y después se introducirá un enfoque más general al final de la sección. La metodología se ilustrará mediante el uso de hojas de cálculo de Excel, pero también pueden emplearse otros paquetes de software.

Uso de Excel Solver para encontrar óptimos locales

A continuación se estudiarán los enfoques directos sobre tipos relativamente simples de problemas de programación no convexa. En particular se considerarán problemas (de maximización) donde la función objetivo es casi cóncava ya sea sobre toda la región factible o dentro de porciones significativas de ella. También se ignorará la complejidad adicional que implica tener funciones de restricción $g_i(\mathbf{x})$ no convexas al usar únicamente restricciones lineales. Se comenzará por ilustrar lo que se puede lograr sólo con aplicar algún algoritmo para programación no convexa a dichos problemas. Aunque se podría seleccionar alguno de esos algoritmos (como los descritos en la sección 12.9), se usará el algoritmo de programación convexa que emplea el Solver de Excel para problemas de programación no lineal.

Por ejemplo, considere el siguiente problema de programación no convexa de una variable.

$$\text{Maximizar} \quad Z = 0.5x^5 - 6x^4 + 24.5x^3 - 39x^2 + 20x,$$



■ **FIGURA 12.18**
Gráfica de ganancia de un ejemplo de programación no convexa.

sujeta a

$$x \leq 5$$

$$x \geq 0,$$

donde Z representa la ganancia en dólares. En la figura 12.18 se muestra una gráfica de la ganancia a través de la región factible que señala el elevado grado de no convexidad de esta función. Sin embargo, si esta gráfica no estuviera disponible, podría no ser claro de inmediato que éste no es un problema de programación convexa puesto que se requiere un pequeño análisis para verificar que la función objetivo no es cóncava en la región factible. Por lo tanto, suponga que Excel Solver, que está diseñado para resolver problemas de programación convexa, se aplica a este ejemplo. En la figura 12.19 se demuestra lo difícil que resulta para Excel Solver tratar de resolver este problema. El modelo se puede formular de manera directa en una hoja de cálculo, con x (C5) como la celda cambiante y la Ganancia (C8) como la celda objetivo. [Observe que la opción de Solver, *Assume Linear Model* (Asumir modelo lineal), no está seleccionada en este caso porque éste no es un modelo de programación lineal.] Cuando se introduce $x = 0$ como el valor inicial en la celda cambiante, la hoja de cálculo de la izquierda de la figura 12.19 muestra que el solver indica que $x = 0.371$ es la solución óptima con Ganancia = \$3.19. Sin embargo, si se introduce $x = 3$ como el valor inicial, como se muestra en la hoja de cálculo de en medio de dicha figura, Solver obtiene $x = 3.126$ como la solución óptima con Ganancia = \$6.13. Si se prueba otro valor inicial de $x = 4.7$ en la hoja de cálculo de la derecha, Solver ahora indica una solución óptima de $x = 5$ con Ganancia = \$0. ¿Qué sucede aquí?

En la figura 12.18 se puede encontrar una explicación de las dificultades de Solver con este problema. A partir de $x = 0$, la gráfica de ganancia en realidad asciende hasta un pico en $x = 0.371$, como se reporta en la hoja de cálculo de la izquierda de la figura 12.19. Si en lugar de eso se inicia en $x = 3$, la gráfica sube hasta un pico en $x = 3.126$ que es la solución que se encontró en la hoja de cálculo de en medio. Si se usa la hoja de cálculo de la derecha a partir de la solución de $x = 4.7$, la gráfica asciende hasta que alcanza la frontera impuesta por la restricción de $x \leq 5$, entonces $x = 5$ es el pico en esa dirección. Estos tres picos son los máximos locales (u *óptimos locales*) porque

■ **FIGURA 12.19**

Ejemplo de un problema de programación no convexa (que se muestra en la figura 12.18), donde Excel Solver obtiene tres soluciones diferentes a partir de tres soluciones iniciales distintas.

	A	B	C	D	E
1	Solución de Solver				
2	(A partir de $x = 0$)				
3					
4					Máximo
5		$x =$	0.371	?	5
6					
7		Ganancia = $0.5x^5 - 6x^4 + 24.5x^3 - 39x^2 + 20x$			
8		=	\$3.19		

	A	B	C	D	E
1	Solución de Solver				
2	(A partir de $x = 3$)				
3					
4					Máximo
5		$x =$	3.126		5
6					
7		Ganancia = $0.5x^5 - 6x^4 + 24.5x^3 - 39x^2 + 20x$			
8		=	\$6.13		

	A	B	C	D	E
1	Solución de Solver				
2	(A partir de $x = 4.7$)				
3					
4					Máximo
5		$x =$	5.000	?	5
6					
7		Ganancia = $0.5x^5 - 6x^4 + 24.5x^3 - 39x^2 + 20x$			
8		=	\$0.00		

	B	C
7	Ganancia =	
8	=	$= 0.5 * x^5 - 6 * x^4 + 24.5 * x^3 - 39 * x^2 + 20 * x$

Nombre del rango	Celda
Máximo	E5
x	C5
Ganancia	C8

Parámetros de Solver

Celda objetivo:

Valor de la celda objetivo: ☒ Máximo ☐ Mínimo ☐

Cambio de celdas

Sujetas a las restricciones

Opciones de Solver

☐ Adoptar modelo lineal

☒ Asumir no negativos

cada uno es un máximo de la gráfica dentro de una vecindad local de ese punto. Sin embargo, sólo el más grande de estos máximos locales es el *máximo global*, esto es, el punto más alto de toda la gráfica. Así, la hoja de cálculo de en medio en la figura 12.19 tuvo éxito al encontrar la solución óptima global en $x = 3.126$ con Ganancia = \$6.13.

El Excel Solver aplica el *método del gradiente reducido generalizado*, que adapta el método de búsqueda del gradiente descrito en la sección 12.5 para resolver problemas de programación convexa. Por tanto, este algoritmo se puede pensar como el procedimiento para escalar una montaña. Comienza con la solución inicial que se introdujo en las celdas cambiantes y después comienza a subir esa montaña hasta que llega al pico (o su ascenso se bloquea al llegar a una frontera impuesta por las restricciones). El procedimiento termina cuando alcanza este pico (o frontera) y reporta la solución. No tiene forma de detectar si existe alguna otra montaña más alta en algún lugar de la gráfica de ganancia.

Lo mismo sucedería con cualquier otro procedimiento para escalar una montaña, como la SUMT (que se describió en la sección 12.9), la cual se detiene cuando encuentra un máximo local. Así, si la SUMT se aplicara a este ejemplo con cada una de las tres soluciones de prueba iniciales que se utilizaron en la figura 12.19, se encontrarían los mismos tres máximos locales que encontró el Excel Solver.

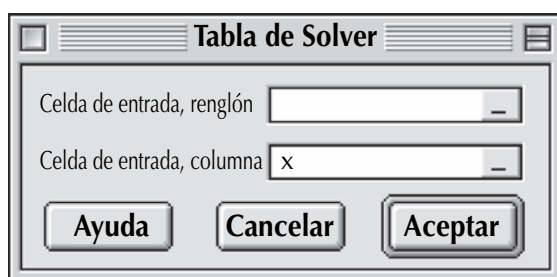
Un enfoque más sistemático para encontrar óptimos locales

Un enfoque usual de los problemas de programación no convexa “fáciles” es aplicar algún procedimiento algorítmico para escalar montañas que se detendrá cuando encuentre un *máximo local* y después reiniciará el algoritmo cierta cantidad de veces desde una variedad de soluciones de prueba iniciales (elegidas ya sea de manera aleatoria o como una sección cruzada sistemática) para encontrar tantos mínimos locales como sea posible. Después, el mejor de estos máximos locales se selecciona para su implantación. Por lo general, un procedimiento para escalar montañas se diseña para encontrar un máximo global cuando se cumplen todos los supuestos de la programación convexa, pero también puede operar para encontrar un máximo local cuando los supuestos no se cumplen.

Cuando se emplea el Solver de Excel, una forma sistemática de aplicar este enfoque es utilizar el complemento de la tabla de Solver que se proporciona en el OR Courseware. Para demostrar su uso se continuará con el modelo en hoja de cálculo que se presentó en la figura 12.19. Por otra parte, en la figura 12.20 se presenta cómo se utiliza la tabla de Solver para probar seis diferentes puntos de inicio (0, 1, 2, 3, 4 y 5) como las soluciones de prueba iniciales para este modelo al ejecutar los siguientes pasos. En el primer renglón de la tabla se introducen las fórmulas que se refieren a la celda cambiante, x (C5), y a la celda objetivo, Ganancia (C8). Los diferentes puntos de inicio se introducen en la primera columna de la tabla (G8:G13). Después se selecciona toda la tabla (G7:I13) y se elige Tabla de Solver de la pestaña Adiciones (en el caso de Excel 2007) o el menú Herramientas (para el caso de versiones anteriores de Excel). La celda de entrada que se especifica en el cuadro de diálogo de la tabla de Solver es la celda cambiante x (C5), puesto que es ahí donde se necesitan los diferentes puntos de inicio en la primera columna de la tabla que se introducirá. (En este cuadro de diálogo no se especifica el renglón de la celda cambiante porque sólo se utiliza una columna para definir los puntos de inicio.) Cuando se hace clic sobre Aceptar, el Solver de Excel resuelve de nuevo el problema para todos estos puntos de inicio en la primera columna y coloca los resultados correspondientes (el máximo local para x y la ganancia que produce en el primer renglón) en las otras columnas de la tabla.

El ejemplo tiene sólo una variable y por ende una sola celda cambiante. Sin embargo, la tabla de Solver también se puede utilizar para probar muchos puntos de inicio de problemas con dos variables (celdas cambiantes). Esto se hace utilizando el primer renglón y la primera columna de la tabla para especificar diferentes puntos de inicio para las dos celdas cambiantes. Se introduce una ecuación referida a la celda objetivo en la esquina superior izquierda de la tabla. Luego, se selecciona la tabla completa y se elige Tabla de Solver del menú Herramientas, con las dos celdas cambiantes seleccionadas como la celda de entrada columna y celda de entrada renglón. Luego, la tabla de Solver resuelve de nuevo el problema de cada combinación de puntos de inicio de las dos celdas cambiantes y llena el cuerpo de la tabla con el valor de la función objetivo que se encontró (un óptimo local) para cada una de estas combinaciones. (Vea la sección 6.8 para mayores detalles acerca de la preparación de la tabla de Solver en dos dimensiones.)

	A	B	C	D	E	F	G	H	I	J
1	Utilización de la tabla de Solver para probar diferentes puntos de inicio									
2										
3										
4					Máximo		Punto de			
5		x=	3.126	≤	5		inicio	Solución		
6							x	x*	Ganancia	
7		Ganancia = $0.5x^5 - 6x^4 + 24.5x^3 - 39x^2 + 20x$						3.126	\$6.13	Seleccione la tabla completa (G7:I13) antes de elegir Tabla de Solver del menú Herramientas
8		=	\$6.13				0	0.371	\$3.19	
9							1	0.371	\$3.19	
10							2	3.126	\$6.13	
11							3	3.126	\$6.13	
12							4	3.126	\$6.13	
13							5	5.000	\$0.00	



	H	I
5	Solución	
6	x*	Ganancia
7	=x	=Ganancia

Nombre de rango Celda	
x	C5
Ganancia	C8

■ FIGURA 12.20

Aplicación de la tabla de Solver (un complemento que se proporciona en el OR Courseware) al ejemplo que se consideró en las figuras 12.18 y 12.19.

Para problemas con más de dos variables (celdas cambiantes), este mismo enfoque se puede seguir usando para probar puntos de inicio múltiples para cualquiera de las dos celdas cambiantes. Sin embargo, este enfoque se convierte en un modo muy incómodo de probar un amplio rango de puntos de inicio para todas las celdas cambiantes cuando hay más de tres o cuatro de estas celdas.

Desafortunadamente, es frecuente que no exista la garantía de encontrar una solución óptima global, sin importar cuántos puntos de inicio se prueben. También, si las gráficas de ganancia no son suaves (por ejemplo, si existen discontinuidades o espirales), entonces Solver puede no ser capaz siquiera de encontrar óptimos locales. Por fortuna, el *Premium Solver* de Excel proporciona otro procedimiento de búsqueda llamado *Evolutionary Solver* para intentar solucionar estos problemas de programación no convexa más complejos.

Evolutionary Solver

Frontline Systems, el desarrollador del Solver estándar que se incluye con Excel, ha desarrollado versiones Premium de Solver. Además, una versión de Premium Solver (Premium Solver for Education) está disponible en el OR Courseware (pero no se incluye con el Excel estándar). Todas las versiones de Premium Solver, entre otras ésta, incluyen un procedimiento de búsqueda llamado **Evolutionary Solver** en el conjunto de herramientas disponible para buscar una solución óptima de un modelo. La filosofía del Evolutionary Solver se basa en la genética, evolución y supervivencia del más apto. Por tanto, este tipo de algoritmo algunas veces se conoce como **algoritmo genético**. La sección 13.4 está dedicada a describir cómo operan los algoritmos genéticos.

El Evolutionary Solver tiene tres ventajas cruciales sobre el Solver estándar (o cualquier otro algoritmo de programación convexa) para resolver problemas de programación no convexa. Primera, la complejidad de la función objetivo no lo afecta. Mientras la función se pueda evaluar para una solución de prueba dada, no importa si la función tiene rizos o discontinuidades o muchos óptimos

locales. Segunda, la complejidad de las restricciones dadas (se incluye restricciones no convexas) tampoco tienen un efecto sustancial en el Evolutionary Solver (aunque el *número* de restricciones sí lo hace). Tercera, como evalúa todas las poblaciones de soluciones de prueba, éstas no están necesariamente en la misma vecindad de la mejor solución de prueba actual. El Evolutionary Solver evita quedar atrapado en un óptimo local. En realidad, en el Evolutionary Solver está garantizado que finalmente se encontrará una solución óptima global para cualquier problema de programación no lineal (incluso problemas de programación no convexa), si éste se corre eternamente (lo que, por supuesto, no es práctico). Por lo tanto, el Evolutionary Solver está bien equipado para tratar con muchos problemas de programación no convexa relativamente pequeños.

Por otro lado, se debe puntualizar que el Evolutionary Solver no es una panacea. Primero, puede tomar *mucho* más tiempo que el Solver estándar para encontrar una solución final. Segundo, el Evolutionary no se desempeña bien con modelos que tienen muchas restricciones. Tercero, el Evolutionary Solver es un proceso aleatorio, por lo que, al correrlo de nuevo sobre un mismo modelo, es usual que se encuentre una solución final diferente. Finalmente, la mejor solución que se encuentre por lo general no es necesariamente la óptima (aunque puede estar muy cerca de serlo). El Evolutionary Solver no se mueve en forma continua hacia mejores soluciones. En lugar de ello es más un dispositivo de búsqueda inteligente, pues prueba distintas soluciones de prueba aleatorias. En consecuencia, mientras es muy probable que termine con una solución muy cercana a la óptima, casi nunca produce la solución óptima global exacta para la mayoría de los tipos de problemas de programación no lineal. En consecuencia, es frecuente obtener beneficios al correr el Solver estándar (GRG opción no lineal) después del Evolutionary Solver, a partir de la solución final que obtuvo este último, para ver si la solución se puede mejorar con una búsqueda alrededor de su vecindad.

12.11 CONCLUSIONES

En la práctica, es frecuente que los problemas de optimización incluyan un comportamiento *no lineal* que debe tomarse en cuenta. A veces es posible *reformular* las no linealidades para que se ajusten al formato de programación lineal, como en los problemas de *programación separable*. Sin embargo, muchas veces es necesario usar formulaciones de *programación no lineal*.

Al contrario del caso del método símplex para programación lineal, no existe un algoritmo eficiente que se pueda utilizar para resolver todos los problemas de programación no lineal. En realidad, algunos de estos problemas no se pueden resolver de modo satisfactorio por ningún método, pero se han hecho grandes progresos en ciertas clases importantes de problemas que incluyen *programación cuadrática*, *programación convexa* y algunos tipos especiales de *programación no convexa*. Se dispone de una gran variedad de algoritmos que casi siempre tienen un buen desempeño en estos casos. Algunos de estos algoritmos incorporan procedimientos de alta eficiencia para la *optimización no restringida* en una parte de cada iteración, y algunos emplean una sucesión de aproximaciones lineales o cuadráticas al problema original.

En los últimos años se ha manifestado un gran interés en el desarrollo de *paquetes de computadora* (software) confiables y de alta calidad para uso general en la aplicación del mejor de estos algoritmos. Por ejemplo, en el Systems Optimization Laboratory de la University of Stanford se han desarrollado varios paquetes poderosos, como el MINOS. Estos paquetes son de uso común en otros centros para la solución de problemas del tipo que se presentó en este capítulo (y de programación lineal). Las considerables mejoras que se han logrado, tanto en los algoritmos como en el software, permiten hoy que algunos problemas grandes estén dentro de la factibilidad computacional.

La investigación en el campo de la programación no lineal sigue muy activa.

REFERENCIAS SELECCIONADAS

1. Fiacco, A. V. y G. P. McCormick, *Nonlinear Programming: Sequential Unconstrained Minimization Techniques*, Classics in Applied Mathematics 4, Society for Industrial and Applied Mathematics, Philadelphia, 1990. (Reimpreso del libro clásico publicado en 1968.)
2. Fletcher, R., *Practical Methods of Optimization*, 2a. ed., Wiley, Nueva York, 2000.

3. Gill, P. E., W. Murray y M. H. Wright, *Practical Optimization*, Academic Press, Londres, 1981.
4. Hillier, F. S. y M. S. Hillier, *Introduction to Management Science: A Modeling and Case Studies Approach with Spreadsheets*, 3a. ed., McGraw-Hill/Irwin, Burr Ridge, IL, 2008, capítulo 8.
5. Leyffer, S. y J. More (eds.), Special Issue on Deterministic Global Optimization and Applications, *Mathematical Programming*, Series B, **103**(2), junio de 2005.
6. Luenberger, D. y Y. Ye, *Linear and Nolinear Programming*, 3a. ed., Springer, Nueva York, 2008.
7. Miller, R. E., *Optimization: Foundations and Applications*, Wiley, Nueva York, 1999.
8. Rardin, D., *Optimization in Operations Research*, Prentice-Hall, Upper Saddle River, NJ, 1998.

■ AYUDAS DE APRENDIZAJE PARA ESTE CAPÍTULO EN EL SITIO EN INTERNET DE ESTE LIBRO (www.mhhe.com/hillier)

Ejemplos resueltos:

Ejemplos para el capítulo 12

Ejemplos de demostración incluidos en el OR Tutor:

Procedimiento de búsqueda del gradiente (*Gradient Search Procedure*)

Algoritmo de Frank-Wolfe (*Frank-Wolfe Algorithm*)

Técnica de minimización secuencial no restringida: SUMT (*Sequential Unconstrained Minimization Technique*)

Rutinas interactivas del IOR Tutorial:

Procedimiento interactivo de búsqueda en una dimensión (*Interactive One-Dimensional Search Procedure*)

Procedimiento de búsqueda del gradiente (*Interactive Gradient Search Procedure*)

Método simplex modificado (*Interactive Modified Simplex Method*)

Algoritmo de Frank-Wolfe (*Interactive Frank-Wolfe Algorithm*)

Rutinas automáticas del IOR Tutorial:

Procedimiento de búsqueda del gradiente (*Automatic Gradient Search Procedure*)

Técnica secuencial de minimización no restringida: SUMT (*Sequential Unconstrained Minimization Technique*)

Complemento de Excel:

Premium Solver for Education

Solver Table

Archivos para resolver los ejemplos, “Ch. 12—Nonlinear Programming”

Archivos de Excel

Archivo de LINGO/LINDO

Archivo de MPL/CPLEX/CONOPT/LGO

Glosario del capítulo 12

Vea en el apéndice 1 la documentación del software.

■ PROBLEMAS

Los símbolos a la izquierda de algunos problemas (o de sus incisos) significan lo siguiente:

- D: El ejemplo de demostración correspondiente que se indicó en las Ayudas de aprendizaje puede ser útil.
- I: Se sugiere usar la rutina interactiva indicada correspondiente que se acaba de listar (la impresión registra su trabajo).
- C: Use la computadora con cualquier software (o según lo indique su profesor) para resolver el problema.

Un asterisco en el número del problema indica que al final del libro se da al menos una respuesta parcial.

12.1-1. Lea el artículo de referencia que describe en su totalidad el estudio de OR que se resume en el recuadro de aplicación que se presentó en la sección 12.1. Describa de manera concisa la forma en que se aplicó la programación lineal a este estudio. Después, elabore una lista de los beneficios financieros y no financieros que arrojó dicho estudio.

12.1-2. Considere el problema de la *mezcla de productos* que se presentó en el problema 3.1-11. Suponga que la empresa encuentra *elasticidad de precios* al vender sus tres productos, de tal suerte que las ganancias son distintas de las que se señalan en el capítulo 3. En particular, suponga que los costos unitarios respectivos de producir los artículos 1, 2 y 3 son de 25, 10 y 15 dólares, y que los precios que se requieren (en dólares) para poder vender x_1 , x_2 y x_3 unidades son $(35 + 100x_1^{-\frac{1}{3}})$, $(15 + 40x_2^{-\frac{1}{4}})$ y $(20 + 50x_3^{-\frac{1}{5}})$, respectivamente.

Formule un modelo de programación no lineal del problema para determinar cuántas unidades de cada producto se deben producir para maximizar la ganancia.

12.1-3. En el problema de la P & T Co., que se describió en la sección 8.1, suponga que existe 10% de descuento en el precio del transporte para las cargas que *sobrepasen* las primeras 40 unidades en cada combinación de enlatadora y almacén. Dibuje figuras como las 12.3 y 12.4 que muestren los costos marginales del transporte de chicharos de la enlatadora 1 al almacén 1. Después, elabore el modelo completo de programación no lineal de este problema.

12.1-4. Un corredor de acciones, Richard Smith, acaba de recibir una llamada de su mejor cliente, Ann Hardy. Ella tiene 50 000 dólares para invertir y quiere usarlos para comprar dos tipos de acciones. La acción 1 es un bono sólido con un potencial de crecimiento respetable y poco riesgo. La acción 2 es más especulativa. Dos boletines de inversionistas afirman que tiene un gran potencial de crecimiento, pero la consideran riesgosa. Ann desea un rendimiento alto pero también tiene una elevada aversión al riesgo. En consecuencia, pide a Richard que analice qué mezcla de inversión de las dos acciones sería adecuada para ella.

Ann está acostumbrada a unidades de miles de dólares y bloques de 1 000 acciones. Con estas unidades, el precio por bloque de la acción 1 es de 20 y de 30 el de la 2. Después de investigar, Richard le presenta las siguientes estimaciones. El rendimiento esperado por bloque de la acción 1 es de 5 y de 10 el de la 2. La covarianza del rendimiento de un bloque de cada acción es de 5.

Sin asignar todavía un valor numérico específico al rendimiento mínimo aceptable, formule un modelo de programación no lineal del problema.

12.2-1. Reconsidere el problema 12.1-2. Verifique que se trata de un problema de programación convexa.

12.2-2. Reconsidere el problema 12.1-4. Demuestre que el modelo formulado es un problema de programación convexa con la prueba dada en el apéndice 2 para demostrar que la función objetivo que se debe minimizar es convexa.

12.2-3. Considere la variación del ejemplo de Wyndor Glass Co., que se representó en la figura 12.5, donde la segunda y tercera restricciones funcionales del problema original (sección 3.1) se sustituyeron por $9x_1^2 + 5x_2^2 \leq 216$. Demuestre que $(x_1, x_2) = (2, 6)$ con $Z = 36$ es óptima mostrando que la recta de la función objetivo, $36 = 3x_1 + 5x_2$ es *tangente* a esta frontera de restricción en $(2, 6)$. (*Sugerencia:* Expresé x_2 en términos de x_1 sobre esta frontera y después obtenga la derivada respecto de x_1 para encontrar la pendiente de la frontera.)

12.2-4. Considere la variación del problema de Wyndor Glass Co., que se presentó en la figura 12.6, donde la función objetivo original (vea la sección 3.1) se sustituyó por $Z = 126x_1 - 9x_1^2 + 182x_2 - 13x_2^2$. Demuestre que $(x_1, x_2) = (\frac{8}{3}, 5)$ con $Z = 857$ es óptima demostrando que la elipse $857 = 126x_1 - 9x_1^2 + 182x_2 - 13x_2^2$ es tangente a la frontera de la restricción $3x_1 + 2x_2 = 18$ en $(\frac{8}{3}, 5)$. (*Sugerencia:* Obtenga x_2 en términos de x_1 en la elipse y después derive la expresión respecto de x_1 para encontrar la pendiente de la elipse.)

12.2-5. Considere la siguiente función:

$$f(x) = 240x - 300x^2 + 10x^3.$$

- a) Utilice la primera y segunda derivadas para encontrar los máximos y mínimos locales de $f(x)$.
- b) Utilice la primera y segunda derivadas para demostrar que $f(x)$ no tiene ni máximo ni mínimo globales porque es no acotada en ambas direcciones.

12.2-6. Demuestre si cada una de las siguientes funciones, es convexa, cóncava o ninguna de las dos.

- a) $f(x) = 10x - x^2$
- b) $f(x) = x^4 + 6x^2 + 12x$
- c) $f(x) = 2x^3 - 3x^2$
- d) $f(x) = x^4 + x^2$
- e) $f(x) = x^3 + x^4$

12.2-7.* Utilice la prueba del apéndice 2 para determinar si cada una de las siguientes funciones es convexa, cóncava o ninguna de las dos.

- a) $f(\mathbf{x}) = x_1x_2 - x_1^2 - x_2^2$
- b) $f(\mathbf{x}) = 3x_1 + 2x_1^2 + 4x_2 + x_2^2 - 2x_1x_2$
- c) $f(\mathbf{x}) = x_1^2 + 3x_1x_2 + 2x_2^2$
- d) $f(\mathbf{x}) = 20x_1 + 10x_2$
- e) $f(\mathbf{x}) = x_1x_2$

12.2-8. Considere la siguiente función:

$$f(\mathbf{x}) = 5x_1 + 2x_2^2 + x_3^2 - 3x_3x_4 + 4x_4^2 + 2x_5^4 + x_5^2 + 3x_5x_6 + 6x_6^2 + 3x_6x_7 + x_7^2.$$

Demuestre que $f(\mathbf{x})$ es convexa; exprésela como suma de funciones de una o dos variables y compruebe (vea el apéndice 2) que todas estas funciones son convexas.

12.2-9. Considere el siguiente problema de programación no lineal:

$$\text{Maximizar } f(\mathbf{x}) = x_1 + x_2,$$

sujeta a

$$x_1^2 + x_2^2 \leq 1$$

y

$$x_1 \geq 0, \quad x_2 \geq 0.$$

- a)** Verifique que se trata un problema de programación convexa.
b) Resuelva el problema gráficamente.

12.2-10. Considere el siguiente problema de programación no lineal:

$$\text{Minimizar } Z = x_1^4 + 2x_2^2,$$

sujeta a

$$x_1^2 + x_2^2 \geq 2.$$

(Sin restricciones de no negatividad.)

- a)** Use el análisis genérico para determinar si la región factible es un conjunto convexo.
b) Ahora utilice álgebra y cálculo para determinar si la región factible es un conjunto convexo.

12.3-1. Reconsidere el problema 12.1-3. Demuestre que se trata de un problema de programación no convexa.

12.3-2. Considere el siguiente problema de optimización restringida:

$$\text{Maximizar } f(x) = -120x + 15x^2 - 10x^3,$$

sujeta a

$$x \geq 0.$$

Utilice sólo la primera y segunda derivadas de $f(x)$ para obtener una solución óptima.

12.3-3. Considere el siguiente problema de programación no lineal:

$$\text{Minimizar } Z = x_1^4 + 2x_1^2 + 2x_1x_2 + 4x_2^2,$$

sujeta a

$$2x_1 + x_2 \geq 10$$

$$x_1 + 2x_2 \geq 10$$

y

$$x_1 \geq 0, \quad x_2 \geq 0.$$

- a)** De los tipos especiales de problemas de programación no lineal que se describieron en la sección 12.3, ¿a cuál o cuáles se puede ajustar este problema? Justifique su respuesta.
b) Ahora suponga que el problema cambia un poco al sustituir las restricciones de no negatividad por $x_1 \geq 1$ y $x_2 \geq 1$. Convierta este nuevo problema en uno equivalente que tenga sólo dos restricciones funcionales, dos variables y dos restricciones de no negatividad.

12.3-4. Considere el siguiente problema de programación geométrica:

$$\text{Minimizar } f(\mathbf{x}) = 2x_1^{-2}x_2^{-1} + x_2^{-2},$$

sujeta a

$$4x_1x_2 + x_1^2x_2^2 \leq 12$$

y

$$x_1 \geq 0, \quad x_2 \geq 0.$$

- a)** Transforme este problema en uno equivalente de programación convexa.
b) Utilice la prueba que se presentó en el apéndice 2 para verificar que el modelo reformulado en el inciso a) es un problema de programación convexa.

12.3-5. Considere el siguiente problema de programación lineal fraccional:

$$\text{Maximizar } f(\mathbf{x}) = \frac{10x_1 + 20x_2 + 10}{3x_1 + 4x_2 + 20},$$

sujeta a

$$\begin{aligned} x_1 + 3x_2 &\leq 50 \\ 3x_1 + 2x_2 &\leq 80 \end{aligned}$$

y

$$x_1 \geq 0, \quad x_2 \geq 0.$$

- a)** Transforme este problema en uno equivalente de programación lineal.
b) Utilice la computadora para resolver el modelo formulado en el inciso a). ¿Cuál es la solución óptima que resulta para el problema original?

12.3-6. Considere las expresiones en notación matricial que se presentaron en la sección 12.7 para la forma general de las condiciones KKT del problema de programación cuadrática. Demuestre que el problema de encontrar una solución factible para estas condiciones es un problema de complementariedad lineal, como el que se presentó en la sección 12.3; para ello, identifique $\mathbf{w}$, $\mathbf{z}$, $\mathbf{q}$ y $\mathbf{M}$ en términos de los vectores y matrices de la sección 12.7.

12.4-1.* Considere el siguiente problema:

$$\text{Maximizar } f(x) = x^3 + 2x - 2x^2 - 0.25x^4.$$

- a)** Aplique el método de bisección para resolver este problema de manera aproximada. Use una tolerancia de error $\epsilon = 0.04$ y cotas iniciales $\underline{x} = 0, \bar{x} = 2.4$.
b) Aplique el método de Newton, con $\epsilon = 0.001$ y $x_1 = 1.2$, en este problema.

12.4-2. Utilice el método de bisección con una tolerancia de error $\epsilon = 0.04$ y las cotas iniciales dadas para obtener de forma interactiva una solución (aproximada) de los problemas:

a) Maximizar $f(x) = 6x - x^2$, con $\underline{x} = 0, \bar{x} = 4.8$.

b) Minimizar $f(x) = 6x + 7x^2 + 4x^3 + x^4$, con $\underline{x} = -4, \bar{x} = 1$.

12.4-3. Considere el siguiente problema:

$$\text{Maximizar } f(x) = 48x^5 + 42x^3 + 3.5x - 16x^6 - 61x^4 - 16.5x^2.$$

- 1 a) Aplique el método de bisección para encontrar una solución (aproximada) a este problema. Emplee una tolerancia de error $\epsilon = 0.08$ y cotas iniciales $\underline{x} = -1$, $\bar{x} = 4$.
- b) Aplique el método de Newton a este problema con $\epsilon = 0.001$ y $x_1 = 1$.

12.4-4. Considere el siguiente problema:

$$\text{Maximizar } f(x) = 10x^3 + 60x - 2x^6 - 3x^4 - 12x^2.$$

- 1 a) Aplique el método de bisección para encontrar una solución (aproximada) a este problema. Emplee una tolerancia de error $\epsilon = 0.07$ y determine las cotas iniciales apropiadas mediante inspección.
- b) Aplique el método de Newton a este problema con $\epsilon = 0.001$ y $x_1 = 1$.

12.4-5. Considere el siguiente problema de programación convexa:

$$\text{Minimizar } Z = x^4 + x^2 - 4x,$$

sujeta a

$$x \leq 2 \quad y \quad x \geq 0.$$

- a) Haga un cálculo sencillo sólo para verificar si la solución óptima está dentro del intervalo $0 \leq x \leq 1$ o de $1 \leq x \leq 2$. (No obtenga la solución óptima para determinar en qué intervalo debe estar.) Explique su lógica.
- 1 b) Use el método de bisección con cotas iniciales $\underline{x} = 0$, $\bar{x} = 2$ y con una tolerancia de error $\epsilon = 0.02$ para obtener, de forma interactiva, una solución (aproximada) de este problema.
- c) Aplique el método de Newton a este problema con $\epsilon = 0.0001$ y $x_1 = 1$.

12.4-6. Considere el problema de maximizar una función diferencial $f(x)$ con una sola variable no restringida x . Sean $\underline{x}_0$ y $\bar{x}_0$ las respectivas cotas inferior y superior válidas sobre el mismo máximo global (si existe). Pruebe las siguientes propiedades generales del método de bisección (como se hizo en la sección 12.4) para intentar resolver un problema de este tipo.

- a) Dadas $\underline{x}_0$, $\bar{x}_0$ y $\epsilon = 0$, la sucesión de soluciones de prueba seleccionadas por la *regla del punto medio* debe *converger* hacia una solución límite. [Sugerencia: Demuestre que $\lim_{n \rightarrow \infty} (\bar{x}_n - \underline{x}_n) = 0$, donde $\bar{x}_n$ y $\underline{x}_n$ son las cotas superior e inferior identificadas en la iteración n .]
- b) Si $f(x)$ es cóncava [de manera que $df(x)/dx$ es una función monótona decreciente de x], entonces la solución límite del inciso a) debe ser un máximo global.
- c) Si $f(x)$ no es cóncava en todas sus partes, pero sería cóncava si su dominio se restringiera al intervalo entre $\underline{x}_0$ y $\bar{x}_0$, entonces la solución límite del inciso a) debe ser un máximo global.
- d) Si $f(x)$ no es cóncava ni siquiera en el intervalo entre $\underline{x}_0$ y $\bar{x}_0$, entonces la solución límite del inciso a) no necesariamente es un máximo global. (Pruebe esto con la construcción de un contraejemplo gráfico.)
- e) Si $df(x)/dx < 0$ para toda x , entonces no existe $\underline{x}_0$. Si $df(x)/dx > 0$ para toda x , entonces no existe $\bar{x}_0$. En cualquier caso, $f(x)$ no tiene un máximo global.
- f) Si $f(x)$ es cóncava y $\lim_{x \rightarrow -\infty} df(x)/dx < 0$, entonces no existe $\underline{x}_0$. Si $f(x)$ es cóncava y $\lim_{x \rightarrow \infty} df(x)/dx > 0$, entonces no existe $\bar{x}_0$. En cualquier caso, $f(x)$ no tiene un máximo global.

12.4-7. Considere el siguiente problema de programación convexa linealmente restringida:

$$\text{Maximizar } f(\mathbf{x}) = 32x_1 + 50x_2 - 10x_2^2 + x_2^3 - x_1^4 - x_2^4,$$

sujeta a

$$3x_1 + x_2 \leq 11$$

$$2x_1 + 5x_2 \leq 16$$

y

$$x_1 \geq 0, \quad x_2 \geq 0.$$

Pase por alto las restricciones y resuelva los problemas de *optimización restringida de una variable*. Utilice cálculo para resolver el problema que incluye a x_1 y use el método de bisección con $\epsilon = 0.001$ y cotas iniciales de 0 y 4, para resolver el problema que incluye a x_2 . Demuestre que la solución que obtenga para (x_1, x_2) satisface todas las restricciones, de manera que en realidad es óptima para el problema original.

12.5-1. Considere el siguiente problema de optimización no restringida:

$$\text{Maximizar } f(\mathbf{x}) = 2x_1x_2 + x_2 - x_1^2 - 2x_2^2.$$

- D,1 a) Si se inicia con la solución de prueba inicial $(x_1, x_2) = (1, 1)$, aplique de manera interactiva el procedimiento de búsqueda del gradiente con $\epsilon = 0.25$ para obtener una solución aproximada.
- b) Resuelva el sistema de ecuaciones lineales que obtenga con $\nabla f(\mathbf{x}) = \mathbf{0}$ para llegar a la solución exacta.
- c) Con referencia a la figura 12.14 como ejemplo de un problema similar, dibuje la trayectoria de las soluciones de prueba del inciso a). Después muestre la *continuación* aparente de esta trayectoria con la mejor estimación de las tres soluciones de prueba que siguen [según el patrón del inciso a) y la figura 12.14]. También muestre la solución exacta del inciso b) hacia la que converge esta sucesión de soluciones de prueba.
- c d) Aplique a este problema la rutina automática del procedimiento de búsqueda del gradiente en el IOR Tutorial (con $\epsilon = 0.01$).

D,1,C **12.5-2.** A partir de la solución de prueba inicial $(x_1, x_2) = (1, 1)$, haga dos iteraciones interactivas del procedimiento de búsqueda del gradiente para comenzar la solución del siguiente problema y aplique la rutina automática para desarrollar este procedimiento (con $\epsilon = 0.01$).

$$\text{Maximizar } f(\mathbf{x}) = 60x_1x_2 - 15x_1^2 - 80x_2^2.$$

Después resuelva $\nabla f(\mathbf{x}) = \mathbf{0}$ directamente para obtener la solución exacta.

D,1,C **12.5-3.*** A partir de la solución de prueba inicial $(x_1, x_2) = (0, 0)$, aplique el procedimiento interactivo de búsqueda del gradiente con $\epsilon = 0.3$ para obtener una solución aproximada del siguiente problema y después la rutina automática para desarrollar este procedimiento (con $\epsilon = 0.01$).

$$\text{Maximizar } f(\mathbf{x}) = 8x_1 - x_1^2 - 12x_2 - 2x_2^2 + 2x_1x_2.$$

Después resuelva $\nabla f(\mathbf{x}) = \mathbf{0}$ directamente para obtener la solución exacta.

D.I.C 12.5-4. A partir de la solución de prueba inicial $(x_1, x_2) = (0, 0)$, aplique dos iteraciones interactivas del procedimiento de búsqueda del gradiente y después la rutina automática para desarrollar este procedimiento (con $\epsilon = 0.01$).

$$\text{Maximizar } f(\mathbf{x}) = 6x_1 + 2x_1x_2 - 2x_2 - 2x_1^2 - x_2^2.$$

Después resuelva $\nabla f(\mathbf{x}) = \mathbf{0}$ directamente para obtener la solución exacta.

12.5-5. A partir de la solución de prueba inicial $(x_1, x_2) = (0, 0)$ aplique a mano una iteración del procedimiento de búsqueda del gradiente al siguiente problema:

$$\text{Maximizar } f(\mathbf{x}) = 4x_1 + 2x_2 + x_1^2 - x_1^4 - 2x_1x_2 - x_2^2.$$

Para completar esta iteración, obtenga una aproximación de t^* mediante la aplicación de *dos* iteraciones del método de bisección con cotas iniciales $\underline{t} = 0, \bar{t} = 1$.

12.5-6. Considere el siguiente problema de optimización no restringida:

$$\text{Maximizar } f(\mathbf{x}) = 3x_1x_2 + 3x_2x_3 - x_1^2 - 6x_2^2 - x_3^2.$$

a) Describa cómo se puede reducir la solución de este problema a la solución de uno de optimización no restringida con *dos variables*.

D.I b) A partir de la solución de prueba inicial $(x_1, x_2, x_3) = (1, 1, 1)$, aplique el procedimiento interactivo de búsqueda del gradiente con $\epsilon = 0.05$ para obtener una solución (aproximada) del problema con dos variables identificado en el inciso a).

c) Repita el inciso b) con la rutina automática para desarrollar este procedimiento (con $\epsilon = 0.005$).

D.I.C 12.5-7.* A partir de la solución de prueba inicial $(x_1, x_2) = (0, 0)$, aplique el *procedimiento interactivo de búsqueda del gradiente* con $\epsilon = 1$ para obtener una solución (aproximada) para el siguiente problema y luego aplique la rutina automática para desarrollar este procedimiento (con $\epsilon = 0.01$).

$$\text{Maximizar } f(\mathbf{x}) = x_1x_2 + 3x_2 - x_1^2 - x_2^2.$$

12.6-1. Reconsidere el modelo de programación convexa de una variable que se plantea en el problema 12.4-5. Utilice las condiciones KKT para derivar una solución óptima.

12.6-2. Reconsidere el problema 12.2-9. Use las condiciones KKT para verificar si $(x_1, x_2) = (1/\sqrt{2}, 1/\sqrt{2})$ es óptima.

12.6-3.* Reconsidere el modelo del problema 12.3-3. ¿Cuáles son las condiciones de este modelo? Úselas para determinar si $(x_1, x_2) = (0, 10)$ puede ser óptima.

12.6-4. Considere el siguiente problema de programación convexa.

$$\text{Maximizar } f(\mathbf{x}) = 12x_1 - x_1^2 + 50x_2 - x_2^2,$$

sujeta a

$$x_1 \leq 10,$$

$$x_2 \leq 15,$$

y

$$x_1 \geq 0, \quad x_2 \geq 0.$$

a) Utilice las condiciones KKT de este problema para derivar una solución óptima.

b) Descomponga este problema en dos problemas de optimización restringida separados que incluyan sólo x_1 y sólo x_2 . Grafique la función objetivo de cada uno sobre la región factible para *demostrar* que el valor de x_1 o de x_2 que obtuvo en el inciso a) es óptimo. Después *pruebe* que este valor es óptimo si se usa nada más la primera y segunda derivadas de la función objetivo y las restricciones de los respectivos problemas.

12.6-5. Considere el siguiente problema de optimización linealmente restringida:

$$\text{Maximizar } f(\mathbf{x}) = \ln(x_1 + 1) - x_2^2,$$

sujeta a

$$x_1 + 2x_2 \leq 3$$

y

$$x_1 \geq 0, \quad x_2 \geq 0,$$

donde $\ln$ denota el logaritmo natural.

a) Verifique que se trata de un problema de programación convexa.

b) Utilice las condiciones KKT para derivar una solución óptima.

c) Utilice un razonamiento intuitivo para demostrar que la solución obtenida en b) en realidad es óptima.

12.6-6.* Considere el problema de programación no lineal que se presentó con el número 10.3-11. Determine si $(x_1, x_2) = (1, 2)$ puede ser óptima si se aplican las condiciones KKT.

12.6-7. Considere el siguiente problema de programación no lineal:

$$\text{Maximizar } f(\mathbf{x}) = \frac{x_1}{x_2 + 1},$$

sujeta a

$$x_1 - x_2 \leq 2$$

y

$$x_1 \geq 0, \quad x_2 \geq 0.$$

a) Use las condiciones KKT para demostrar que $(x_1, x_2) = (4, 2)$ *no* es óptima.

b) Derive una solución que cumpla las condiciones KKT.

c) Demuestre que este problema no es de programación convexa.

d) A pesar de la conclusión del inciso c), utilice un razonamiento *intuitivo* para demostrar que la solución que se obtuvo en el inciso b) es óptima. [La razón teórica es que $f(\mathbf{x})$ es *seudocóncava*.]

e) Use el hecho de que éste es un problema de programación lineal fraccional para transformarlo en uno equivalente de programación lineal. Resuelva el último e identifique la solución óptima del problema original. (*Sugerencia*: Use la restricción de igualdad en el problema de programación lineal para sustituir una variable del modelo y luego resuelva este modelo en forma gráfica.)

12.6-8.* Use las condiciones KKT para derivar una solución óptima en cada uno de los siguientes problemas.

$$\text{a) Maximizar } f(\mathbf{x}) = x_1 + 2x_2 - x_2^3,$$

sujeta a

$$x_1 + x_2 \leq 1$$

y

$$x_1 \geq 0, \quad x_2 \geq 0.$$

b) Maximizar $f(\mathbf{x}) = 20x_1 + 10x_2$,

sujeta a

$$x_1^2 + x_2^2 \leq 1$$

$$x_1 + 2x_2 \leq 2$$

y

$$x_1 \geq 0, \quad x_2 \geq 0.$$

12.6-9. ¿Cuáles son las condiciones KKT aplicables a problemas de programación no lineal de la forma siguiente?

$$\text{Minimizar} \quad f(\mathbf{x}),$$

sujeta a

$$g_i(\mathbf{x}) \geq b_i, \quad \text{para } i = 1, 2, \dots, m$$

y

$$\mathbf{x} \geq \mathbf{0}.$$

(Sugerencia: Convierta esta forma a nuestra forma estándar, que en este capítulo quedó implícita, mediante las técnicas que se presentaron en la sección 4.6; después, aplique las condiciones KKT como se dan en la sección 12.6.)

12.6-10. Considere el siguiente problema de programación no lineal:

$$\text{Minimizar} \quad Z = 2x_1^2 + x_2^2,$$

sujeta a

$$x_1 + x_2 = 10$$

y

$$x_1 \geq 0, \quad x_2 \geq 0.$$

a) De los tipos especiales de problemas de programación no lineal descritos en la sección 12.3, ¿a cuál o a cuáles se puede ajustar este problema? Justifique su respuesta. (Sugerencia: Primero convierta este problema en uno equivalente de programación no lineal que se ajuste a la forma dada en el segundo párrafo del capítulo, con $m = 2$ y $n = 2$.)

b) Obtenga las condiciones KKT aplicables a este problema.

c) Use las condiciones KKT para derivar una solución óptima.

12.6-11. Considere el siguiente problema de programación linealmente restringida:

$$\text{Minimizar} \quad f(\mathbf{x}) = x_1^3 + 4x_2^2 + 16x_3,$$

sujeta a

$$x_1 + x_2 + x_3 = 5$$

y

$$x_1 \geq 1, \quad x_2 \geq 1, \quad x_3 \geq 1.$$

a) Convierta este problema en uno de programación no lineal equivalente que se ajuste a la forma que se dio al principio del capítulo (segundo párrafo), con $m = 2$ y $n = 3$.

b) Utilice la forma que obtuvo en el inciso **a)** para construir las condiciones KKT de este problema.

c) Utilice las condiciones KKT para verificar si $(x_1, x_2, x_3) = (2, 1, 2)$ es óptima.

12.6-12. Considere el siguiente problema de programación convexa linealmente restringida:

$$\text{Minimizar} \quad Z = x_1^2 - 6x_1 + x_2^3 - 3x_2,$$

sujeta a

$$x_1 + x_2 \leq 1$$

y

$$x_1 \geq 0, \quad x_2 \geq 0.$$

a) Obtenga las condiciones KKT de este problema.

b) Utilice las condiciones KKT para verificar si $(x_1, x_2) = (\frac{1}{2}, \frac{1}{2})$ es una solución óptima.

c) Utilice las condiciones KKT para derivar una solución óptima.

12.6-13. Considere el siguiente problema de programación convexa linealmente restringida:

$$\text{Maximizar} \quad f(\mathbf{x}) = 8x_1 - x_1^2 + 2x_2 + x_3,$$

sujeta a

$$x_1 + 3x_2 + 2x_3 \leq 12$$

y

$$x_1 \geq 0, \quad x_2 \geq 0, \quad x_3 \geq 0.$$

a) Use las condiciones KKT para demostrar que $(x_1, x_2, x_3) = (2, 2, 2)$ no es una solución óptima.

b) Utilice las condiciones KKT para derivar una solución óptima. (Sugerencia: Haga un análisis intuitivo preliminar para determinar el caso más prometedor sobre las variables que son diferentes de cero y las que son iguales a cero.)

12.6-14. Utilice las condiciones KKT para determinar si $(x_1, x_2, x_3) = (1, 1, 1)$ puede ser óptima para el problema siguiente:

$$\text{Minimizar} \quad Z = 2x_1 + x_2^3 + x_3^2,$$

sujeta a

$$x_1^2 + 2x_2^2 + x_3^2 \geq 4$$

y

$$x_1 \geq 0, \quad x_2 \geq 0, \quad x_3 \geq 0.$$

12.6-15. Reconsidere el modelo que se presentó en el problema 12.2-10. ¿Cuáles son las condiciones KKT de este problema? Utilice estas condiciones para determinar si $(x_1, x_2) = (1, 1)$ puede ser óptima.

12.6-16. Reconsidere el modelo de programación convexa linealmente restringida del problema 12.4-7. Use las condiciones KKT para determinar si $(x_1, x_2) = (2, 2)$ puede ser óptima.

12.7-1. Considere el problema de programación cuadrática que se presentó en la sección 12.7.

- a) Utilice la prueba que se dio en el apéndice 2 para demostrar que la función objetivo es *estrictamente cóncava*.
- b) Verifique que la función objetivo es estrictamente cóncava al demostrar que $\mathbf{Q}$ es una matriz *positiva definida*; esto es, $\mathbf{x}^T \mathbf{Q} \mathbf{x} > 0$ para toda $\mathbf{x} \neq \mathbf{0}$. (Sugerencia: Reduzca $\mathbf{x}^T \mathbf{Q} \mathbf{x}$ a una suma de cuadrados.)
- c) Demuestre que $x_1 = 12$, $x_2 = 9$ y $u_1 = 3$ satisfacen las condiciones KKT cuando se escriben en la forma que se dio en la sección 12.6.

12.7-2.* Considere el siguiente problema de programación cuadrática.

$$\text{Maximizar } f(\mathbf{x}) = 8x_1 - x_1^2 + 4x_2 - x_2^2,$$

sujeta a

$$x_1 + x_2 \leq 2$$

y

$$x_1 \geq 0, \quad x_2 \geq 0.$$

- a) Utilice las condiciones KKT para derivar una solución óptima.
- b) Ahora suponga que este problema debe resolverse por el método simplex modificado. Formule el problema de programación lineal que se va a manejar en forma explícita y después identifique la restricción de complementariedad adicional a la que obliga el algoritmo de manera automática.
- c) Aplique el método simplex modificado al problema que se formuló en el inciso b).
- d) Utilice la computadora para resolver el problema de programación cuadrática en forma directa.

12.7-3. Considere el siguiente problema de programación cuadrática.

$$\text{Maximizar } f(\mathbf{x}) = 250x_1 - 25x_1^2 + 100x_2 - 100x_2^2 + 90x_1x_2,$$

sujeta a

$$20x_1 + 5x_2 \leq 90$$

$$10x_1 + 10x_2 \leq 60$$

y

$$x_1 \geq 0, \quad x_2 \geq 0.$$

Suponga que este problema se debe resolver por el método simplex modificado.

- a) Formule el problema de programación lineal que se debe manejar en forma explícita; después identifique la restricción de complementariedad adicional a la que obliga automáticamente el algoritmo.
- b) Aplique el método simplex modificado al problema que se formuló en el inciso a).

12.7-4. Considere el siguiente problema de programación cuadrática.

$$\text{Maximizar } f(\mathbf{x}) = 2x_1 + 3x_2 - x_1^2 - x_2^2,$$

sujeta a

$$x_1 + x_2 \leq 2$$

y

$$x_1 \geq 0, \quad x_2 \geq 0.$$

- a) Utilice las condiciones KKT para derivar una solución óptima de manera directa.
- b) Ahora suponga que este problema debe resolverse por el método simplex modificado. Formule el problema de programación lineal que se debe manejar en forma explícita y después identifique la restricción de complementariedad adicional a la que obliga el algoritmo.
- c) Sin aplicar el método simplex modificado, demuestre que la solución que obtuvo en el inciso a) es óptima ($Z = 0$) para el problema equivalente que se formuló en el inciso b).
- d) Aplique el método simplex modificado al problema que se presentó en el inciso b).
- e) Use la computadora para resolver el problema de programación cuadrática.

12.7-5. Reconsidere la primera variación de programación cuadrática del problema de la Wyndor Glass Co., que se presentó en la sección 12.2 (vea la figura 12.6). Analice este problema de acuerdo con las instrucciones de los incisos a), b) y c) del problema 12.7-4.

c 12.7-6. Reconsidere el problema 12.1-4 y su modelo de programación cuadrática.

- a) Presente este modelo [y los valores de $R(\mathbf{x})$ y $V(\mathbf{x})$] en una hoja de cálculo de Excel.
- b) Resuelva cuatro casos de este modelo: rendimiento esperado mínimo aceptable = 13, 14, 15, 16.
- c) Para distribuciones de probabilidad típicas (con media μ y varianza σ^2) del rendimiento total de una carrera, la probabilidad de que el rendimiento exceda $\mu - \sigma$ es alta (de 0.8 a 0.9) y la probabilidad de que exceda $\mu - 3\sigma$ es muy alta (muy cercana a 0.999). Calcule $\mu - \sigma$ y $\mu - 3\sigma$ de los cuatro portafolios que obtuvo en el inciso b). ¿Cuál cartera dará la μ más alta entre las que también cumplen que $\mu - \sigma \geq 0$?

12.8-1. La corporación MFG planea fabricar y vender tres productos. Sea x_1 , x_2 y x_3 el número de unidades de cada producto que debe fabricarse. Las estimaciones preliminares de la ganancia potencial son las siguientes.

La ganancia unitaria de las primeras 15 unidades fabricadas del producto 1 será de aproximadamente 500 dólares. La ganancia unitaria de cualquier unidad adicional será de sólo 60 dólares. Se estima que la ganancia unitaria de las primeras 20 unidades del producto 2 será de 400 dólares, las siguientes 20 unidades tendrán una ganancia de 200 dólares y será de 100 dólares la de cualquier unidad adicional. La ganancia unitaria de las primeras 20 unidades del producto 3 será de 600 dólares, la de las siguientes diez unidades disminuirá a 400 dólares y a 200 dólares la de cualquier unidad adicional.

Ciertas limitaciones en el uso de los recursos necesarios imponen las siguientes restricciones de fabricación:

$$2x_1 + 3x_2 + 4x_3 \leq 180$$

$$3x_1 + x_2 \leq 150$$

$$x_1 + 3x_3 \leq 100$$

La administración desea saber qué valores de x_1 , x_2 y x_3 deben elegirse a fin de maximizar la ganancia total.

- a) Grafique la ganancia de cada producto.

- b) Use programación separable para formular un modelo de programación lineal para representar el problema.
- c) Resuelva el modelo. ¿Qué se recomienda a la administración sobre los valores de x_1 , x_2 y x_3 ?
- d) Ahora suponga que existe una restricción adicional sobre la ganancia de los productos 1 y 2, que debe dar un total de 20 000 dólares. Utilice la técnica que se presentó en la subsección de “Extensiones” de la sección 12.8 para agregar esta restricción al modelo que se formuló en el inciso b).
- e) Repita el inciso c) para el modelo que se formuló en d).

12.8-2.* La Dorwyn Company tiene dos productos nuevos para competir con los dos productos nuevos de la Wyndor Glass (sección 3.1). Con unidades de cientos de dólares en la función objetivo, se formuló el siguiente modelo de programación lineal para determinar la mezcla de productos más redituable.

$$\text{Maximizar} \quad Z = 4x_1 + 6x_2,$$

sujeta a

$$x_1 + 3x_2 \leq 8$$

$$5x_1 + 2x_2 \leq 14$$

y

$$x_1 \geq 0, \quad x_2 \geq 0.$$

Sin embargo, debido a la fuerte competencia de la Wyndor, la administración de Dorwyn sabe que debe hacer un gran esfuerzo para generar grandes ventas de sus productos. En particular, estima que lograr una tasa de producción y ventas de x_1 unidades del producto 1 por semana requerirá un costo semanal de comercialización de x_1^3 cientos de dólares. Los costos de comercialización correspondientes del producto 2 se estiman en $2x_2^2$ cientos de dólares. En consecuencia, la función objetivo del modelo debe ser $Z = 4x_1 + 6x_2 - x_1^3 - 2x_2^2$.

La administración de Dorwyn desea usar el modelo revisado para determinar la mezcla más redituable.

- a) Verifique que $(x_1, x_2) = (2/\sqrt{3}, \frac{3}{2})$ es una solución óptima aplicando las condiciones KKT.
- b) Construya tablas con los datos de ganancias de cada producto con tasas de producción de 0, 1, 2, 3.
- c) Dibuje una figura como la 12.15b con los puntos de ganancia semanales de cada producto cuando la tasa de producción es 0, 1, 2, 3. Conecte los pares de puntos consecutivos con líneas punteadas.
- d) Con programación separable y la figura formule un modelo aproximado de programación lineal.
- e) Resuelva el modelo. ¿Qué le indica el resultado a la administración de Dorwyn sobre la mezcla de productos?

12.8-3. La compañía J. B. Jensen se especializa en la producción de sierras eléctricas para uso doméstico. Las ventas son estables durante el año y suben durante la temporada de Navidad. Como el trabajo de producción requiere de mucha experiencia, la compañía mantiene un nivel de empleados estable y usa tiempo extra para aumentar la producción en noviembre. Los trabajadores aprecian esta oportunidad de ganar dinero extra para las fiestas.

B. J. Jensen, Jr., el presidente actual, estudia los planes de producción para el próximo noviembre y cuenta con los siguientes datos.

	Producción mensual máxima*		Utilidad por unidad producida	
	Tiempo normal	Horas extra	Tiempo normal	Horas extras
Sierras	12 000	8 000	\$240	\$80
Taladros	20 000	12 000	\$160	\$120

* Se supone que los suministros de materiales de los proveedores son adecuados.

Sin embargo, el Sr. Jensen ahora sabe que, además del número limitado de horas de mano de obra, otros dos factores limitarán los niveles de producción en noviembre. Uno es que los proveedores de los motores sólo podrán entregar 40 000 unidades para noviembre (8 000 más que la entrega usual). Cada sierra y cada taladro requieren una unidad. Segundo, el proveedor de una parte clave de los ensambles de engranes podrá surtir 60 000 para noviembre (16 000 más que otros meses). Cada sierra requiere dos de estas partes, y cada taladro, una.

El Sr. Jensen desea determinar cuántas sierras y taladros eléctricos debe producir en noviembre para maximizar la ganancia total de la compañía.

- a) Grafique la ganancia de cada producto.
- b) Use programación separable para formular un modelo de programación lineal para representar el problema.
- c) Resuelva el modelo. ¿Qué indica la solución sobre cuántas sierras y taladros eléctricos se deben producir en noviembre?

12.8-4. Reconsidere el modelo de programación convexa linealmente restringida del problema 12.4-7.

- a) Use la técnica de programación separable de la sección 12.8 para formular un modelo de programación lineal aproximado. Use $x_1 = 0, 1, 2, 3$ y $x_2 = 0, 1, 2, 3$ como puntos de salto de las funciones lineales por partes.
- b) Utilice el método símplex para resolver el modelo formulado en a). Después reescriba esta solución en términos de las variables *originales* del problema.

12.8-5. Suponga que se ha aplicado la técnica de programación separable a cierto problema (el “problema original”) para convertirlo en el siguiente problema equivalente de programación lineal:

$$\text{Maximizar} \quad Z = 5x_{11} + 4x_{12} + 2x_{13} + 4x_{21} + x_{22},$$

sujeta a

$$3x_{11} + 3x_{12} + 3x_{13} + 2x_{21} + 2x_{22} \leq 25$$

$$2x_{11} + 2x_{12} + 2x_{13} - x_{21} - x_{22} \leq 10$$

y

$$0 \leq x_{11} \leq 2 \quad 0 \leq x_{21} \leq 3$$

$$0 \leq x_{12} \leq 3 \quad 0 \leq x_{22} \leq 1.$$

$$0 \leq x_{13}$$

¿Cuál era el modelo matemático del problema original? (Defina la función objetivo de manera algebraica o gráfica, pero exprese las restricciones en forma algebraica.)

12.8-6. Pruebe que, en cada uno de los siguientes casos, debe cumplirse la propiedad esencial de programación separable que se expuso

en la sección 12.8. (*Sugerencia:* Suponga que existe una solución óptima que viola esta propiedad y después contradiga esta suposición demostrando que existe una solución factible mejor.)

- a) El caso especial donde las $g_i(\mathbf{x})$ son funciones lineales.
- b) El caso general de programación separable donde todas las funciones son no lineales de la forma indicada. [*Sugerencia:* Considere las restricciones funcionales como restricciones sobre los recursos, donde $g_{ij}(x_j)$ representa la cantidad empleada del recurso i al realizar la actividad j en el nivel x_j ; después, utilice la implicación del supuesto de convexidad sobre las pendientes de las funciones lineales por partes.]

12.8-7. La compañía MFG produce cierto subensamble en cada una de dos plantas. Estos subensambles se llevan a una tercera planta cercana donde se usan para fabricar determinado producto. Se acerca la temporada pico de la demanda de este producto, y para mantener las tasas de producción dentro de un margen deseado, es necesario usar temporalmente horas extra para hacer los subensambles. En la tabla que sigue se muestra el costo de cada uno de ellos en tiempo normal (TN) y en horas extra (HE) de ambas plantas, y el máximo número de subensambles que se puede producir en TN y en HE al día.

	Costo unitario		Capacidad	
	TN	HE	TN	HE
Planta 1	\$23	\$38	6 000	3 000
Planta 2	\$24	\$36	3 000	1 500

Sean x_1 y x_2 el número total de subensambles que se fabrica al día en las plantas 1 y 2, respectivamente. Suponga que el objetivo es maximizar $Z = x_1 + x_2$ sujeta a la restricción de que el costo diario total no puede exceder de 270 000 dólares. Observe que la formulación de programación matemática de este problema (con x_1 y x_2 como variables de decisión) tiene la misma forma que el caso principal de programación separable descrito en la sección 12.8, excepto que las funciones separables aparecen en una función de restricción y no en la función objetivo. No obstante, si se permite usar HE cuando aún la capacidad de TN en esa planta no está agotada, es posible usar el mismo enfoque para reformular el problema como uno de programación lineal.

- a) Formule el modelo de programación lineal.
- b) Explique por qué en este caso también se puede aplicar la lógica de programación separable para garantizar que una solución óptima del modelo formulado en el inciso a) nunca emplee HE, a menos que la capacidad de TN en esa planta esté agotada.

12.8-8. Considere el siguiente problema de programación no lineal

$$\text{Maximizar } Z = 5x_1 + x_2,$$

sujeta a

$$2x_1^2 + x_2 \leq 13$$

$$x_1^2 + x_2 \leq 9$$

y

$$x_1 \geq 0, \quad x_2 \geq 0.$$

- a) Demuestre que éste es un problema de programación convexa.
- b) Utilice la técnica de programación separable de la sección 12.8 para formular un modelo aproximado de programación lineal.

Use los enteros como puntos de salto de la función lineal por partes.

- c) Utilice el método simplex para resolver el modelo que formuló en b); después, reexpresé esta solución en términos de las variables originales del problema.

12.8-9. Considere el siguiente problema de programación convexa:

$$\text{Maximizar } Z = 32x_1 - x_1^4 + 4x_2 - x_2^2,$$

sujeta a

$$x_1^2 + x_2^2 \leq 9$$

y

$$x_1 \geq 0, \quad x_2 \geq 0.$$

- a) Aplique la técnica de programación separable que se presentó en la sección 12.8, con $x_1 = 0, 1, 2, 3$ y $x_2 = 0, 1, 2, 3$ como puntos de salto de las funciones lineales por partes para formular un modelo de programación lineal aproximado de este problema.
- b) Utilice el método simplex para resolver el modelo que formuló en a). Después reescriba esta solución en términos de las variables *originales* del problema.
- c) Aplique las condiciones KKT para determinar si la solución de las variables originales de b) es óptima para el problema original (no para el modelo aproximado).

12.8-10. Reconsidere el modelo de programación no lineal entera del problema 10.3-9.

- a) Demuestre que la función objetivo no es cóncava.
- b) Formule un modelo de programación *lineal* entera *binaria pura* para representar este problema, como sigue. Aplique la técnica de programación separable con los enteros como puntos de salto de las funciones lineales por partes, de forma que las variables auxiliares sean variables binarias. Después agregue restricciones de programación lineal sobre estas variables para reforzar la *restricción especial* de programación separable. (Observe que la *propiedad clave* de programación separable no se cumple porque la función objetivo no es cóncava.)
- c) Utilice la computadora para resolver este problema según lo formuló en b). Después reescriba esta solución en términos de las variables *originales* del problema.

D.1 12.9-1. Reconsidere el modelo de programación convexa linealmente restringida del problema 12.6-5. A partir de la solución de prueba inicial $(x_1, x_2) = (0, 0)$, realice una iteración del algoritmo de Frank-Wolfe para obtener exactamente la misma solución que encontró en el inciso b) del problema 12.6-5 y después ejecute una segunda iteración para verificar que se trata de una solución óptima (porque es una réplica exacta).

D.1 12.9-2. Reconsidere el modelo de programación convexa linealmente restringida del problema 12.6-12. A partir de la solución de prueba inicial $(x_1, x_2) = (0, 0)$, realice una iteración del algoritmo de Frank-Wolfe para obtener exactamente la misma solución que encontró en el inciso c) del problema 12.6-12 y después lleve a cabo una segunda iteración para verificar que se trata de una solución óptima (porque es una réplica exacta). Explique por qué se obtendrían los mismos resultados en estas dos iteraciones con cualquier otra solución de prueba.

D,I **12.9-3.** Reconsidere el modelo de programación convexa linealmente restringida del problema 12.6-13. Comience con la solución de prueba inicial $(x_1, x_2, x_3) = (0, 0, 0)$ y luego realice dos iteraciones del algoritmo de Frank-Wolfe.

D,I **12.9-4.** Considere el ejemplo de programación cuadrática que se presentó en la sección 12.7. A partir de la solución de prueba inicial $(x_1, x_2) = (5, 5)$, aplique ocho iteraciones del algoritmo de Frank-Wolfe.

12.9-5. Reconsidere el modelo de programación cuadrática del problema 12.7-4.

D,I **a)** Comience con la solución de prueba inicial $(x_1, x_2) = (0, 0)$ utilice el algoritmo de Frank-Wolfe (seis iteraciones) para resolver el problema (aproximado).

b) Grafique la forma en que se puede extrapolar la sucesión de soluciones de prueba que obtuvo en *a)* para obtener una aproximación más cercana a una solución óptima. ¿Qué estimación de esta solución obtuvo?

D,I **12.9-6.** Reconsidere el modelo de programación convexa linealmente restringida del problema 12.4-7. A partir de la solución de prueba inicial $(x_1, x_2) = (0, 0)$, utilice el algoritmo de Frank-Wolfe (cuatro iteraciones) para resolver este modelo (aproximado).

D,I **12.9-7.** Considere el siguiente problema de programación convexa linealmente restringida:

$$\begin{aligned} \text{Maximizar} \quad f(\mathbf{x}) &= 3x_1x_2 + 40x_1 + 30x_2 - 4x_1^2 - x_1^4 \\ &\quad - 3x_2^2 - x_2^4, \end{aligned}$$

sujeta a

$$\begin{aligned} 4x_1 + 3x_2 &\leq 12 \\ x_1 + 2x_2 &\leq 4 \end{aligned}$$

y

$$x_1 \geq 0, \quad x_2 \geq 0.$$

A partir de la solución de prueba inicial $(x_1, x_2) = (0, 0)$, aplique dos iteraciones del algoritmo de Frank-Wolfe.

D,I **12.9-8.*** Considere el siguiente problema de programación convexa linealmente restringida:

$$\text{Maximizar} \quad f(\mathbf{x}) = 3x_1 + 4x_2 - x_1^3 - x_2^2,$$

sujeta a

$$x_1 + x_2 \leq 1$$

y

$$x_1 \geq 0, \quad x_2 \geq 0.$$

a) Comience con la solución de prueba inicial $(x_1, x_2) = (\frac{1}{4}, \frac{1}{4})$, para realizar tres iteraciones del algoritmo de Frank-Wolfe.

b) Utilice las condiciones KKT para verificar si la solución que obtuvo en *a)* en realidad es óptima.

12.9-9. Considere el siguiente problema de programación convexa linealmente restringida.

$$\text{Maximizar} \quad f(\mathbf{x}) = 4x_1 - x_1^4 + 2x_2 - x_2^2,$$

sujeta a

$$4x_1 + 2x_2 \leq 5$$

y

$$x_1 \geq 0, \quad x_2 \geq 0.$$

a) Comience con la solución de prueba inicial $(x_1, x_2) = (\frac{1}{2}, \frac{1}{2})$ para realizar cuatro iteraciones del algoritmo de Frank-Wolfe.

b) Demuestre en forma gráfica cómo se puede extrapolar la sucesión de soluciones de prueba que obtuvo en *a)* para obtener una aproximación más cercana a la solución óptima. ¿Cuál es la estimación que resulta de esta solución?

c) Utilice las condiciones KKT para verificar si la solución que obtuvo en *b)* es en realidad óptima. Si no lo es, utilice estas condiciones para derivar una solución óptima exacta.

12.9-10. Reconsidere el modelo de programación convexa linealmente restringido del problema 12.9-8.

a) Si se aplicara SUMT directamente a este problema, ¿cuál sería la función no restringida $P(\mathbf{x}; r)$ que debe minimizarse en cada iteración?

b) Establezca $r = 1$ y use $(\frac{1}{4}, \frac{1}{4})$ como solución de prueba inicial, para aplicar una iteración del procedimiento de búsqueda del gradiente (deténgase antes de despejar t^*) para maximizar la función $P(\mathbf{x}; r)$ que obtuvo en el inciso *a)*.

D,C **c)** A partir de la misma solución de prueba inicial del inciso *b)*, use una computadora para aplicar la técnica SUMT a este problema con $r = 1, 10^{-2}, 10^{-4}$.

d) Compare el resultado del inciso *c)* de la solución óptima del problema 12.9-8. ¿Cuál es el porcentaje de error en x_1 , en x_2 y en $f(\mathbf{x})$?

12.9-11. Reconsidere el modelo de programación convexa linealmente restringido del problema 12.9-9. Siga las instrucciones de los incisos *a)*, *b)* y *c)* del problema 12.9-10 para este modelo, pero use $(x_1, x_2) = (\frac{1}{2}, \frac{1}{2})$ como solución de prueba inicial y $r = 1, 10^{-2}, 10^{-4}, 10^{-6}$.

12.9-12. Reconsidere el modelo del problema 12.3-3.

a) Si se aplicara SUMT directamente a este problema, ¿cuál sería la función no restringida $P(\mathbf{x}; r)$ que debe minimizarse en cada iteración?

b) Establezca $r = 100$ y use $(x_1, x_2) = (5, 5)$ como solución de prueba inicial para aplicar una iteración manual del procedimiento de búsqueda del gradiente (deténgase antes de despejar t^*) para minimizar la función $P(\mathbf{x}; r)$ que obtuvo en el inciso *a)*.

D,C **c)** A partir de la misma solución de prueba inicial del inciso *b)*, use el procedimiento automático del IOR Tutorial para aplicar la técnica SUMT a este problema con $r = 100, 1, 10^{-2}, 10^{-4}$. (*Suponga:* La rutina de computadora supone que el problema se ha convertido a la forma de maximización con restricciones funcionales de la forma $\leq$.)

12.9-13. Considere el ejemplo que se dio en la sección 12.9 con el objeto de aplicar la técnica SUMT.

a) Demuestre que $(x_1, x_2) = (1, 2)$ satisface las condiciones KKT.

b) Muestre la región factible en una gráfica, después localice los puntos $x_1x_2 = 2$ para demostrar que $(x_1, x_2) = (1, 2)$ con $f(1, 2) = 2$ es en realidad un *máximo global*.

12.9-14.* Considere el siguiente problema de programación convexa.

$$\text{Maximizar} \quad f(\mathbf{x}) = -2x_1 - (x_2 - 3)^2,$$

sujeta a

$$x_1 \geq 3 \quad y \quad x_2 \geq 3.$$

- a) Si se aplicara la técnica SUMT a este problema, ¿cuál sería la función no restringida $P(\mathbf{x}; r)$ que debe maximizarse en cada iteración?
- b) Derive la solución de maximización de $P(\mathbf{x}; r)$ analíticamente y después calcule esta solución con $r = 1, 10^{-2}, 10^{-4}, 10^{-6}$.
- D.C c) A partir de la solución prueba inicial $(x_1, x_2) = (4, 4)$, utilice el procedimiento automático del IOR Tutorial para aplicar la técnica SUMT a este problema con $r = 1, 10^{-2}, 10^{-4}, 10^{-6}$.

D.C **12.9-15.** Considere el siguiente problema de programación convexa:

$$\text{Maximizar} \quad f(\mathbf{x}) = x_1x_2 - x_1 - x_1^2 - x_2 - x_2^2,$$

sujeta a

$$x_2 \geq 0.$$

A partir de la solución de prueba inicial $(x_1, x_2) = (1, 1)$, utilice el procedimiento automático del IOR Tutorial para aplicar SUMT a este problema, con $r = 1, 10^{-2}, 10^{-4}$.

D.C **12.9-16.** Reconsidere el modelo de programación cuadrática del problema 12.7-4. Comience con la solución de prueba inicial $(x_1, x_2) = (\frac{1}{2}, \frac{1}{2})$, luego, utilice el procedimiento automático del IOR Tutorial para aplicar SUMT al modelo con $r = 1, 10^{-2}, 10^{-4}, 10^{-6}$.

D.C **12.9-17.** Reconsidere la primera variación de programación cuadrática del problema de la Wyndor Glass de la sección 12.2 (vea la figura 12.6). A partir de la solución de prueba inicial $(x_1, x_2) = (2, 3)$, utilice el procedimiento automático del IOR Tutorial para aplicar SUMT a este problema con $r = 10^2, 1, 10^{-2}, 10^{-4}$.

12.9-18. Reconsidere el modelo de programación convexa con una restricción de igualdad del problema 12.6.11.

- a) Si se aplicara la técnica SUMT a este problema, ¿cuál sería la función no restringida $P(\mathbf{x}; r)$ que debe minimizarse en cada iteración?
- D.C b) A partir de la solución de prueba inicial $(x_1, x_2, x_3) = (\frac{3}{2}, \frac{3}{2}, 2)$, utilice el procedimiento automático del IOR Tutorial para aplicar la técnica SUMT a este problema con $r = 10^{-2}, 10^{-4}, 10^{-6}, 10^{-8}$.
- c) Utilice el Excel Solver estándar para resolver este problema.
- d) Utilice el Evolutionary Solver para resolver este problema.
- e) Utilice LINGO para resolver este problema.

12.10-1. Considere el siguiente problema de programación no convexa:

$$\text{Maximizar} \quad f(x) = 1\,000x - 400x^2 + 40x^3 - x^4,$$

sujeta a

$$x^2 + x \leq 500$$

y

$$x \geq 0.$$

a) Identifique los valores factibles de x . Obtenga las expresiones generales de las tres primeras derivadas de $f(x)$. Utilice esta información como ayuda para hacer un bosquejo de $f(x)$ sobre la región factible de x . Sin calcular sus valores, marque en la gráfica los puntos que corresponden a máximos y mínimos locales.

b) Utilice el procedimiento de búsqueda de bisección con $\epsilon = 0.05$ para encontrar todos los máximos locales. Use el bosquejo del inciso a) para identificar las cotas iniciales apropiadas de cada una de estas búsquedas. ¿Cuál de los máximos locales es un máximo global?

c) A partir de $x = 3$ y $x = 15$ como las soluciones de prueba iniciales, utilice el método de Newton con $\epsilon = 0.001$ para encontrar cada uno de los máximos locales.

D.C d) Utilice el procedimiento automático del IOR Tutorial para aplicar SUMT a este problema, con $r = 10^3, 10^2, 10, 1$ para encontrar cada máximo local. Considere $x = 3$ y $x = 15$ como soluciones prueba iniciales para estas búsquedas. ¿Cuál de los máximos locales es un máximo global?

e) Formule este problema en una hoja de cálculo, donde $f(x)$ representa ganancia, y después utilice la tabla de Solver para generar las soluciones con los siguientes puntos de inicio: $x = 0, 5, 10, 15, 20, 25$. Incluya el valor x y la ganancia como las celdas de salida de la tabla de Solver.

f) Utilice el Evolutionary Solver para resolver este problema.

g) Basese en la característica de optimización global de LINGO para resolver este problema.

h) Emplee MPL y su optimizador global LGO para resolver este problema.

12.10-2. Considere el siguiente problema de programación no convexa:

$$\text{Maximizar} \quad f(\mathbf{x}) = 3x_1x_2 - 2x_1^2 - x_2^2,$$

sujeta a

$$x_1^2 + 2x_2^2 \leq 4$$

$$2x_1 - x_2 \leq 3$$

$$x_1x_2^2 + x_1^2x_2 = 2$$

y

$$x_1 \geq 0, \quad x_2 \geq 0.$$

a) Si se aplicara la SUMT al problema, ¿cuál sería la función no restringida $P(\mathbf{x}; r)$ que debe minimizarse en cada iteración?

D.C b) A partir de la solución de prueba inicial $(x_1, x_2) = (1, 1)$, utilice el procedimiento automático del IOR Tutorial para aplicar SUMT a este problema con $r = 1, 10^{-2}, 10^{-4}$.

c) Utilice el Evolutionary Solver para resolver este problema.

d) Use la característica de optimización global de LINGO para resolver este problema.

e) Use MPL y su optimizador global para resolver este problema.

12.10-3. Considere el siguiente problema de programación no convexa.

$$\text{Minimizar} \quad f(\mathbf{x}) = \sin 3x_1 + \cos 3x_2 + \sin(x_1 + x_2),$$

sujeta a

$$\begin{aligned} x_1^2 - 10x_2 &\geq -1 \\ 10x_1 + x_2^2 &\leq 100 \end{aligned}$$

y

$$x_1 \geq 0, \quad x_2 \geq 0.$$

- a) Si se aplicara la SUMT al problema, ¿cuál sería la función no restringida $P(\mathbf{x}; r)$ que debe minimizarse en cada iteración?
- b) Describa cómo podría aplicarse la SUMT para intentar obtener un mínimo global (no resuelva).
- c) Utilice la característica de optimización global de LINGO para resolver este problema.
- d) Báse en MPL y su optimizador global para resolver este problema.

c 12.10-4. Considere el siguiente problema de programación no convexa:

$$\text{Maximizar ganancia} = x^5 - 13x^4 + 59x^3 - 107x^2 + 61x,$$

sujeta a

$$0 \leq x \leq 5.$$

- a) Formule este problema en una hoja de cálculo y después utilice la tabla de Solver para generar las soluciones con los siguientes puntos de inicio: $x = 0, 1, 2, 3, 4$ y 5 . Incluya el valor x y la ganancia como las celdas de salida en la tabla de Solver.
- b) Utilice el Evolutionary Solver para resolver este problema.

c 12.10-5. Considere el siguiente problema de programación no convexa:

$$\begin{aligned} \text{Maximizar ganancia} = & 100x^6 - 1\,359x^5 + 6\,836x^4 \\ & - 15\,670x^3 + 15\,870x^2 - 5\,095x, \end{aligned}$$

sujeta a

$$0 \leq x \leq 5.$$

- a) Formule este problema en una hoja de cálculo y después utilice la tabla de Solver para generar las soluciones con los siguientes puntos de inicio: $x = 0, 1, 2, 3, 4$ y 5 . Incluya el valor x y la ganancia como las celdas de salida en la tabla de Solver.
- b) Utilice el Evolutionary Solver para resolver este problema.

c 12.10-6. Debido al crecimiento de la población, el estado de Washington ha recibido un asiento adicional en la cámara de representantes, con lo que ahora cuenta con un total de 10 escaños. La Legislatura, que en la actualidad está controlada por los republicanos, necesita desarrollar un plan para redefinir los distritos en que se dividirá el estado. Existen 18 ciudades importantes en él que deben asignarse a uno de los 10 distritos del Congreso. En la tabla siguiente se proporcionan los números de demócratas y republicanos registrados en cada ciudad. Cada distrito debe contener entre 150 000 y 350 000 de estos votantes registrados. Use el Evolutionary Solver para asignar cada ciudad a uno de los 10 distritos del Congreso de forma que se maximice el número de distritos que tenga registrados más republicanos que demócratas. (*Sugerencia:* Utilice la función SUMIF.)

Ciudad	Demócratas (miles)	Republicanos (miles)
1	152	62
2	81	59
3	75	83
4	34	52
5	62	87
6	38	87
7	48	69
8	74	49
9	98	62
10	66	72
11	83	75
12	86	82
13	72	83
14	28	53
15	112	98
16	45	82
17	93	68
18	72	98

12.10-7. Reconsidere el problema de la Wyndor Glass Co., que se presentó en la sección 3.1.

- c a) Resuelva este problema mediante el uso del Excel Solver estándar.
- c b) A partir de una solución inicial de producir 0 lotes de puertas y 0 lotes de ventanas, resuelva este problema mediante el uso del Evolutionary Solver.
- c) Comente acerca del desempeño de cada uno de los dos enfoques.

12.11-1. Considere el siguiente problema:

$$\text{Maximizar } Z = 4x_1 - x_1^2 + 10x_2 - x_2^2,$$

sujeta a

$$x_1^2 + 4x_2^2 \leq 16$$

y

$$x_1 \geq 0, \quad x_2 \geq 0.$$

- a) ¿Es éste un problema de programación convexa? Responda sí o no y justifique su respuesta.
- b) ¿Se puede modificar el método simplex para usarlo en la solución de este problema? Responda sí o no y justifique su respuesta (pero no lo resuelva).
- c) ¿Puede usarse el algoritmo de Frank-Wolfe para resolver este problema? Responda sí o no y después justifique su respuesta (pero no lo resuelva).
- d) ¿Cuáles son las condiciones KKT de este problema? Utilice estas condiciones para determinar si $(x_1, x_2) = (1, 1)$ puede ser óptima.
- e) Utilice la técnica de programación separable para formular un modelo de programación lineal *aproximado* para manejar este problema. Use los enteros factibles como puntos de salto de la función lineal por partes.
- c f) Utilice el método simplex para resolver el problema según se formuló en el inciso e).

- g) Dada la función $P(x; r)$, maximizar en cada iteración al aplicar SUMT al problema. (No resuelva.)
- D.C h) Use SUMT para resolver el problema que se formuló en g). Comience con la solución de prueba inicial $(x_1, x_2) = (2, 1)$ y utilice $r = 1, 10^{-2}, 10^{-4}, 10^{-6}$.

- c i) Formule este problema en una hoja de cálculo y después utilice el Excel Solver estándar para resolver este problema.
- c j) Use el Evolutionary Solver para resolver este problema.
- c k) Use LINGO para resolver este problema.

CASOS

Caso 12.1 Selección inteligente de acciones

Desde el día que tomó su primer curso de economía en la preparatoria, Lidia se preguntó sobre las prácticas financieras de sus padres, que trabajaban duro para ganar suficiente dinero para vivir una vida cómoda, pero nunca hicieron que su dinero trabajara para ellos. Sólo depositaban sus cheques ganados con esfuerzo en cuentas de ahorro que ganaban un interés nominal. (Por fortuna, siempre hubo suficiente dinero cuando llegó el momento de pagar la universidad.) Se prometió a sí misma que cuando llegara a la adultez, ella no seguiría las mismas prácticas conservadoras de sus padres.

Lidia cumplió su promesa. Cada mañana mientras se alista para el trabajo ve el reporte financiero de CNN. Participa en los juegos de inversión de internet, en donde encuentra carteras que maximizan su rendimiento y minimizan su riesgo. Lee *The Wall Street Journal* y el *Financial Times* con ansia de aprender.

Lidia también presta atención a las columnas de los consultores de inversiones de las revistas financieras y ha observado que, en promedio, los consejos que aparecen en ellas resultan muy buenos. Por lo tanto, decide seguir las sugerencias que presentan los últimos números de esas revistas. En su columna mensual, el editor Jonathan Taylor recomienda tres acciones que piensa que subirán muy por arriba del promedio del mercado. Además, la conocida gurú de fondos mutuos Donna Carter insiste en la compra de otras tres acciones que cree que tendrán el mejor desempeño del mercado durante el siguiente año.

BIGBELL (símbolo en el mercado de valores: BB), una de las compañías de telecomunicaciones más grandes del país, se vende con una tasa de ganancia por debajo del promedio del mercado. Grandes inversiones durante los últimos 8 meses han deprimido las ganancias de manera considerable. Sin embargo, debido a la nueva tecnología, se espera que la compañía eleve sus márgenes de utilidades en forma significativa. Taylor predice que las acciones subirán de su precio actual de 60 dólares cada una a 72 dólares en un año.

LOTSOFPLACE (LOP) es una de las más poderosas manufactureras del mundo. Hace poco la industria se consolidó, cuando una feroz guerra de precios durante los últimos años provocó que muchos competidores quebraran o vendieran sus compañías a LOTSOFPLACE y sus competidores. Debido a la menor competencia en el mercado, se espera que los ingresos y ganancias se eleven de manera considerable en el siguiente año. Taylor pronostica que, de su precio unitario actual de 127 dólares, en un año las acciones de LOTSOFPLACE subirán 42%.

INTERNETLIFE (ILI) ha sobrevivido a las muchas altas y bajas de las compañías de internet. Con el furor por internet que se avecina, Taylor espera que la compañía duplique el precio de sus acciones de 4 a 8 dólares en un año.

HEALTHTOMORROW (HEAL) es una compañía biotecnológica líder que está a punto de obtener la aprobación de la FDA para varios medicamentos nuevos, lo que ayudará a que sus ganancias se eleven 20% en los próximos años. En particular, se supone que una nueva medicina para reducir significativamente el riesgo de ataques al corazón dará grandes ganancias. Además, debido al lanzamiento de varios medicamentos con sabor delicioso para niños, la compañía se ha forjado una excelente imagen en el medio. Sin duda, esta hazaña de relaciones públicas tendrá efectos positivos sobre sus ventas de medicamentos sin receta. Carter está convencida de que el precio unitario de las acciones se elevará de 50 a 75 dólares en un año.

QUICKY (QUI) es una cadena de restaurantes de comida rápida que se ha expandido por todo Estados Unidos. Carter ha seguido de cerca a esta compañía desde su entrada a la Bolsa hace cerca de 15 años cuando tenía unas docenas de restaurantes en la costa oeste. Desde entonces se ha expandido y ahora cubre todos los estados. Debido a su preocupación por la comida sana, capta un porcentaje de mercado creciente. Carter cree que las acciones continuarán con el buen desempeño arriba del promedio de mercado con un incremento de 46% en un año sobre el precio actual de 150 dólares.

AUTOMOBILE ALLIANCE (AUA) es el fabricante líder de automóviles en el área de Detroit que acaba de introducir dos modelos nuevos, los cuales muestran ventas iniciales muy fuertes. Por lo tanto, se predice que las acciones de la compañía subirán de 20 a 26 dólares en un año.

En internet, Lidia encontró datos acerca del riesgo involucrado en las acciones de estas compañías. Las varianzas históricas de los rendimientos de las seis acciones y sus covarianzas se muestran en las siguientes tablas.

- Primero, Lidia quiere pasar por alto el riesgo de todas las inversiones. Dada esta estrategia, ¿cuál es su cartera de inversión óptima, es decir, qué fracción de su dinero debe invertir en cada una de las seis acciones? ¿Cuál es el riesgo total de su cartera?
- Lidia decide que no quiere invertir más de 40% en una sola acción. Si aún pasa por alto el riesgo, ¿cuál es su nueva cartera de inversión óptima? ¿Cuál es el riesgo total de su nueva cartera?
- Ahora Lidia quiere tomar en cuenta el riesgo de sus oportunidades de inversión. Para usarlo en los siguientes incisos, formule un modelo de programación cuadrática que maxi-

Compañía	BB	LOP	ILI	HEAL	QUI	AUA
Varianza	0.032	0.1	0.333	0.125	0.065	0.08

Covarianza	LOP	ILI	HEAL	QUI	AUA
BB	0.005	0.03	-0.031	-0.027	0.01
LOP		0.085	-0.07	-0.05	0.02
ILI			-0.11	-0.02	0.042
HEAL				0.05	-0.06
QUI					-0.02

mice su riesgo (medido por la varianza del beneficio que obtendrá de su cartera), mientras asegura que el beneficio que obtenga sea al menos tan grande como la elección de un valor mínimo aceptable.

- d) Lidia quiere asegurar que recibirá un rendimiento esperado de, al menos, 35%. Desea lograr su meta con un riesgo mínimo. ¿Qué cartera de inversión le permite hacerlo?

- e) ¿Cuál es el riesgo mínimo que Lidia puede lograr si desea un rendimiento esperado de 25%? ¿Y de al menos 40%?

- f) ¿Percibe usted problemas o desventajas en el enfoque de Lidia hacia su estrategia de inversión?

(Nota: Para su conveniencia, en el sitio en internet de este libro se proporciona un archivo de datos para este caso.)

■ RESUMEN DE CASOS ADICIONALES EN NUESTRO SITIO EN INTERNET (www.mhhe.com/hillier)

CASO 12.2 Inversiones internacionales

Un analista financiero posee algunos bonos alemanes que ofrecen incrementar su tasa de interés si se conservan hasta su madurez completa en tres años más. También se pueden negociar en cualquier momento para obtener el principal original más el interés generado. El gobierno alemán acaba de introducir un impuesto a la ganancia de capital sobre el ingreso por intereses mayor a cierto nivel, por lo que, en estas circunstancias, conservar los bonos hasta su madurez resulta menos atractivo. Por tanto, el analista tiene que determinar su estrategia óptima de inversión relativa a cuántos bonos debe vender en cada uno de los siguientes tres años bajo unos cuantos escenarios diferentes.

CASO 12.3 Promoción de un cereal para el desayuno, revisado

Este caso es una continuación del caso 3.4 que involucra una campaña de publicidad del nuevo cereal para desayuno de Super Gain Corporation. El análisis que se requiere para dicho caso conduce a la aplicación de programación lineal. Sin embargo, existen ciertos supuestos de programación lineal que son bastante cuestionables en esta situación. En particular, es claro que el supuesto de que la ganancia total por la introducción del nuevo producto es proporcional al número total de exposiciones debidas a la campaña de publicidad, es sólo una aproximación gruesa. Para perfeccionar el análisis, se deben formular, aplicar y comparar un modelo de programación no lineal general y un modelo de programación separable.

13

CAPÍTULO

Metaheurística

En algunos de los capítulos anteriores se han descrito algoritmos que pueden usarse para obtener una solución óptima para varios tipos de modelos de IO, entre los cuales se incluyen ciertas clases de modelos de programación lineal, programación entera y programación no lineal. Estos algoritmos han probado ser invaluable para abordar una amplia variedad de problemas prácticos. Sin embargo, este enfoque no siempre funciona. Algunos problemas (y los correspondientes modelos de IO) pueden ser tan complicados que no es posible resolverlos para encontrar una solución óptima. En tales situaciones, aún es importante encontrar una buena solución factible que al menos esté razonablemente cerca de ser óptima. Por lo general, para buscar esa solución se utilizan métodos heurísticos.

Un **método heurístico** es un procedimiento que trata de descubrir una solución factible muy buena, pero no necesariamente una solución óptima, para el problema específico bajo consideración. No puede darse una garantía acerca de la calidad de la solución que se obtiene, pero un método heurístico bien diseñado puede proporcionar una solución que al menos está cerca de ser óptima (o concluir que no existen tales soluciones). El procedimiento también debe ser suficientemente eficiente como para manejar problemas muy grandes. Con frecuencia, el procedimiento es un *algoritmo iterativo* novedoso, donde cada iteración implica la realización de una búsqueda de una nueva solución que puede ser mejor que la solución que se encontró con anterioridad. Cuando el algoritmo termina después de un tiempo razonable, la solución que proporciona es la mejor que se pudo encontrar en cualquier iteración.

Con frecuencia los métodos heurísticos se basan en ideas bastante simples, de sentido común, acerca de la forma en que se debe buscar una buena solución. Estas ideas deben ajustarse al problema específico de interés. En consecuencia, los métodos heurísticos tienden a ser *ad hoc* por naturaleza. Esto es, por lo general cada método se diseña para abordar un tipo específico de problema en vez de una variedad de aplicaciones.

Por muchos años esta situación significó que, para desarrollar un método heurístico, un equipo de IO necesitaría comenzar desde cero para así ajustarse al problema bajo consideración, siempre que no exista un algoritmo disponible para encontrar una solución óptima. Este panorama ha cambiado en años relativamente recientes con el desarrollo de metaheurísticas poderosas. Una **metaheurística** es un método de solución general que proporciona tanto una estructura general como criterios estratégicos para desarrollar un método heurístico específico que se ajuste a un tipo particular de problema. La metaheurística se ha convertido en una de las técnicas más importantes del paquete de herramientas que utilizan los profesionales de la IO.

En este capítulo se proporciona una introducción elemental a la metaheurística. En la primera sección se describirá la naturaleza general de la metaheurística. Después, en las siguientes tres secciones, se presentarán e ilustrarán las tres metaheurísticas que se utilizan con mayor frecuencia.

13.1 NATURALEZA DE LA METAHEURÍSTICA

Para ilustrar la naturaleza de la metaheurística se iniciará con un ejemplo de un problema de programación lineal pequeño pero moderadamente difícil.

Ejemplo: Un problema de programación no-lineal con óptimos locales múltiples

Considere el siguiente problema:

$$\text{Maximizar } f(x) = 12x^5 - 975x^4 + 28\,000x^3 - 345\,000x^2 + 1\,800\,000x,$$

sujeto a

$$0 \leq x \leq 31.$$

En la figura 13.1 se grafica la función objetivo $f(x)$ sobre los valores factibles de la variable única x . Esta gráfica revela que el problema tiene tres óptimos locales, uno en $x = 5$, otro en $x = 20$ y el tercero en $x = 31$, donde el óptimo global se encuentra en $x = 20$.

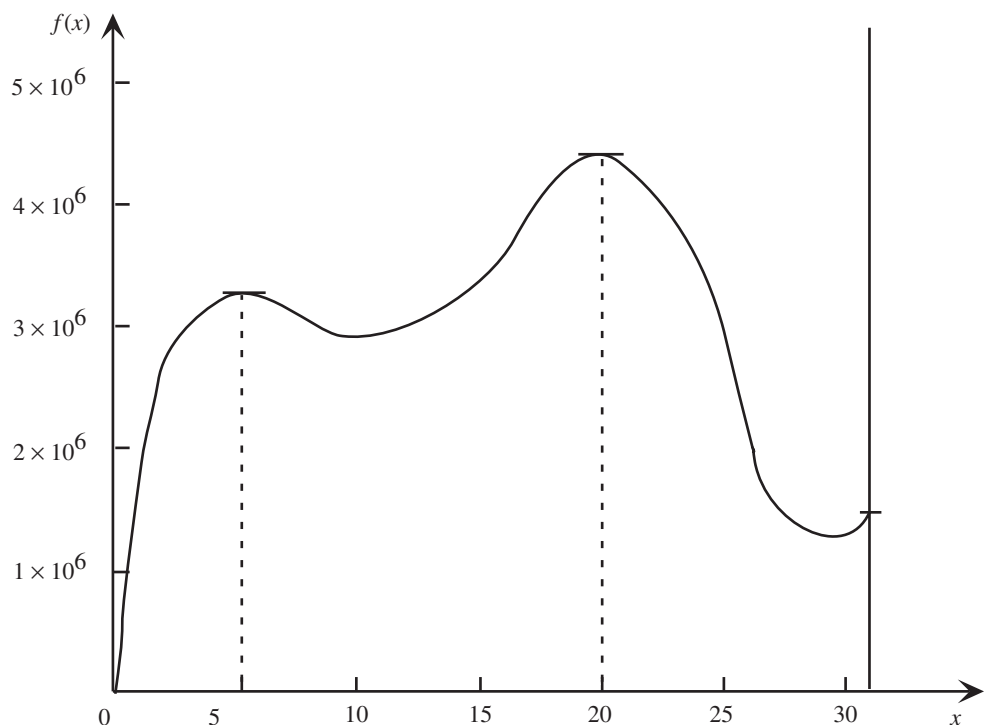
La función objetivo $f(x)$ es tan complicada que sería difícil determinar dónde se encuentra el óptimo global sin el beneficio de observar la gráfica de la figura 13.1. Se podría usar el cálculo, pero esto requeriría resolver una ecuación polinomial de cuarto grado (después de igualar a cero la primera derivada) para determinar dónde se encuentran los puntos críticos. Incluso sería difícil determinar que $f(x)$ tiene óptimos locales múltiples en vez de un solo óptimo global.

Este problema es un ejemplo de un problema de *programación no convexa*, un tipo especial de programación no lineal que por lo general tiene óptimos locales múltiples. En la sección 12.10 se explica la programación convexa e incluso se introduce un paquete de software (Solver evolutivo) que utiliza el tipo de metaheurística que se describe en la sección 13.4.

Para problemas de programación no lineal que parecen ser muy difíciles, como éste, un método heurístico común es ejecutar un **procedimiento de mejora local**. Tal procedimiento comienza con una solución de prueba inicial y después, en cada iteración, busca en la vecindad de la solución de prueba para tratar de encontrar una mejor solución que la actual. Este proceso continúa hasta que no se pueda encontrar una solución mejorada en la vecindad de la solución de prueba actual. De esta manera, este tipo de procedimiento puede verse como un procedimiento de *escalada de montaña* que se mantiene en ascenso en la gráfica de la función objetivo (si el objetivo es una maximización) hasta que, en esencia, alcanza la cumbre de la montaña. Un procedimiento de mejora local bien diseñado, por lo general convergirá hacia un óptimo *local* (una cumbre de montaña), pero se detendrá aun si este óptimo local no es un óptimo *global* (la cumbre más alta de la montaña).

■ FIGURA 13.1

Gráfica del valor de la función objetivo en un intervalo factible, $0 \leq x \leq 31$, del ejemplo de programación no lineal. Los óptimos locales están en $x = 5$, $x = 20$ y $x = 31$, pero sólo $x = 20$ es un óptimo global.



Por ejemplo, el *procedimiento de búsqueda de gradiente* descrito en la sección 12.5 es un procedimiento de mejora local. Si éste comienza, por ejemplo, con $x = 0$ como la solución de prueba inicial en la figura 13.1, ascendería por la pendiente al probar en forma exitosa valores más grandes de x hasta que, en esencia, llegara a la cumbre de la montaña en $x = 5$, punto en el que se detendría. En la figura 13.2 se muestra una secuencia típica de valores de $f(x)$ que se obtendría mediante este procedimiento de mejora local cuando se inicia desde la parte baja de la gráfica.

Como el ejemplo de programación no lineal que se muestra en la figura 13.1 involucra sólo una variable, el método de bisección que se describió en la sección 12.4 también se podría aplicar a este problema particular. Éste es otro ejemplo de un procedimiento de mejora local, puesto que cada iteración comienza en la solución de prueba actual a buscar una mejor solución en su vecindad (que se define por las cotas inferior y superior actuales sobre el valor de la variable). Por ejemplo, si la búsqueda comenzara con una cota inferior de $x = 0$ y una cota superior de $x = 6$ en la figura 13.1, la secuencia de soluciones de prueba que se obtendría por el método de bisección sería $x = 3$, $x = 4.5$, $x = 5.25$, $x = 4.875$, hasta convergir a $x = 5$. Los valores correspondientes de la función objetivo de estas cuatro soluciones de prueba son 2.975 millones, 3.286 millones, 3.300 millones y 3.302 millones, respectivamente. Por consiguiente, la segunda iteración proporciona una mejora más o menos grande sobre la primera (311 000), la tercera iteración da una mejora mucho más pequeña (14 000), mientras que la cuarta sólo ofrece una mejora marginal (2 000). Como se muestra en la figura 13.2, este patrón es bastante típico de los procedimientos de mejora local (aunque con alguna variación en la tasa de convergencia al máximo local).

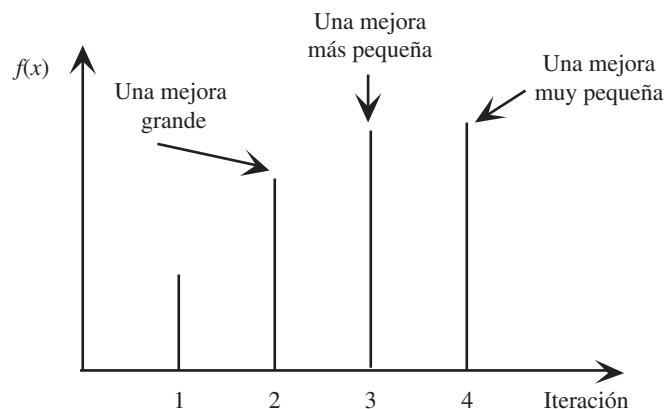
Igual que en el procedimiento de búsqueda de gradiente, esta búsqueda con el método de bisección se quedaría atrapada en el óptimo local de $x = 5$, por lo que nunca encontraría el óptimo global en $x = 20$. Como otros procedimientos de mejora local, tanto el de búsqueda de gradiente como el método de bisección fueron diseñados sólo para obtener una mejoría sobre la solución de prueba actual dentro de su vecindad local. Una vez que éstos ascienden hasta el pico de la montaña, se deben detener porque ya no pueden ascender más dentro de la vecindad local de la solución de prueba en la cumbre. Este inconveniente ilustra la desventaja de cualquier procedimiento de mejora local.

Desventaja de un procedimiento de mejora local: Cuando se aplica un procedimiento de mejora local bien diseñado a un problema de optimización con múltiples óptimos locales, el procedimiento convergirá hacia un óptimo local y se detendrá. El óptimo local que encuentre dependerá del punto en el que el procedimiento inicia su búsqueda. Por lo tanto, el procedimiento encontrará el óptimo global sólo si inicia la búsqueda en la vecindad de dicho óptimo global.

Para tratar de evitar esta desventaja se puede reiniciar el procedimiento de mejora local cierto número de veces a partir de soluciones de prueba aleatorias. Con frecuencia, cuando se reinicia desde una parte nueva de la región factible se llega a un nuevo óptimo local. Si se repite esta rutina cierta cantidad de veces se incrementa la posibilidad de que el mejor óptimo local que se obtuvo en realidad sea el óptimo global. Este enfoque funciona bien con problemas pequeños, como el

■ FIGURA 13.2

Secuencia típica de valores de la función objetivo de las soluciones obtenidas por un procedimiento de mejora local, cuando éste converge hacia un óptimo local durante su aplicación a un problema de maximización.



problema de programación no lineal de una sola variable de la figura 13.1. Sin embargo, es mucho menos exitoso en problemas grandes con muchas variables y una región factible complicada. Cuando la región factible tiene muchos “recovecos” y sólo al reiniciar un procedimiento de mejora local en uno de ellos conducirá al óptimo global, la aplicación del procedimiento a partir de soluciones iniciales de prueba aleatorias se convierte en una forma muy fortuita de llegar al óptimo global.

Lo que se necesita en lugar de esta mecánica es un enfoque más estructurado que utilice la información recopilada para dirigir la búsqueda hacia el óptimo global. Éste es el papel que tiene la metaheurística.

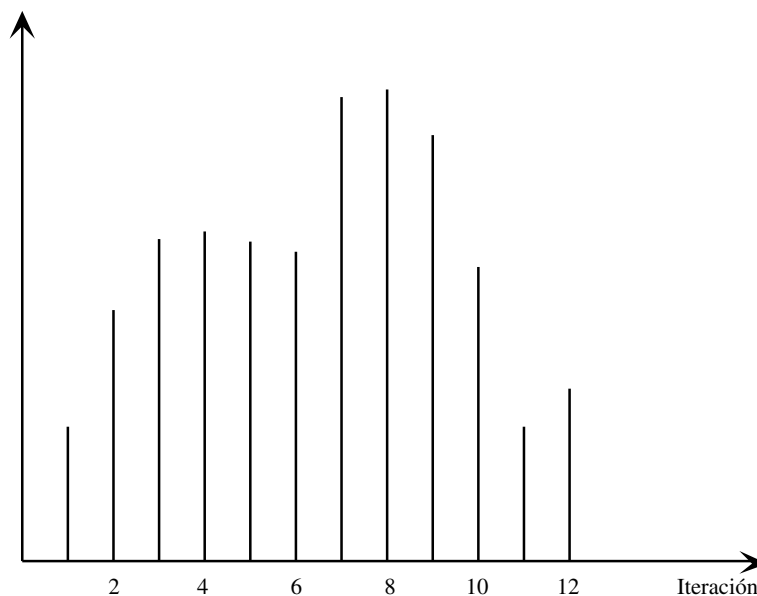
Naturaleza de la metaheurística: La metaheurística es un tipo general de método de solución que organiza la interacción entre los procedimientos de mejora local y las estrategias de más alto nivel para crear un proceso que sea capaz de escapar de un óptimo local y realizar una búsqueda vigorosa de una región factible.

En este contexto, una característica clave de la metaheurística es la capacidad de escapar de un óptimo local. Después de encontrar (o casi encontrar) un óptimo local, las diferentes metaheurísticas ejecutan este escape de modos diferentes. Sin embargo, una característica común es que las soluciones de prueba que de inmediato siguen un óptimo local pueden ser menores que éste. En consecuencia, cuando se aplica una metaheurística a un problema de maximización (como en el ejemplo que se muestra en la figura 13.1), los valores de la función objetivo de la secuencia de soluciones de prueba que se obtengan por lo general seguirán un patrón similar al que se muestra en la figura 13.3. Como con la figura 13.2, el proceso comienza con la utilización de un procedimiento de búsqueda local para ascender a la cumbre actual (iteración 4). Sin embargo, en lugar de detenerse aquí, la metaheurística podría conducir la búsqueda un poco abajo hacia el otro lado de la cumbre hasta poder comenzar a ascender hasta la cumbre más alta (iteración 8). Para verificar que éste parece ser el óptimo global, una metaheurística continúa con la exploración antes de detenerse (iteración 12).

En la figura 13.3 se ilustra tanto la ventaja como la desventaja de una metaheurística bien diseñada. La ventaja es que tiende a moverse relativamente rápido hacia soluciones muy buenas, por lo que proporciona una forma muy eficiente de abordar problemas grandes y complicados. La desventaja es que no existe garantía de que la mejor solución que se encuentre sea una solución óptima o incluso que esté cerca de serlo. Por lo tanto, siempre que sea posible resolver un problema mediante un algoritmo que pueda garantizar optimalidad, debe usarse éste en lugar de la metaheurística. El papel de las metaheurísticas es abordar problemas que son muy grandes y complicados como para resolverlos por medio de algoritmos exactos. Todos los ejemplos de este

■ FIGURA 13.3

Secuencia típica de valores de la función objetivo de las soluciones que se obtuvieron por una metaheurística, cuando ésta converge primero hacia un óptimo local (iteración 4) y después escapa para hacerlo hacia el óptimo global (iteración 8) de un problema de maximización, antes de concluir su búsqueda (iteración 12).



capítulo son demasiado pequeños como para requerir el uso de las metaheurísticas, ya que sólo intentan ilustrar de una manera directa la forma en que las metaheurísticas pueden enfrentar problemas mucho más complicados.

En la sección 13.3 se ilustrará la aplicación de una metaheurística particular al ejemplo de programación no lineal que se muestra en la figura 13.1. Después, en la sección 13.4 se aplicará otra metaheurística a la versión para programación entera del mismo ejemplo.

Aunque algunas veces se aplican metaheurísticas a problemas difíciles de programación no lineal y entera, un área de aplicación más común es la de los problemas de *optimización combinatoria*. El siguiente ejemplo es de este tipo.

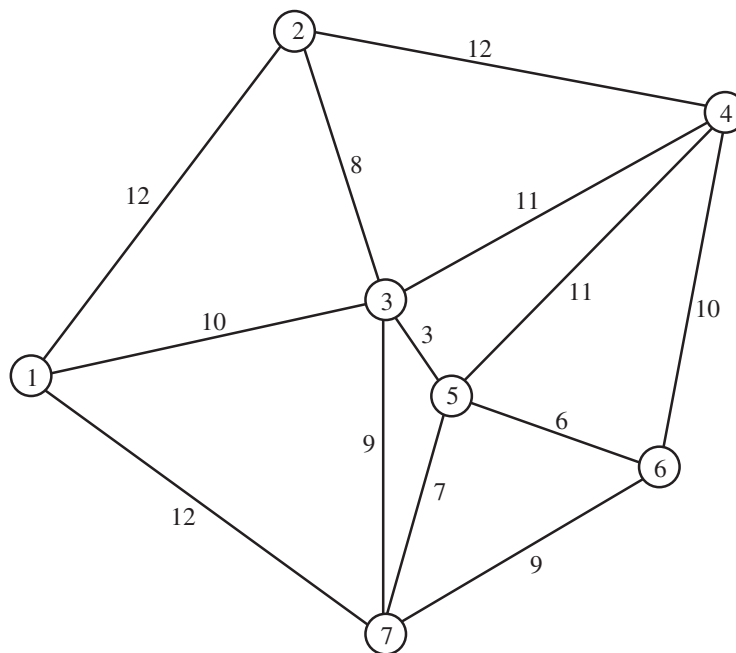
Ejemplo: Problema del agente viajero

Quizá el problema clásico de optimización combinatoria más famoso es conocido como el *problema del agente viajero*. Recibió este nombre pintoresco porque puede describirse en términos de un agente de ventas que debe visitar cierta cantidad de ciudades en un solo viaje. Si comienza desde su ciudad de residencia, el agente determinará cuál ruta debe seguir para visitar cada ciudad exactamente una vez antes de regresar a su casa de manera que se minimice la longitud total del viaje.

En la figura 13.4 se muestra un pequeño problema del agente viajero con siete ciudades. La ciudad 1 es el lugar de residencia del agente. Por lo tanto, si comienza desde su ciudad, el agente debe elegir una ruta para visitar cada una de las otras ciudades exactamente una vez antes de regresar a su punto de partida. El número colocado junto a la ligadura entre cada par de ciudades representa la distancia (o el costo o el tiempo) entre estas ciudades. Se supone que la distancia es la misma en cualquier dirección. (Esto se conoce como el problema *simétrico* del agente viajero.) Aunque por lo general existe una ligadura directa entre cada par de ciudades, en este caso el ejemplo se simplifica al suponer que las únicas ligaduras directas son las que se muestran en la figura. El objetivo es determinar cuál ruta minimizará la distancia total que el agente viajero debe viajar.

Ha existido una serie de aplicaciones del problema del agente viajero a problemas que no tienen nada que ver con un agente de ventas. Por ejemplo, cuando un camión sale de un centro de distribución para entregar bienes a cierta cantidad de ubicaciones, el problema de determinar la ruta más corta es un problema del agente viajero. Otro ejemplo involucra la fabricación de tableros de circuitos impresos, semiconductores cableados y otros componentes. Cuando es necesario perforar muchos orificios en un tablero de circuitos impresos, el problema de encontrar la secuencia de perforaciones más eficiente constituye un problema del agente viajero.

■ FIGURA 13.4
Ejemplo del problema del agente viajero que se usará con propósitos ilustrativos a lo largo de este capítulo.



El problema del agente viajero se complica con rapidez a medida que aumenta el número de ciudades. En el caso de un problema con n ciudades y una ligadura entre cada par de ellas, el número de rutas factibles que debe considerarse es $(n-1)!/2$ puesto que hay $(n-1)$ posibilidades para la primera ciudad después de la ciudad de residencia del agente, $(n-2)$ posibilidades para la siguiente ciudad y así sucesivamente. El denominador de 2 surge porque cada ruta tiene una vía inversa equivalente con la misma distancia. Así, mientras un problema del agente viajero con 10 ciudades tiene menos de 200 000 soluciones factibles que deben ser consideradas, un problema con 20 ciudades tiene alrededor de 10^{16} soluciones factibles, mientras que un problema con 50 ciudades tiene alrededor de 10^{62} .

Resulta sorprendente, pero algunos algoritmos poderosos basados en el enfoque de ramificación y acotamiento que se introdujo en la sección 11.8 han tenido éxito en resolver ciertos problemas del agente viajero muy grandes con muchos cientos (o incluso miles) de ciudades. Sin embargo, debido a la enorme dificultad para resolver grandes problemas de este tipo, los métodos heurísticos guiados por metaheurísticas son una forma popular de enfrentar tales problemas.

Por lo general, estos métodos heurísticos implican la generación de una secuencia de soluciones de prueba factibles, donde cada nueva solución de prueba se obtiene al hacer cierto tipo de pequeños ajustes en la solución de prueba actual. Se han sugerido algunos métodos para ajustar la solución de prueba actual. Debido a la facilidad de su implantación, un método popular usa el siguiente tipo de ajuste.

Un **subviaje inverso** ajusta la secuencia de ciudades visitadas en la solución de prueba actual mediante la selección de una subsecuencia de las ciudades y simplemente invertir el orden en el cual se visita esa subsecuencia de ciudades. (La subsecuencia invertida puede consistir cuando menos en dos ciudades.)

Para ilustrar un subviaje inverso, suponga que la solución de prueba inicial del ejemplo de la figura 13.4 es visitar las ciudades en orden numérico:

1-2-3-4-5-6-7-1 Distancia = 69

Por ejemplo, si se selecciona la secuencia 3-4 y se invierte, se obtiene la siguiente nueva solución de prueba:

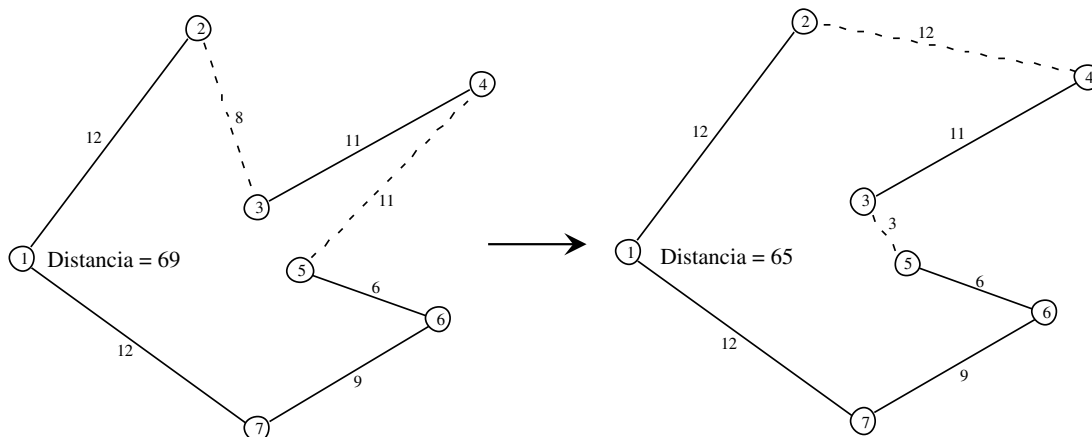
1-2-4-3-5-6-7-1 Distancia = 65

En consecuencia, este subviaje inverso en particular ha tenido éxito en reducir la distancia del viaje completo de 69 a 65.

En la figura 13.5 se muestra este subviaje inverso, el cual conduce de la solución de prueba inicial de la izquierda a la nueva solución de prueba de la derecha. Las líneas punteadas indican las

■ FIGURA 13.5

Inversión de un subviaje que reemplaza al viaje de la izquierda (la solución de prueba inicial) por el viaje de la derecha (la nueva solución de prueba) al invertir el orden en que se visitan las ciudades 3 y 4. Esta inversión del subviaje resulta en la sustitución de las líneas punteadas de la izquierda por las líneas punteadas de la derecha como las ligaduras que se invierten en el nuevo viaje.



ligaduras que se borran del viaje (a la izquierda) o se agregan al viaje (a la derecha) por el subviaje inverso. Observe que la nueva solución de prueba elimina exactamente dos ligaduras del viaje previo y las reemplaza por exactamente dos nuevas ligaduras para formar el nuevo viaje. Ésta es una característica de cualquier subviaje inverso (incluso aquéllos donde la subsecuencia de ciudades invertida consiste en más de dos ciudades). Por consiguiente, un subviaje inverso particular es posible sólo si las dos nuevas ligaduras correspondientes en realidad existen.

Este éxito en la obtención de un viaje mejorado por la simple inversión de un subviaje sugiere el siguiente método heurístico para buscar una buena solución factible para cualquier problema del agente viajero.

Algoritmo del subviaje inverso

Paso inicial. Inicie con cualquier viaje factible como la solución de prueba inicial.

Iteración. Para la solución de prueba actual, considere todas las formas posibles de realizar un subviaje inverso (sólo excluya el inverso del viaje completo). Seleccione aquel que proporcione la mayor disminución en la distancia viajada para que sea la nueva solución de prueba. (Los empates se rompen de manera arbitraria.)

Regla de detención. El proceso se detiene cuando ninguna inversión de un subviaje mejora la solución de prueba actual, la cual se acepta como la solución final.

Ahora se aplicará este algoritmo al ejemplo. Se inicia con 1-2-3-4-5-6-7-1 como la solución de prueba inicial. Existen cuatro posibles subviajes que mejorarían esta solución, como a continuación se presenta en el segundo, tercero, cuarto y quinto renglones.

	1-2-3-4-5-6-7-1	Distancia = 69
inverso 2-3:	1-3-2-4-5-6-7-1	Distancia = 68
inverso 3-4:	1-2-4-3-5-6-7-1	Distancia = 65
inverso 4-5:	1-2-3-5-4-6-7-1	Distancia = 65
inverso 5-6:	1-2-3-4-6-5-7-1	Distancia = 66

Las dos soluciones con distancia = 65 empatan pues proporcionan la mayor disminución en la distancia viajada; luego, suponga que se elige de manera arbitraria la primera de éstas, 1-2-4-3-5-6-7-1 (como se muestra en el lado derecho de la figura 13.5), como la siguiente solución de prueba, con lo cual se completa la primera iteración.

La segunda iteración comienza con el viaje que se muestra en el lado derecho de la figura 13.5 como la solución de prueba actual. En el caso de esta solución, existe sólo un subviaje que proporciona una mejora, como se menciona a continuación en el segundo renglón:

	1-2-4-3-5-6-7-1	Distancia = 65
inverso 3-5-6:	1-2-4-6-5-3-7-1	Distancia = 64

En la figura 13.6 se muestra este subviaje inverso, donde la subsecuencia de ciudades 3-5-6 de la izquierda ahora se modifica en orden inverso (6-5-3) a la derecha. Por lo tanto, en el viaje a la derecha ahora se presenta la ligadura 4-6 en lugar de 4-3, así como la ligadura 3-7 en lugar de 6-7, para usar el orden inverso 6-5-3 entre las ciudades 4 y 7. Con lo anterior se completa la segunda iteración.

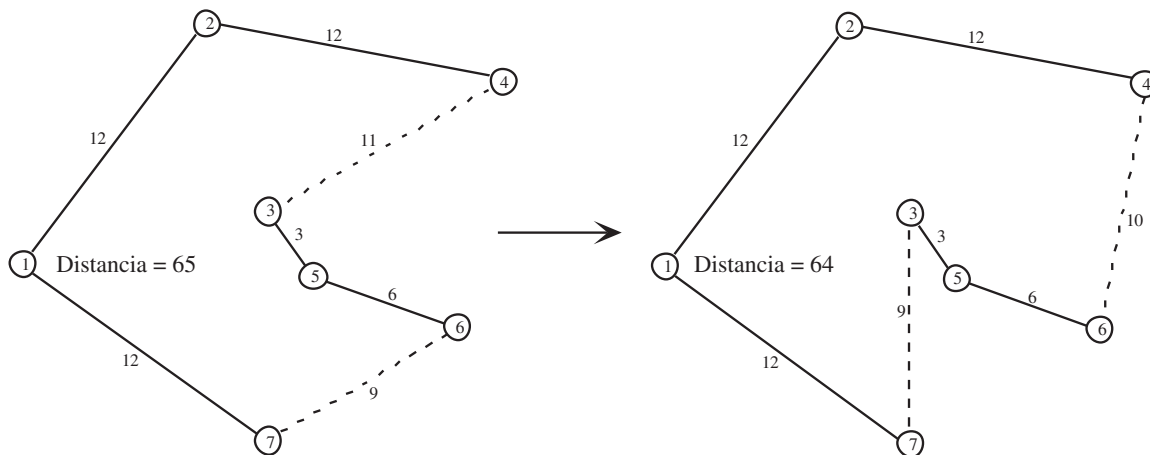
A continuación se tratará de encontrar el inverso que mejorará esta nueva solución de prueba. Sin embargo, no existe ninguna, por lo que se detiene el algoritmo del subviaje inverso con esta solución de prueba como la solución final.

¿La solución óptima es 1-2-4-6-5-3-7-1? Desafortunadamente no, pues la solución óptima resulta ser

1-2-4-6-7-5-3-1 Distancia = 63

(o 1-3-5-7-6-4-2-1 al invertir la dirección de este viaje completo)

Sin embargo, no se puede llegar a esta solución al realizar la inversión de un subviaje que mejore 1-2-4-6-5-3-7-1.



■ FIGURA 13.6

Inversión del subviaje 3-5-6 que conduce desde la solución de prueba de la izquierda hasta una solución de prueba mejorada de la derecha.

El algoritmo del subviaje inverso es otro ejemplo de un *procedimiento de mejora local*. Encuentra una mejoría sobre la solución de prueba actual en cada iteración. Cuando ya no puede encontrar una mejor solución se detiene porque la solución de prueba actual es un *óptimo local*. En este caso, en realidad 1-2-4-6-5-3-7-1 es un *óptimo local* porque no hay una mejor solución dentro de su vecindad local que pueda ser alcanzada mediante la inversión de un subviaje.

Lo que podría dar una mejor oportunidad de encontrar un *óptimo local* es utilizar una metaheurística que permita el proceso de escapar de un *óptimo local*. En las tres secciones siguientes se verá cómo tres metaheurísticas distintas logran este resultado con el mismo ejemplo.

13.2 BÚSQUEDA TABÚ

La búsqueda tabú es una metaheurística muy usada que utiliza algunas ideas de sentido común para permitir que el proceso de búsqueda escape de un *óptimo local*. Después de introducir los conceptos básicos, se abordará un ejemplo simple y después se regresará al ejemplo del agente viajero.

Conceptos básicos

Cualquier aplicación de la búsqueda tabú incluye como una subrutina algún *procedimiento de búsqueda local* que parezca apropiado para el problema bajo consideración. (Un **procedimiento de búsqueda local** opera como un procedimiento de mejora local excepto que no requiere que cada nueva solución de prueba sea mejor que la solución de prueba anterior.) El proceso comienza con este método como un procedimiento de *mejora local* de la manera usual (es decir, al aceptar sólo una solución mejorada en cada iteración) para encontrar un *óptimo local*. Una estrategia de la búsqueda tabú es que continúa la búsqueda pero permite *movimientos sin mejora* hacia las mejores soluciones en la vecindad del *óptimo local*. Una vez que se alcanza un punto en el que se pueden encontrar mejores soluciones en la vecindad de la solución de prueba, se aplica de nuevo el procedimiento de mejora local para encontrar un nuevo *óptimo local*.

Debido a la analogía con la escalada de una montaña, algunas veces este proceso se conoce como **enfoque del ascenso más empujado/descenso más suave** porque cada iteración selecciona el movimiento disponible que sube más en la pendiente o, cuando no hay disponible algún movimiento hacia arriba, selecciona el movimiento que baja menos en la pendiente. Si todo sale bien, el proceso seguirá un patrón como el que se muestra en la figura 13.3, donde se deja atrás un *óptimo local* con la intención de escalar hacia el *óptimo global*.

El peligro de este enfoque es que después de dejar un *óptimo local*, el proceso se puede ciclar y regresar al mismo *óptimo local*. Para evitar este círculo vicioso, la búsqueda tabú prohíbe en forma